WEST UNIVERSITY OF TIMIŞOARA
FACULTY OF COMPUTER SCIENCE
MASTER STUDY PROGRAM: INTELLIGENT SOFTWARE ROBOTICS

MASTER THESIS

SUPERVISOR:
Lect. Dr. Todor Ivașcu

GRADUATE:
Raluca-Bianca Biraș

TIMIŞOARA
2026

WEST UNIVERSITY OF TIMIŞOARA
FACULTY OF COMPUTER SCIENCE
MASTER STUDY PROGRAM: INTELLIGENT SOFTWARE ROBOTICS

AffectLab: Design and Evaluation of an Emotion-Aware Multimodal System for Adaptive Difficult-Conversation Rehearsal

SUPERVISOR:
Lect. Dr. Todor Ivașcu

GRADUATE:
Raluca-Bianca Biraș

TIMIŞOARA
2026

Abstract

Practising difficult interpersonal conversations can be inaccessible, inconsistent, or uncomfortable, while conventional conversational systems often ignore context, vocal cues, and uncertainty. This dissertation presents AffectLab, a non-clinical application for adaptive communication rehearsal that combines contextual text and speech emotion recognition with deterministic dialogue, feedback, safety, and fallback policies. Emotion models were evaluated on five speaker-independent IEMOCAP folds using classification, calibration, and dialogue-level bootstrap measures; the integrated application was assessed through authored cases, automated and full-stack tests, accessibility checks, restart recovery, and local latency measurements. Three-turn context increased text macro F1 from 0.6651 to 0.7332. Equal-weight text–audio fusion reached 0.7883 macro F1 and 0.7832 accuracy, improving macro F1 over contextual text by 0.0551 (95% interval 0.0407–0.0713), although its ECE of 0.1619 exposed a calibration trade-off. The application passed 245 backend tests, 76 desktop/mobile browser cases, production frontend checks, and two database-backed restart journeys; deterministic conversation turns had a 5.836 ms 95th-percentile local latency. In a supplementary single-arm pilot, 12 of 15 enrollees completed the protocol, while realism and feedback usefulness each had a median rating of 5/7. Required pilot tasks bypassed multimodal inference and affect-aware pacing, so these ratings describe only the baseline rehearsal workflow and provide no evidence about the acceptability or benefit of emotion-aware adaptation. The small, uncontrolled pilot also cannot establish causal benefit or real-world transfer. Within these limits, AffectLab demonstrates an implementation-complete, reproducibly evaluated, uncertainty-aware system whose suitability for controlled deployment depends on target-environment validation.

Keywords: affective computing; emotion recognition; multimodal fusion; conversational context; difficult-conversation rehearsal; human–computer interaction.

Statement of Scientific integrity

I declare that this MSc thesis is an original report and represents my own work, conducted under supervision of Lect. Dr. Todor Ivașcu. All research methods, data collection, analysis and interpretation presented in this thesis are accurately reported and all sources have been cited appropriately where I have made use of ideas and/or visuals of others.

I used generative AI during the writing and research process. The details are in the Appendix A.

Contents

List of Figures xi

List of Tables xiii

List of Abbreviations xv

1 Introduction 1
- 1.1 Context and Problem . . . 1
- 1.2 Aim and Objectives . . . 2
- 1.3 Research Questions . . . 2
- 1.4 Approach and Contributions . . . 3
- 1.5 Scope and Dissertation Structure . . . 4

2 State of the Art 5
- 2.1 Introduction . . . 5
- 2.2 Role-Play and Simulation for Communication-Skills Development . 5
  - 2.2.1 Role-play as a practice-based learning method . . . 5
  - 2.2.2 Simulation realism, repeatability, and transfer . . . 6
- 2.3 Feedback and Reflection . . . 7
  - 2.3.1 From conversational experience to actionable feedback . . . 7
  - 2.3.2 Reviewing alternatives instead of rewarding one script . . . 8
- 2.4 Conversational Agents for Structured and Generative Role-Play . . 8
  - 2.4.1 Deterministic dialogue management . . . 8
  - 2.4.2 Language-model simulation and its limits . . . 8
- 2.5 Affective Computing and Emotion Labels . . . 9
  - 2.5.1 Categorical and dimensional representations . . . 9
  - 2.5.2 Expression, interpretation, and uncertainty . . . 10
- 2.6 Emotion-Recognition Datasets and Protocols . . . 10
  - 2.6.1 IEMOCAP and the four-class benchmark . . . 10
  - 2.6.2 Other datasets and domain differences . . . 11
  - 2.6.3 Evaluation splits and information leakage . . . 11
- 2.7 Textual Emotion Recognition and Conversational Context . . . 12
  - 2.7.1 Pre-trained language representations . . . 12
  - 2.7.2 Speaker and dialogue-context modelling . . . 12
  - 2.7.3 Causal context and deployment compatibility . . . 13
- 2.8 Speech Emotion Recognition . . . 13
  - 2.8.1 Acoustic cues and conventional representations . . . 13
  - 2.8.2 Self-supervised speech representations . . . 14
  - 2.8.3 Operational constraints of speech input . . . 14

2.9 Multimodal Emotion Recognition and Fusion . . . 14
2.9.1 Complementarity of text and speech . . . 14
2.9.2 Decision-level fusion and its trade-offs . . . 15
2.10 Confidence Calibration, Uncertainty, and Generalisation . . . 16
2.10.1 Accuracy does not establish trustworthy probabilities . . . 16
2.10.2 Evaluating classification and calibration separately . . . 16
2.10.3 Distribution shift and restrained use of predictions . . . 17
2.11 Affective and Empathetic Conversational Agents . . . 17
2.11.1 Generating emotion-sensitive responses . . . 17
2.11.2 Bounded adaptation rather than emotion-driven control . . . 17
2.12 Trust, Safety, Privacy, and Human Oversight . . . 18
2.12.1 Boundaries between rehearsal and psychological support . . . 18
2.12.2 Data minimisation and legal context . . . 18
2.13 Model and Application Evaluation . . . 19
2.14 Synthesis of Existing Research . . . 19
2.15 Research Gap and Positioning of AffectLab . . . 21
2.16 Chapter Summary . . . 22

3 Proposed System Architecture 23
3.1 Architecture and Design Principles . . . 23
3.2 Grouped System Requirements . . . 24
3.3 Dialogue, Affect Adaptation, and Safety Boundaries . . . 25
3.4 Ethics, Privacy, and Design Limitations . . . 26

4 Implementation 29
4.1 Chapter Overview . . . 29
4.2 Technology Stack and Project Organisation . . . 30
4.2.1 Frontend Technologies . . . 30
4.2.2 Backend Technologies . . . 31
4.2.3 Persistence and External Services . . . 32
4.2.4 Repository Structure and Configuration . . . 33
4.3 Emotion-Recognition Pipeline . . . 34
4.3.1 Dataset Preparation and Label Mapping . . . 34
4.3.2 Contextual Text-Emotion Model . . . 36
4.3.3 Speech-Emotion Model . . . 37
4.3.4 Multimodal Late Fusion . . . 37
4.3.5 Confidence Calibration and Uncertainty Handling . . . 38
4.3.6 Model Artefacts, Provenance, and Inference Loading . . . 39
4.4 Backend Application . . . 40
4.4.1 Application and Service Architecture . . . 40
4.4.2 Domain Models and API Schemas . . . 41
4.4.3 Authentication and Account Management . . . 42
4.4.4 Session Persistence and Ownership Controls . . . 43
4.4.5 Conversation Request Processing . . . 43
4.4.6 Optional Services and Deterministic Fallbacks . . . 44
4.4.7 Rule-Based Cognitive Features and Support Routing . . . 45
4.5 Role-Play and Adaptation Engine . . . 46
4.5.1 Scenario Representation and Configuration . . . 46

4.5.2 Dialogue State and Transition Logic 47
4.5.3 Communication-Evidence Extraction 49
4.5.4 Affect-Aware Pacing Policy 49
4.5.5 Response Realisation 50
4.5.6 Safety Precedence and Crisis Interruption 50
4.6 Feedback and Reflection Features 51
4.6.1 Evidence-Grounded Feedback Generation 51
4.6.2 Conversation Replay 51
4.6.3 Rewind, Branching, and Comparison 52
4.6.4 Takeaways and Action Cards 52
4.7 Frontend Application 54
4.7.1 Application Interface Overview 54
4.7.2 Application Navigation and User Workflow 54
4.7.3 Conversation Preparation 58
4.7.4 Role-Play Workspace 58
4.7.5 Voice Capture and Multimodal Interaction 59
4.7.6 Presentation of Affect and Uncertainty 59
4.7.7 Feedback, Replay, and Comparison Interfaces 60
4.7.8 Optional Research Interfaces 60
4.8 Optional Research Support and Data Export 62
4.8.1 Participant Enrolment and Consent 62
4.8.2 Study Progress and Questionnaires 63
4.8.3 Pseudonymous Research Records 63
4.8.4 Research Dashboard and Data Export 64
4.9 Security, Privacy, and Data Protection 64
4.9.1 Authentication and Token Management 65
4.9.2 Authorisation and Resource Isolation 65
4.9.3 Audio and Conversation Data Handling 66
4.9.4 Retention, Withdrawal, and Account Deletion 66
4.9.5 Deployment Security Controls 66
4.10 Deployment and Configuration 67
4.10.1 Local and Containerised Deployment 67
4.10.2 Offline Demonstration Mode 68
4.10.3 Environment-Based Configuration 68
4.10.4 Health Checks and Operational Failure Handling 69
4.11 Implementation Verification 69
4.11.1 Backend Unit and Integration Testing 69
4.11.2 Frontend End-to-End Testing 70
4.11.3 Performance Regression Testing 70
4.11.4 Full-Stack Smoke Testing 71
4.11.5 Reproducibility and Model-Provenance Checks 71
4.12 Chapter Summary 72

5 Evaluation and Results **75**
5.1 Chapter Overview 75
5.2 Evaluation Scope and Evidence Hierarchy 76
5.2.1 Research Questions and Evaluation Levels 76

5.2.2 Separation of Development, Evaluation, and Deployment Evidence . . . 76
5.3 Emotion-Recognition Evaluation Methodology . . . 77
5.3.1 Benchmark Task and Speaker-Independent Splits . . . 77
5.3.2 Classification and Calibration Measures . . . 77
5.3.3 Paired Dialogue-Level Statistical Comparison . . . 78
5.4 Contextual Text-Emotion Results . . . 78
5.4.1 Primary Four-Class Comparison . . . 78
5.4.2 Exploratory Six-Class Comparison . . . 79
5.4.3 Answer to RQ1 . . . 80
5.5 Speech and Multimodal Results . . . 81
5.5.1 Audio-Only Baseline . . . 81
5.5.2 Predeclared Equal-Weight Late Fusion . . . 81
5.5.3 Qualitative Error Analysis . . . 81
5.5.4 Classification–Calibration Trade-Off . . . 82
5.5.5 Comparison with Published IEMOCAP Results . . . 88
5.5.6 Answer to RQ2 . . . 89
5.6 Application Verification and Readiness . . . 89
5.6.1 Deterministic Dialogue Cases . . . 89
5.6.2 Cognitive-Heuristic and Support-Routing Cases . . . 91
5.6.3 Evidence Extraction and Affect-Pacing Cases . . . 92
5.6.4 Automated and Full-Stack Verification Results . . . 92
5.6.5 Performance and Responsive-Interface Checks . . . 94
5.6.6 Answer to RQ3 . . . 95
5.6.7 Answer to RQ4: Integrated Application Quality . . . 95
5.6.8 Supplementary User-Pilot Results . . . 95
5.7 Threats to Validity and Limitations . . . 97
5.7.1 Model and Statistical Validity . . . 97
5.7.2 External and Construct Validity . . . 97
5.7.3 Supplementary User-Evidence Limitations . . . 98
5.8 Chapter Summary . . . 98

6 Conclusions and Future Work **101**
6.1 Chapter Overview . . . 101
6.2 Summary of the Dissertation . . . 101
6.3 Conclusions for the Research Questions . . . 102
6.3.1 RQ1: The Contribution of Conversational Context . . . 102
6.3.2 RQ2: Multimodal Complementarity and Calibration . . . 103
6.3.3 RQ3: Bounded Integration of Affective Evidence . . . 103
6.3.4 RQ4: Integrated Application Quality and Readiness . . . 104
6.4 Principal Contributions . . . 104
6.4.1 Empirical and Machine-Learning Contributions . . . 104
6.4.2 System and Interaction-Design Contributions . . . 105
6.4.3 Application-Verification and Infrastructure Contributions . . . 105
6.5 Limitations of the Conclusions . . . 106
6.5.1 Model and Data Limitations . . . 106
6.5.2 Application and Measurement Limitations . . . 106
6.5.3 Supporting User-Evidence Limitations . . . 107

6.6 Future Work 107
6.6.1 Validate Deployment Performance and Reliability 107
6.6.2 Broaden Model Validation and Improve Uncertainty Handling 108
6.6.3 Evaluate and Refine the Adaptation Policy 108
6.6.4 Validate Feedback and Communication Measures 109
6.6.5 Study Transfer, Accessibility, and Long-Term Use 109
6.6.6 Strengthen Deployment, Privacy, and Governance 110
6.7 Final Conclusion 110

Bibliography **117**

A Generative AI Tools Usage **119**
A.1 ChatGPT Details 119
A.1.1 Literature Exploration 120
A.1.2 Academic Writing Assistance 120
A.1.3 LaTeX Formatting Assistance 120
A.1.4 Technical Guidance and Debugging 120

List of Figures

2.1 Relationship between the reviewed research streams, the AffectLab integration gap, and its complementary model, application, and supporting user evidence. 21

3.1 Logical architecture of AffectLab and separation between probabilistic evidence and authoritative application control. 24

4.1 Emotion-recognition evaluation and deployment pipeline. The lower path packages a frozen configuration for inference but does not create a new held-out performance result. 34
4.2 Sequence of a version-bound conversation request, including safety precedence and failure-tolerant optional processing. 48
4.3 Principal dialogue-state progression for workload, boundary, and relationship rehearsal. 48
4.4 Decision tree for bounded affect-aware pacing. Every terminal action preserves the authoritative dialogue state and evidence. 53
4.5 Replay, rewind, and copy-on-write branching from a stored pre-turn dialogue snapshot. 53
4.6 AffectLab home dashboard, showing the available practice paths, recommended starting point, and access to recent sessions and saved reflections. 55
4.7 Active role-play workspace, combining the conversation transcript and message composer with scenario progress, session controls, and optional voice and affect-aware functions. The standard affect panel shown here is the lexical estimate; a trained voice–text result, when available, appears in a separate panel. 56
4.8 Post-rehearsal feedback and review interface, presenting evidence-linked observations and access to replay, alternative branches, personal takeaways, and the editable action card. 57
4.9 Research-data lifecycle from the richer account-owned session to the pseudonymous live record and immutable de-identified export. . . . 64
4.10 Containerised deployment topology and external-service boundary. . 68

5.1 Aggregate four-class results from independently evaluated fold configurations. Panel (a) compares macro F1, while Panel (b) shows why the highest-classification equal-weight fusion is not the preferred configuration when confidence is consumed operationally. 84

5.2 Pooled row-normalised confusion matrices over the five held-out IEMOCAP sessions (n = 5,531). Rows are true labels and columns are predicted labels; cell values are percentages of the corresponding true-label row. All panels use the same 0–100% colour scale. Every fitted parameter represented here was estimated without the corresponding test session. 84
5.3 Reliability curves from pooled held-out predictions. Each point represents a non-empty equal-width bin of maximum class probability; the dashed diagonal denotes perfect calibration. Panel (a) compares the unimodal systems with predeclared equal fusion, and Panel (b) shows the effect of validation-derived fusion calibration. ECE values in Table 5.8 use the prespecified 15-bin calculation, while the curves use ten bins for legibility. 86

List of Tables

2.1 Representative conversational datasets and implications for AffectLab. 11
2.2 Selected research approaches, demonstrated contributions, and limitations for communication rehearsal. 20

3.1 Grouped functional and non-functional requirements. 25

4.1 Principal technologies used in the AffectLab implementation. 30
4.2 IEMOCAP source-label mappings implemented for the two experimental tasks. 35
4.3 Distinction between the trained affect model and the application's deterministic analysis and decision components. 61

5.1 Mapping between research questions, evidence, and permitted conclusions. 76
5.2 Four-class contextual-text results on five held-out IEMOCAP sessions. 78
5.3 Per-class precision, recall, F1, and support for the four-class text configurations. 79
5.4 Exploratory six-class contextual-text comparison. 79
5.5 Fold variation in macro F1 for the exploratory six-class comparison. Each fold holds out one complete IEMOCAP session. 80
5.6 Per-class precision, recall, F1, and support for the exploratory six-class text configurations. 80
5.7 Predeclared equal-weight fusion compared with contextual text. . . 81
5.8 Classification and probability-quality trade-off for independently evaluated fold configurations. 82
5.9 Per-class precision, recall, F1, and support for the audio and fusion configurations. 83
5.10 Macro F1 by held-out IEMOCAP session, with unweighted variation across the five folds. 83
5.11 Operational global-chain summary; these descriptive values are not an independent held-out performance estimate. 87
5.12 Descriptive comparison with protocol-compatible four-class, session-held-out IEMOCAP results reported by Li et al. [LWC+22]. Values are percentages. 88
5.13 Compact requirement-to-evidence traceability matrix. 90
5.14 Authored enhanced-dialogue trajectory results. 91
5.15 Implemented cognitive-heuristic and support-routing checks. 91
5.16 Local single-client application latency results. 94

5.17 Supplementary pilot questionnaire results from available first required attempts. Ratings use seven-point scales; paired change is post-confidence minus pre-confidence. 96

6.1 Prioritised future-work roadmap. 107

A.1 Overview of generative AI usage 119

List of Abbreviations

AI	Artificial intelligence
API	Application programming interface
BERT	Bidirectional Encoder Representations from Transformers
BSON	Binary JSON
CI	Continuous integration
CORS	Cross-origin resource sharing
CPU	Central processing unit
CSV	Comma-separated values
DOM	Document Object Model
ECE	Expected calibration error
ERC	Emotion recognition in conversation
F1	Harmonic mean of precision and recall
GDPR	General Data Protection Regulation
GPU	Graphics processing unit
HTML	HyperText Markup Language
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
IEMOCAP	Interactive Emotional Dyadic Motion Capture
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large language model
NLL	Negative log-likelihood
P95	95th percentile

PCM	Pulse-code modulation
RQ	Research question
SD	Standard deviation
SER	Speech emotion recognition
SHA-256	Secure Hash Algorithm 256-bit
SMTP	Simple Mail Transfer Protocol
TLS	Transport Layer Security
TTL	Time to live
UA	Unweighted accuracy
UI	User interface
WA	Weighted accuracy
WAV	Waveform Audio File Format
WCAG	Web Content Accessibility Guidelines

Chapter 1

Introduction

1.1 Context and Problem

Difficult interpersonal conversations arise in ordinary situations such as negotiating workload, maintaining a personal boundary, or expressing a relationship need. Although a person may know what they want to communicate, pressure and disagreement can make it harder to state a clear request, respond constructively, or reflect on alternative wording. Structured role-play offers a way to practise these skills before a real interaction, particularly when it provides clear objectives, resistance, reflection, and feedback.

Conventional practice may require another person, a facilitator, or a scheduled and potentially uncomfortable session. A digital system can instead provide private, repeatable access and preserve the same scenario for comparison between attempts. Language models can add conversational variety, but unrestricted generation is not a sufficient foundation for dependable practice: a model may leave its assigned role, vary the intended difficulty, or produce feedback unsupported by the user's words. Scenario progression, outcomes, assessment, and safety therefore require explicit control.

A second challenge concerns affect. Meaning can depend on preceding dialogue and on acoustic delivery as well as the current transcript, which motivates contextual and multimodal emotion recognition. Yet a benchmark label is not a direct measurement of a person's internal state. Predictions can be wrong, overconfident, or inconsistent across text and audio, particularly outside the data on which the models were evaluated. An application must therefore decide not only how to infer affective information, but when that information is sufficiently reliable and appropriate to influence an interaction.

This dissertation addresses the integration problem created by these requirements: how to combine contextual text and speech models with flexible role-play while keeping dialogue outcomes, feedback, safety, privacy, and failure handling predictable. AffectLab treats affect predictions as bounded evidence for modest and reversible pacing choices, not as authoritative psychological judgements. Deterministic policies remain responsible for scenario state, observable communication features, and safety precedence, while optional generative services may realise an already constrained response. Model performance, application conformance, and supporting user feedback are evaluated separately so that evidence from one level is not used to justify claims at another.

1.2 Aim and Objectives

The aim of this dissertation is to design, implement, and evaluate AffectLab, an emotion-aware system for rehearsing difficult interpersonal conversations. The system combines structured role-play, contextual text and audio-based affect recognition, uncertainty-aware adaptation, and evidence-grounded feedback while preserving transparency, user control, privacy, safety, and deterministic fallback behaviour.

The objectives are:

1. to establish, from relevant research, the design constraints for structured communication rehearsal, contextual and multimodal affect recognition, calibrated uncertainty, and responsible human–AI interaction;
2. to develop and compare utterance-only text, contextual text, audio, and late-fusion models under a speaker-independent protocol, evaluating classification and probability quality as distinct concerns;
3. to design and implement scenario-specific dialogue policies and a bounded adaptation policy in which model confidence, modality agreement, availability, user preference, and safety determine only permitted, inspectable actions;
4. to provide evidence-linked feedback and reflective functions within a privacy- and safety-conscious application whose core workflow survives optional-service failure; and
5. to evaluate model behaviour and integrated application quality with evidence appropriate to each claim, while reporting the small user pilot only within its non-adaptive scope.

1.3 Research Questions

The dissertation is guided by four research questions:

RQ1. Does incorporating preceding conversational turns improve text-based emotion recognition compared with classifying each utterance independently?

RQ2. Does combining contextual textual information with acoustic information improve emotion-classification performance compared with the individual text and audio modalities, and what trade-off exists between classification performance and confidence calibration?

RQ3. How can uncertain multimodal affect predictions be integrated into a structured role-play dialogue policy while keeping the system's behaviour explainable, controllable, safe, and robust to disagreement or inference failure?

RQ4. To what extent does the completed AffectLab application satisfy its functional, reliability, recovery, accessibility, responsiveness, and deployment requirements under reproducible engineering evaluation?

1.4 Approach and Contributions

AffectLab uses a layered architecture that separates affect inference, dialogue control, response realisation, feedback, and safety. The experimental pipeline compares an utterance-only text classifier with a model that adds preceding causal dialogue turns. A separate speech model provides acoustic evidence, and late fusion combines the modality distributions. Classification metrics establish predictive performance, while calibration measures assess whether confidence is suitable for operational thresholds. Speaker-independent session folds and paired dialogue-level comparisons keep the principal model comparisons aligned.

Within the application, a deterministic controller represents each scenario as stages, character behaviour, completion conditions, and unresolved outcomes. It selects the next conversational action from observable evidence rather than allowing a language model to decide whether the user has succeeded. An optional generative service may phrase that action more naturally, but validation and deterministic templates preserve the recorded decision when generation is unavailable or unsuitable.

Multimodal estimates enter the interaction through a restricted affect policy. Confidence, agreement between modalities, inference availability, eligibility, and an explicit user preference determine whether the system preserves baseline behaviour, offers a pacing choice, or permits a brief acknowledgement. Safety interruption takes precedence and does not depend on the emotion classifier. This separation also prevents affect predictions from determining scenario success, feedback scores, or participation decisions.

Feedback is derived from inspectable language features associated with each scenario, such as a clear request, maintained boundary, non-blaming statement, acknowledgement, or practical next step. The relevant conversational turns remain available as evidence. Replay, retry, branching, comparison, personal takeaways, and action cards support reflection on alternative responses rather than reducing practice to a single score.

Evaluation follows the same separation of responsibilities. The model study addresses context, modality complementarity, and calibration. Authored policy cases and automated checks address dialogue transitions, fallbacks, persistence, recovery, accessibility, responsive behaviour, and the complete application workflow. Local timing measurements characterise only the tested deterministic path. A small pilot supplies supplementary feedback on the baseline rehearsal workflow; because its required tasks did not exercise multimodal adaptation, it is not evidence that adaptation benefits users.

The dissertation makes five principal contributions:

1. **A controlled evaluation of causal conversational context and multimodal complementarity.** Utterance-only text, contextual text, speech, and fused configurations are compared on matched speaker-independent data, with dialogue-level uncertainty estimates.
2. **An explicit separation between classification performance and operational confidence.** Predeclared fusion and validation-fitted calibration expose the trade-off between the strongest classifier and the probability estimates used by an interactive policy.

3. **A bounded architecture for affect-aware role-play.** Deterministic dialogue and safety logic remain authoritative, while uncertain affect estimates and optional generation are limited to inspectable, reversible actions with defined fallbacks.
4. **Evidence-grounded rehearsal and reflection.** Scenario-specific outcomes and feedback are connected to observable turns, and replay and branching allow alternative responses to be compared from the same conversational context.
5. **A layered verification framework for an integrated application.** Model evaluation, policy conformance, application verification, local performance, and supplementary user feedback are kept distinct, making the scope of each resulting claim explicit.

Together, these contributions concern the responsible integration of affective models into an implemented communication-practice system. They do not imply that AffectLab identifies a user's true emotion or that completing a rehearsal improves a real relationship.

1.5 Scope and Dissertation Structure

AffectLab is intended for adults rehearsing ordinary difficult conversations. The implemented scenarios cover workload negotiation, personal boundary-setting, and expression of a relationship need. Text interaction is primary and voice input is optional. The affective component is limited to supervised categorical recognition from text and audio; it excludes video, physiological signals, multilingual inference, diagnosis, therapy, autonomous crisis assessment, and consequential decisions about another person. Emotion estimates are uncertain application inputs, not findings about an individual's internal state.

The empirical model claims are limited to the defined IEMOCAP protocol. Its ten actors, controlled English-language interactions, annotation process, and categorical labels do not establish generalisation across cultures, accents, devices, populations, or spontaneous application use [BBL+08]. Deterministic language rules are reproducible but cannot recognise every appropriate paraphrase and are not validated measures of general communication ability or psychological constructs. Optional providers introduce additional privacy and reliability boundaries, so the text-based fallback remains part of the intended system rather than an exceptional mode.

Application evaluation establishes behaviour only for the tested cases and configurations. It does not replace independent security review, target-environment load and recovery testing, live-provider evaluation, participatory accessibility work, or validation of trained inference under deployment conditions. The small, uncontrolled pilot concerns the non-adaptive baseline workflow and cannot establish causal benefit, sustained skill development, or transfer to real conversations. AffectLab is therefore presented as an implementation-complete, deployment-oriented research application whose wider use remains conditional on further technical, ethical, and human evaluation.

Chapters 2–6 cover the literature, architecture, implementation, evaluation, and conclusions, respectively.

Chapter 2

State of the Art

2.1 Introduction

The design of an emotion-aware communication-rehearsal system lies at the intersection of communication-skills education, conversational artificial intelligence, affective computing, and human–computer interaction. Progress in any one of these areas does not, by itself, establish that an integrated application will be useful, reliable, or safe. A classifier may perform well on annotated recordings without being reliable for a new speaker; a dialogue generator may produce plausible replies without consistently following a scenario; and a realistic simulation may be enjoyable without producing a measurable learning benefit. A state-of-the-art review must therefore examine both the capabilities of existing techniques and the evidence that supports their intended use.

This chapter first reviews role-play, digital simulation, feedback, and the design of conversational agents. It then examines the theoretical and empirical foundations of affect recognition, with particular attention to the IEMOCAP benchmark, emotion recognition in conversation, contextual text modelling, speech-based methods, and multimodal fusion. The subsequent sections discuss uncertainty calibration, affect-conditioned dialogue, generative systems, interpretability, safety, privacy, and evaluation. Finally, the literature is synthesised to identify what has already been demonstrated, which questions remain open, and how the proposed AffectLab system is positioned with respect to prior work. Throughout, a distinction is maintained between *predicting a dataset label*, *controlling an application*, and *improving a person's communication skills*. These are different tasks and require different forms of evidence.

2.2 Role-Play and Simulation for Communication-Skills Development

2.2.1 Role-play as a practice-based learning method

Interpersonal communication requires more than knowledge of recommended expressions or communication rules. A speaker must formulate a message, respond to objections, clarify intentions, and sometimes sustain a position without controlling the other person's reaction. Role-play makes these decisions observable by placing

the learner in a simulated exchange with an interlocutor. It permits errors and alternative responses in a setting with lower consequences than the corresponding real-world interaction. However, the educational value of role-play is not inherent in the mere presence of a simulation: the target skills, the quality of the interaction, and the opportunity to receive and use feedback also matter.

Evidence for structured communication rehearsal is available in professional education. Bosse et al. conducted a randomised controlled trial with 103 medical students assigned to peer role-play, training with standardised patients, or a control group. Both active training approaches improved measures of communication performance and self-efficacy relative to the control condition, with some outcomes favouring peer role-play [BSN+12]. The findings support the value of active and repeatable practice, but the participants, clinical tasks, and assessment environment were specific. They do not establish that a software-based rehearsal system will produce equivalent improvements in everyday workplace or relationship conversations.

A systematic review by Lee et al. examined 14 studies of virtual-patient systems for medical communication training [LKK+20]. The review associated more effective systems with sound instructional features, such as preparation, scaffolding, debriefing, reflection, and feedback, rather than with technological sophistication alone. Importantly, the evidence was heterogeneous, and only some comparative evaluations found significant advantages for virtual patients. The review also distinguished human feedback from automated feedback; it did not establish that automatically generated comments are sufficient to produce learning. These qualifications matter for the design of non-clinical practice tools because conversational realism must not be confused with educational effectiveness.

2.2.2 Simulation realism, repeatability, and transfer

Digital simulations offer properties that conventional role-play cannot always guarantee: the same initial situation can be replayed, different responses can be tried from a common starting point, and an interaction can be reviewed without the presence of another person. Consistent scenarios also permit defined learning objectives and repeatable assessment. Their limitations are equally important. Simulated interlocutors cannot reproduce every social cue, personal history, consequence, or unpredictable response of real people. A user may learn to satisfy the rules of the simulation rather than improve performance in an unfamiliar situation.

A particularly relevant contemporary system is *IMBUE*, proposed by Lin et al., which uses language models to simulate personalised interpersonal situations and provide just-in-time feedback based on an established interpersonal-effectiveness framework [LSR+24]. In a randomised study involving 86 participants, the authors reported improvements in measures including self-efficacy, negative emotion, and communication-skill mastery. They also found that transfer to new and more difficult situations was more limited than improvements within the trained situations. IMBUE provides evidence that language-model-based rehearsal can be organised as a learning activity rather than as an open-ended chatbot, but its results depend on the specific instructional framework, outcome measures, and study design. They should not be interpreted as proof of general or long-term transfer for unrelated applications.

Taken together, these studies support a practice environment with clear scenarios, observable communication goals, structured resistance, multiple attempts, and opportunities for reflection [BSN+12, LKK+20, LSR+24]. For AffectLab, this literature motivates separate scenario definitions for workload negotiation, personal boundary-setting, and the expression of relationship needs. It also suggests that successful rehearsal should not be equated with persuading the simulated character: stating and maintaining a boundary, agreeing on a feasible next step, or documenting an unresolved disagreement may all be legitimate conversational outcomes.

2.3 Feedback, Reflection, and Evidence in Rehearsal Systems

2.3.1 From conversational experience to actionable feedback

Practice becomes more useful when users can relate an assessment to something they actually said and identify a feasible improvement for their next attempt. Hattie and Timperley's synthesis of educational feedback research distinguishes several functions of feedback, including clarification of the learning goal, information about current performance, and guidance about what to do next [HT07]. This provides a useful rationale for moving beyond a generic score or an approving response. In a communication rehearsal, feedback can refer to explicit behaviours, such as stating a concrete request, giving a relevant reason, using non-accusatory language, acknowledging another perspective, or maintaining an expressed boundary.

An important design distinction concerns *feedback evidence* and *feedback presentation*. Evidence may be extracted using observable language features, annotations, or human assessment, whereas presentation determines how that evidence is explained to the learner. IMBUE combines domain-informed assessment and just-in-time guidance within its training process [LSR+24]. By contrast, the virtual-patient review by Lee et al. found stronger support for the overall instructional design, including guided reflection and human facilitation, than for the intrinsic effectiveness of automated feedback [LKK+20]. Consequently, an application should not assume that fluent or detailed comments generated by a language model are necessarily valid.

For automated rehearsal systems, this motivates traceable assessments in which each recorded feature points to the turn that activated it. If a message is credited for providing a specific request, the system should identify the relevant text rather than merely assert that the user was assertive. Rule-based detection makes these decisions relatively reproducible, but it is not synonymous with semantic understanding. A rule can miss a perfectly acceptable paraphrase or incorrectly detect a feature in negated text, quotation, sarcasm, or reported speech. Transparent feedback therefore requires both inspectable evidence and explicit acknowledgement that its measures are partial proxies for communication behaviour, not validated measures of an individual's competence.

2.3.2 Reviewing alternatives instead of rewarding one script

Replay, retry, and branch-based review allow users to examine the consequences of a wording choice while preserving the common conversational context. These mechanisms are consistent with the emphasis on reflection and post-simulation activities in effective virtual-patient designs [LKK+20]. Comparing alternative continuations can highlight differences between a vague request and a specific one, or between a boundary that is repeated and one that is abandoned under resistance. This remains an instructional design rationale rather than evidence that branching, by itself, causes real-world improvement.

The evaluation of such tools should also avoid a narrow definition of success. A simulation with deterministic scoring can inadvertently train users to repeat the phrases that trigger its rules. Accordingly, evidence about improvement inside a system must be separated from assessment by independent human raters or behaviour in new settings. This distinction also limits what can be inferred from changes in turn counts, rule-based scores, or self-reported confidence [HT07, LSR+24].

2.4 Conversational Agents for Structured and Generative Role-Play

2.4.1 Deterministic dialogue management

Classical task-oriented dialogue systems use explicit representations of conversational state and action selection. Their main advantage is not that they reproduce spontaneous conversation perfectly, but that state transitions and permissible outcomes can be specified and tested. For example, the RavenClaw framework described by Bohus and Rudnicky separates domain-specific task logic from reusable dialogue-management capabilities such as error handling and turn-taking [BR09]. This separation provides a precedent for keeping dialogue control distinct from the surface wording of an utterance.

In communication rehearsal, explicit dialogue states can represent stages such as opening an issue, clarifying a concern, responding to resistance, and agreeing on a next step. Scenario-specific rules can define both completed and unresolved outcomes. Such a design facilitates regression tests and makes it possible to explain why a scenario advanced or terminated. Its disadvantages include the cost of authoring policies and the difficulty of anticipating diverse linguistic expressions. A fully templated system may also appear repetitive. These limitations create a reasonable motivation for adding generative wording while retaining explicit state transitions.

2.4.2 Language-model simulation and its limits

Large language models extend conversational flexibility because they can paraphrase, preserve aspects of a character profile, and respond to inputs not anticipated in a fixed script. IMBUE demonstrates one use of this capability for personalised interpersonal-skills training [LSR+24]. Tamoyan et al. subsequently investigated

LLM-based role-play as a method for simulating multi-turn human–chatbot interaction, including persona construction, dialogue goals, and recovery from conversational failures [TSG25]. These studies are useful evidence that explicit roles and goals can structure generation, even though their evaluation tasks are not identical to everyday communication rehearsal.

Generative flexibility comes with risks. Lu et al. compared LLM-generated with human-authored responses in multi-turn professional role-play dialogues and reported that, in their evaluated setting, human judgements of generated-response quality deteriorated across turns, particularly for naturalness and context maintenance [LJG25]. This result cautions against evaluating a role-playing agent using only isolated responses. A simulated character may respond convincingly at the beginning of an exchange yet lose track of a constraint or become overly accommodating later. Likewise, a response that sounds supportive can undermine the exercise if it removes intended resistance or declares a difficult situation resolved prematurely.

The literature thus suggests a distinction between a *dialogue decision* and its *linguistic realisation*. In a hybrid design, the application determines the current stage, whether a communication feature has been observed, which outcomes are allowed, and how strongly the simulated character should resist. Optional generation then realises a bounded response plan; the generated text is checked against the plan and can be replaced by a deterministic template when necessary. This design is motivated by dialogue-management principles and observed weaknesses of multi-turn generation, rather than by a claim that prompting can eliminate all generative failures [BR09, LJG25].

2.5 Affective Computing and the Meaning of Emotion Labels

2.5.1 Categorical and dimensional representations

Affective computing investigates computational systems that model or respond to affect-related signals, but the underlying concept of emotion is not universally defined. Ekman's work argues for distinguishing a set of basic emotions with characteristic features [Ekm92]. An alternative tradition represents affect using continuous dimensions. Russell's circumplex model, for example, describes affective concepts in terms of related dimensions conventionally interpreted as pleasantness or valence and activation or arousal [Rus80]. Scherer emphasises the importance of distinguishing emotion from related constructs such as mood, feeling, and other affective processes, and of making the operational definition explicit [Sch05].

These approaches lead to different machine-learning targets. A categorical classifier may predict *anger*, *happiness*, *sadness*, or *neutral*, whereas a dimensional predictor may estimate valence and arousal. Neither representation captures every relevant aspect of lived emotional experience. Categorical labels are convenient for benchmarking and error analysis, but require a finite label vocabulary, sometimes collapse distinct experiences into one class, and may impose a single answer on an ambiguous utterance. Dimensional representations can express degrees of affect, but introduce their own annotation and interpretation difficulties. Therefore, se-

lecting a classification taxonomy is a modelling decision, not a demonstration that emotion naturally consists of the selected labels.

2.5.2 Expression, interpretation, and uncertainty

The observable relationship between an expression and a person's internal state is imperfect. Barrett et al. reviewed evidence challenging a simple one-to-one inference from facial movements to discrete emotional states [BAM+19b]. Their analysis specifically concerns faces and should not be misrepresented as an experimental result about text or speech. Nevertheless, its broader methodological warning is pertinent: observed behaviour and annotator interpretations are not direct access to subjective experience. Linguistic statements and vocal characteristics can likewise be shaped by social setting, personal speaking style, acting, irony, and the demands of a particular conversation [Sch05].

For this reason, the term *emotion recognition* is used in the conventional machine-learning sense of predicting operational labels, not as proof that a system understands the individual. The result of a classifier can be useful as a contextual signal even when its label is uncertain, provided that its effects remain modest and reversible. An application that uses uncertain predictions to diagnose distress, infer intention, or judge a person's competence would require a substantially stronger evidential basis than ordinary benchmark classification.

2.6 Datasets and Evaluation Protocols for Emotion Recognition

2.6.1 IEMOCAP and the four-class benchmark

The Interactive Emotional Dyadic Motion Capture database (IEMOCAP) was introduced by Busso et al. to study expressive communication through multiple channels [BBL+08]. It contains approximately twelve hours of scripted and improvised dyadic interactions by ten actors arranged into five recording sessions, together with textual, acoustic, visual, and annotation information. Because participants engage in exchanges rather than isolated monologues, the corpus permits investigations of conversational context and of how speech and text complement each other. Its availability has made it a common reference point for text-only, audio-only, and multimodal emotion-recognition work.

IEMOCAP experiments do not all use an identical classification problem. A frequently adopted configuration includes four categories: anger, happiness, sadness, and neutral. In this configuration, *excitement* may be merged into *happiness* [LWC+22]. Papers may instead use different subsets, annotation aggregation rules, segmentation procedures, or label mappings. Such differences can change class prevalence and reported performance. Consequently, results should only be compared quantitatively when the data, labels, folds, and metrics are compatible. The primary experiments in this dissertation adopt the four-class definition specified in Chapter 1; the exact label mapping and preprocessing are documented in Chapter 4.

2.6.2 Other datasets and domain differences

MELD was introduced by Poria et al. as a multimodal, multi-party conversational dataset derived from the television programme *Friends* [PHM+19]. It contains approximately 13,000 annotated utterances from 1,433 dialogues and includes textual, acoustic, and visual information. Unlike IEMOCAP's paired actor sessions, MELD contains multi-speaker television interactions, with different production, editing, and linguistic characteristics. DailyDialog, developed by Li et al., instead consists of human-written everyday dialogues with manually assigned emotion and communicative-intention labels [LSS+17]. It is useful for text-based conversational studies, but it does not provide a directly equivalent speech-recording benchmark.

Table 2.1: Representative conversational datasets and implications for AffectLab.

Corpus	**Main characteristics**	**Relevance and limitations**
IEMOCAP [BBL+08]	Five dyadic sessions with ten actors; dialogue transcripts, recorded speech, additional modalities, and emotion annotations.	Supports matched text–audio experiments and held-out-session testing; acted/improvised English speech and a small speaker pool constrain generalisation.
MELD [PHM+19]	Multi-party television dialogues with text, audio, visual information, and emotion labels.	Useful for studying multi-speaker context; sitcom interactions, production conditions, and label definitions differ from IEMOCAP.
DailyDialog [LSS+17]	Human-written multi-turn everyday dialogues labelled for emotion and communicative intention.	Relevant to dialogue semantics and context but cannot support the same matched audio–text inference task.

A benchmark corpus should be evaluated according to the claims it can support. High accuracy on acted English-language interactions does not demonstrate equivalent performance with spontaneous speech, different microphones, background noise, unfamiliar accents, other languages, or speakers whose expressive styles differ from the training actors. These discrepancies are especially consequential when an application makes use of a model's confidence to change its behaviour. Although additional datasets could be valuable for external validation, incompatible corpora cannot simply be pooled without addressing annotation and domain differences [BBL+08, PHM+19, LSS+17].

2.6.3 Evaluation splits and information leakage

Conversational emotion-recognition samples are not independent in the same way as unrelated text documents. Consecutive utterances share speakers, recording conditions, scenario content, and emotional trajectories. Randomly assigning individual utterances to training and test sets can create substantial overlap in the information observed during development and evaluation. IEMOCAP's five sessions enable a more conservative protocol in which a complete session is held out and the remaining sessions are used for training and validation. Li et al. explicitly evaluate their model under a five-fold, leave-one-session-out configuration, as well as a separate Session 5 setting [LWC+22].

Holding out a session limits direct overlap between training and test speakers, but it does not guarantee generalisation beyond IEMOCAP. The model may still benefit from the same recording studio, language, task structure, and annotation conventions. Validation and hyperparameter selection must also respect group boundaries, and conversational features for a held-out utterance must not contain annotation labels or future information unavailable at the moment of inference. Such precautions are central to a meaningful comparison of utterance-only, contextual, and multimodal systems.

2.7 Textual Emotion Recognition and Conversational Context

2.7.1 Pre-trained language representations

Textual emotion classification has progressed from handcrafted lexical indicators towards contextual neural representations. BERT demonstrates how a transformer can learn general-purpose bidirectional language representations through pre-training and then be adapted to downstream classification tasks [DCLT19]. RoBERTa revisits the training procedure and scale of BERT, showing that pre-training choices strongly influence performance [LOG+19]. For affective tasks, such encoders offer representations of sentence meaning and local word context that are more flexible than matching isolated emotional keywords. Their representations are nevertheless optimised for the training objectives and examples they have encountered; apparent emotional understanding must be assessed through downstream labels and held-out cases.

An utterance-only classifier predicts a label from the current utterance u_t:

$$\hat{y}_t = f(u_t). \tag{2.1}$$

This formulation is reproducible and computationally convenient, but an utterance such as "That's fine" can express agreement, disappointment, anger, or sarcasm depending on earlier dialogue and delivery. An isolated classifier cannot infer those relationships from the transcript alone. This motivates emotion recognition in conversation (ERC), where the sequence and speakers of the preceding utterances are treated as relevant context.

2.7.2 Speaker and dialogue-context modelling

DialogueRNN, proposed by Majumder et al., models the evolving states of the speakers and uses attention over the conversation for emotion classification [MPH+19]. DialogueGCN, proposed by Ghosal et al., represents utterances and intra- and inter-speaker relations using graph convolution, allowing information to propagate between connected turns [GMP+19]. COSMIC extends context modelling by incorporating commonsense representations of events, mental states, and social interactions [GMG+20]. These studies demonstrate that the relevant unit of analysis is frequently a sequence of social exchanges rather than an isolated text string.

CoMPM, developed by Lee and Lee, combines a context model with representations derived from a speaker's preceding utterances, avoiding dependence on a structured commonsense knowledge base [LL22]. This work is particularly relevant to live interaction because it focuses on information available from conversational history. More sophisticated contextual architectures can represent additional relations, but their extra components may impose larger training, inference, and reproducibility costs. For an application-oriented dissertation, a simpler controlled comparison between context-free and context-aware representations may therefore provide a more interpretable empirical question than an attempt to reproduce every contemporary ERC architecture.

2.7.3 Causal context and deployment compatibility

Not every offline ERC configuration is directly usable in an interactive agent. A model that attends to the full dialogue, including utterances after the target turn, may be valid for retrospective transcript annotation but cannot be deployed unchanged to classify a live user's current message. Similarly, a pipeline that provides the gold emotion labels of neighbouring utterances during testing would rely on information unavailable to the application. For real-time adaptation, the model should use only the current message and permitted history:

$$\hat{y}_t = f(u_t, u_{t-1}, \ldots, u_{t-k}), \tag{2.2}$$

where k is the context-window length and no future user or character turns are visible. Previous utterances can be separated by role markers, preserving the distinction between the user and simulated character. It is important that a bidirectional *token encoder* such as BERT not be confused with bidirectional access to the *conversation timeline*: the encoder can attend to all tokens inside a causally assembled input while the input still excludes future turns.

The literature establishes that context can be useful, but its specific contribution depends on how context is selected, which model is trained, and how the experiment is split [MPH+19, GMP+19, LL22]. For AffectLab, this motivates a matched evaluation of utterance-only and history-conditioned text models with the same session-held-out test items. The aim is to measure a reproducible difference rather than to claim that all contextual methods must outperform simpler approaches.

2.8 Speech Emotion Recognition

2.8.1 Acoustic cues and conventional representations

Speech conveys properties that transcripts discard, including pitch movement, timing, intensity, speaking rate, vocal quality, and pauses. These may provide evidence about expressive delivery, although they are also affected by individual voice characteristics, linguistic content, microphone distance, health, and environment. The survey by El Ayadi et al. identifies feature selection, classification strategies, and database design as core problems in speech emotion recognition (SER) [EAKK11]. This perspective is important because improving the architecture alone cannot remove measurement problems in the recording or weaknesses in the labels.

The Geneva Minimalistic Acoustic Parameter Set (GeMAPS) was proposed by Eyben et al. to standardise a compact group of theoretically motivated descriptors for affective voice research [ESS+16]. Such handcrafted descriptors provide interpretable baseline features and can be suitable for smaller datasets. However, their predefined statistics may not capture all temporal and contextual properties relevant to a classification task, prompting the development of learned representations based on large-scale speech pre-training.

2.8.2 Self-supervised speech representations

Self-supervised learning makes it possible to develop acoustic representations using substantial amounts of unlabelled or weakly labelled speech. wav2vec 2.0 learns speech representations using masking and contrastive prediction in a latent representation space [BZMA20]. HuBERT instead uses masked prediction of hidden units produced by an iterative clustering process [HBT+21]. WavLM extends speech pre-training to support a wider range of tasks through denoising and representation learning that capture multiple properties of the signal [CWC+22]. These papers establish the general value of pre-trained speech representations; their original improvements are not all results on emotion classification, so reported speech-recognition error rates must not be presented as SER accuracy.

For an emotion classifier, learned acoustic representations can be pooled or otherwise summarised before applying a task-specific classification head. The choice to freeze or fine-tune the encoder depends on data quantity, available compute, and overfitting risk. Li et al. provide a more directly relevant affective example by combining pre-trained textual and speech representations on IEMOCAP and examining how feature adaptation changes performance [LWC+22]. Even with strong pre-training, an SER model may learn speaker identity, lexical content, or recording artefacts correlated with benchmark labels. Held-out-speaker evaluation and careful interpretation are therefore essential.

2.8.3 Operational constraints of speech input

Audio has additional application-level costs that may not appear in offline classification results. Microphone quality, permissions, ambient noise, sampling-rate conversion, silence, and short or incomplete recordings all affect the inference pathway. Voice recordings may also contain identifying information. A research application should therefore process audio only when the user explicitly chooses it, minimise retained raw recordings, and make unsuccessful audio inference a recoverable condition. For AffectLab, speech provides a supplementary signal rather than a requirement for completing a text-based rehearsal; the implementation of this behaviour is described in Chapter 4.

2.9 Multimodal Emotion Recognition and Fusion

2.9.1 Complementarity of text and speech

Text and speech are not interchangeable observations. A transcript can clarify what a user explicitly says, while acoustic features may capture emphasis, tension,

or delivery. Combining the two is a multimodal learning problem because their representations have different time scales, noise patterns, and failure modes. Baltrušaitis et al. classify multimodal challenges in terms of representation, alignment, fusion, and related processes, showing why the task is more complex than concatenating measurements [BAM19a]. In particular, acoustic frames and word tokens do not automatically correspond one-to-one.

Broadly, *early* or feature-level fusion combines modality representations before classification, while *late* or decision-level fusion combines outputs from separately trained classifiers. Intermediate cross-modal architectures permit richer interactions between modalities. The Multimodal Transformer (MulT) developed by Tsai et al. applies directional cross-modal attention to potentially unaligned multimodal sequences [TBL+19]. Li et al. apply context-aware pre-trained representations and investigate several fusion approaches for IEMOCAP, obtaining reported weighted accuracies of 84.45% in a Session 5 evaluation and 80.36% under their five-fold cross-validation configuration [LWC+22]. These are results under the authors' particular protocols, not performance guarantees for other implementations or datasets.

2.9.2 Decision-level fusion and its trade-offs

A decision-level approach retains two independent class-probability distributions, $p_T(y)$ for the text model and $p_A(y)$ for the audio model. A simple mixture is

$$p_F(y) = \lambda p_T(y) + (1 - \lambda)p_A(y), \qquad 0 \leq \lambda \leq 1, \tag{2.3}$$

where λ determines the relative contribution of text. Equal-weight fusion fixes $\lambda = 0.5$; a fitted configuration estimates the weight from appropriate validation data. The resulting prediction can be inspected alongside both modality outputs. This is useful for determining whether a change in classification is attributable to text, audio, or their combination, and for recognising disagreement between the predictors.

Decision-level fusion is less expressive than cross-modal attention because one classifier does not directly inspect low-level features from the other modality. Its advantages are modularity, relatively modest computational coupling, explicit missing-modality handling, and an auditable fusion rule. It also avoids the need for word-to-frame alignment at fusion time. These properties suit an application whose primary model question is whether adding audio improves over a contextual text model, rather than whether a particular complex cross-modal architecture establishes a new benchmark [BAM19a, TBL+19, LWC+22].

The limitations remain important. A fused model may be confidently incorrect if both modalities rely on the same misleading cue. It can also underperform the stronger modality when one source is especially noisy or domain-shifted. Weights chosen on the test set inflate apparent performance, while weights fitted on validation data may not transfer to new microphones or speakers. Therefore, the fusion method and its selection procedure must be predeclared and evaluated separately from the final held-out test data.

2.10 Confidence Calibration, Uncertainty, and Generalisation

2.10.1 Accuracy does not establish trustworthy probabilities

A classifier's predicted probability is not automatically a calibrated statement about correctness. For example, among samples assigned confidence near 0.8, a well-calibrated model should be correct approximately 80% of the time under the evaluated distribution. Guo et al. found that modern neural networks often produce miscalibrated confidence estimates and showed that temperature scaling is a practical post-processing approach [GPSW17]. Temperature scaling transforms classifier logits using a positive validation-fitted temperature before applying softmax; it changes confidence magnitudes while preserving the top-class ranking for a given model under the usual single-temperature formulation.

For a model with logit vector z and temperature $T > 0$, the calibrated class probability is

$$p_T(y = c \mid z) = \frac{\exp(z_c/T)}{\sum_{j=1}^{C} \exp(z_j/T)}. \tag{2.4}$$

Temperature scaling is relatively simple, but other approaches are possible. Kull et al. introduced Dirichlet calibration as a more flexible multiclass mapping and compared it using metrics including log-loss and the Brier score [KPNK$^+$19]. More flexible calibration models may require more validation data and can overfit, so complexity is not necessarily beneficial for a small, speaker-grouped dataset.

2.10.2 Evaluating classification and calibration separately

Classification measures describe the labels predicted; calibration measures describe the probabilities used to express confidence. Accuracy can obscure class imbalance, and a high weighted score can coexist with weak recognition of a less frequent emotion. Macro-averaged F1, per-class precision and recall, confusion matrices, and support counts are therefore useful complements. Probability quality can be examined using negative log-likelihood, the multiclass Brier score, and reliability diagrams. Expected calibration error (ECE) summarises differences between binned confidence and observed accuracy, but depends on the chosen bins and does not completely characterise multiclass calibration [GPSW17, KPNK$^+$19].

The distinction is especially relevant to fusion. Equal weighting may obtain the best classification score while a validation-fitted or post-calibrated configuration produces more meaningful confidence estimates. Selecting whichever configuration scores highest on the final test set would invalidate the comparison. Instead, training, model selection, calibration fitting, and testing should remain separate within each session-held-out experiment. Differences should be accompanied by uncertainty estimates and class-sensitive analysis rather than presented only as a single benchmark number.

2.10.3 Distribution shift and restrained use of predictions

Calibration is a property of a model relative to a data distribution. Ovadia et al. demonstrated that methods that produce useful probabilities under ordinary benchmark conditions can deteriorate when the input distribution changes [OFR+19]. This is directly relevant when moving from studio-recorded English conversations to speech captured through personal devices. Even a calibrated IEMOCAP probability must not be interpreted as a guarantee that a user's private emotional experience has been correctly identified.

For an adaptive application, a threshold on maximum probability should consequently be viewed as an operational policy choice rather than a validated psychological boundary. Agreement between text and audio can be informative but is not proof of correctness. A suitable conservative policy may default to baseline dialogue, offer the user an explicit pacing choice, or suppress automated emotional claims when inference is uncertain, conflicting, or unavailable. Importantly, such policies can be evaluated for predictable behaviour even without establishing that the classifier correctly identifies the user's actual feelings [GPSW17, OFR+19].

2.11 Affective and Empathetic Conversational Agents

2.11.1 Generating emotion-sensitive responses

Prior work on affective conversational agents has studied both the expression of emotion and responses to another speaker's expressed experiences. Zhou et al. proposed an Emotional Chatting Machine that conditions response generation on emotional categories and mechanisms representing emotional state and vocabulary [ZHZ+18]. Rashkin et al. introduced the EmpatheticDialogues dataset and found that models trained with emotionally situated conversations were judged more empathetic than comparison systems trained only on broad conversational data [RSLB19]. These contributions show that affect-related signals can shape dialogue-generation objectives and perceived response quality.

However, generating an apparently empathetic response is not identical to correctly recognising a speaker's emotional state, and neither result establishes safe adaptation in a learning system. Empathy can be perceived through wording even when the model's assumptions are wrong. Furthermore, in a role-play exercise the simulated character's purpose is not always to be maximally agreeable or reassuring: a productive practice scenario may involve scepticism, delay, or refusal. Therefore, an affective mechanism must be subordinated to the scenario's instructional purpose and the user's control over the interaction.

2.11.2 Bounded adaptation rather than emotion-driven control

AffectLab adopts an application-oriented interpretation of emotion awareness. Model predictions can provide context for a small set of allowable adjustments, such as optional acknowledgement or a choice to slow the exchange. They do not

define what the user really feels, determine character compliance, or modify success and safety criteria. If modalities disagree or confidence is low, asking the user to choose a pace is less presumptive than asserting an emotional state. If inference fails, the normal dialogue remains available. This division of responsibilities is informed by the uncertainty literature and the distinction between affective response generation and validated educational action [OFR+19, RSLB19, LSR+24].

This approach also limits the meaning of *adaptivity*. A system may be adaptive because it changes an interface decision in response to input, but this does not automatically make the change beneficial. A valid engineering evaluation can establish that the intended branch was selected under specified inputs. Demonstrating educational benefit from affect adaptation would require a controlled user study comparing otherwise similar adaptive and non-adaptive versions; the small supporting pilot cannot, on its own, establish that causal effect.

2.12 Trust, Safety, Privacy, and Human Oversight

2.12.1 Boundaries between rehearsal and psychological support

Difficult interpersonal conversations may contain disappointment, conflict, or serious distress. This makes the interpretation and presentation of automated responses particularly consequential. De Freitas and Cohen describe health risks associated with generative AI wellness applications, including occasions when users bring crisis-related concerns to systems not adequately prepared to respond [DFC24]. AffectLab is not presented as a mental-health intervention, and these risks do not imply that rehearsal and treatment are equivalent. They do, however, justify clear scope limits, emergency-service guidance when appropriate, and independence between safety decisions and optional probabilistic emotion inference.

Separating safety handling from the classifier prevents a four-category benchmark model from becoming an unvalidated crisis detector. Similarly, a generator should not be allowed to override a safety interruption or invent authoritative assessments of a participant. Independently inspectable dialogue controls, deterministic fallbacks, and the ability to pause or finish a scenario are therefore part of responsible system design rather than merely conveniences. The need for such boundaries is reinforced by concerns over long-running generated role-play and context preservation [LJG25, DFC24].

2.12.2 Data minimisation and legal context

Voice recordings and personal conversations can reveal identifying or sensitive information beyond the narrow task of emotion classification. Appropriate safeguards include informed participation, explicit microphone activation, transient audio handling when possible, restricted access to saved conversations, pseudonymous research identifiers, retention limits, and withdrawal procedures. These are design requirements whose adequacy must also be examined against the actual deployment environment; code-level safeguards alone cannot establish regulatory compliance.

In the European context, Regulation (EU) 2024/1689, the Artificial Intelligence Act, includes restrictions on certain uses of emotion inference, notably its prohibition concerning emotion inference in workplace and educational settings subject to the regulation's stated exceptions [Eur24]. This legal provision should not be simplified into a universal ban on all emotion-aware software, nor does a non-clinical label remove obligations relating to personal data. AffectLab is intended for voluntary individual rehearsal rather than employer or institutional assessment, surveillance, or consequential decisions about other people. Its applicable obligations would still require case-specific legal and ethical assessment before real-world deployment.

2.13 Model and Application Evaluation

An integrated application requires evidence at complementary levels. *Model evaluation* asks whether a classifier predicts predefined labels for held-out examples and whether its probabilities are calibrated. *Application evaluation* asks whether dialogue stages, feedback evidence, safety overrides, permissions, persistence, accessibility, recovery, responsive layouts, and performance behave as specified. Limited user feedback can reveal workflow friction or perceived usefulness, but it is supplementary to these technical forms of evidence. Confusing the levels produces exaggerated conclusions: passing application tests does not validate emotion recognition, while model accuracy alone does not establish that the completed product is robust or helpful.

For the modelling component, the surveyed literature motivates five session-held-out IEMOCAP experiments with consistent examples across text-only, audio-only, contextual, and fused configurations. Results should include accuracy, macro-F1, weighted F1, class-level error patterns, and calibration measures. Because utterances from one conversation are dependent, paired comparisons should respect conversational grouping rather than treating every turn as an independent participant. The precise statistical procedure and development-data separation are presented in Chapter 5 [BBL+08, LWC+22, GPSW17].

For the complete application, controlled test cases should include normal progression and failure paths: lack of a required communication feature, an unresolved conflict, model disagreement, low confidence, missing audio, unavailable language-model generation, a safety-triggering phrase, an interrupted session, stale or duplicate requests, persistence across restart, and desktop/mobile interaction. Performance checks should report their transport, persistence, model, provider, and concurrency scope so that local latency is not misrepresented as production capacity. A small user pilot may add feedback about completion, realism, and usefulness, but any observed change must not be interpreted as a causal learning effect without an appropriate comparison group [LKK+20, LSR+24].

2.14 Synthesis of Existing Research

The studies reviewed above address complementary parts of the overall problem. Research on communication training supports structured practice and reflection. ERC research develops contextual models and multimodal features that can improve benchmark predictions. Calibration research warns against treating predic-

tive confidence as self-validating. Dialogue research shows both the flexibility of generated responses and the need to test their consistency over multiple turns. None of these results, considered independently, validates the complete chain from emotion inference to safe rehearsal behaviour and meaningful user feedback.

Table 2.2: Selected research approaches, demonstrated contributions, and limitations for communication rehearsal.

Work	**Main contribution or evidence**	**Limit for AffectLab's problem**
Bosse et al. [BSN+12] and Lee et al. [LKK+20]	Controlled role-play evidence and synthesis of virtual-patient instructional designs.	Mostly clinical training; do not evaluate the proposed multimodal, uncertainty-aware individual rehearsal workflow.
IMBUE [LSR+24]	Personalised LLM-assisted interpersonal simulation with expert-informed feedback and participant evaluation.	Transfer varies across outcomes and situations; it does not establish the correctness of AffectLab's separate text–audio inference policy.
DialogueRNN, DialogueGCN, COSMIC, CoMPM [MPH+19, GMP+19, GMG+20, LL22]	Speaker-aware, contextual, graph-based, and commonsense-informed ERC modelling.	Classification architectures are not themselves instructional or safety-controlled rehearsal systems.
Li et al. [LWC+22]	Context-sensitive text–audio emotion classification with IEMOCAP session-aware experiments.	Benchmark classification does not establish calibrated behaviour or reliable deployment inference in an interactive role-play application.
Tsai et al. [TBL+19] and Baltrušaitis et al. [BAM19a]	Cross-modal modelling and a taxonomy of multimodal learning problems.	Rich fusion architectures do not automatically address missing inputs, transparent decisions, or learning outcomes.
Guo et al. [GPSW17] and Ovadia et al. [OFR+19]	Confidence calibration methods and evidence of degradation under distribution shift.	General findings require task-specific validation and cannot guarantee calibration on AffectLab participants.
Lu et al. [LJG25]	Multi-turn comparison exposing quality degradation in generated professional role-play dialogue.	Evaluates dialogue quality rather than the effect of a bounded hybrid controller on participants' learning.

Three conclusions are particularly important. First, *performance measures are not interchangeable*. Weighted emotion-classification accuracy, perceived empathy, realistic dialogue, acceptability, and communication-skill scores describe different phenomena. Second, *context and multimodality create opportunities as well as vulnerabilities*: adding information can improve a benchmark result but increases dependence on data quality, compatible inputs, and the assumed distribution. Third, *reliable application behaviour requires explicit policies*, especially when machine-learning results are uncertain or generative services are unavailable. A coherent system should therefore preserve provenance from model output to adaptation decision and from observed utterances to feedback.

Figure 2.1 summarises how these research streams motivate an integration problem while retaining separate standards of evidence.

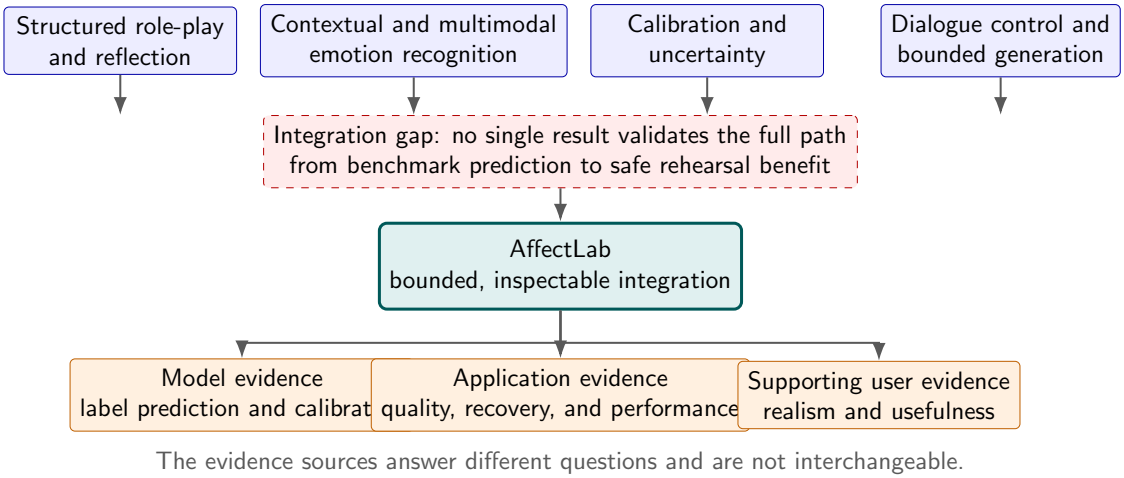


Figure 2.1: Relationship between the reviewed research streams, the AffectLab integration gap, and its complementary model, application, and supporting user evidence.

2.15 Research Gap and Positioning of AffectLab

The identified research gap is not that conversational role-play, emotion recognition, multimodal fusion, or feedback generation have never been studied. Each has an extensive literature. Rather, the literature motivates a need for a carefully bounded, reproducibly evaluated integration in which (i) a live-compatible textual context model is compared with an utterance-only baseline; (ii) independent text and speech models are compared with a transparent fusion mechanism; (iii) classification performance is distinguished from calibrated operational confidence; and (iv) affect predictions are allowed to influence only explicit, low-consequence role-play actions. This is an *integration and evaluation gap*, not a claim of universal novelty or proof that no similar application exists.

AffectLab addresses this gap through the following linked decisions. Its machine-learning study uses a four-category IEMOCAP task with five held-out sessions, a causal history window for text, an independent audio branch, and late fusion. Equal-weight and validation-fitted configurations are distinguished, and calibration is examined separately from classification. This is intended to provide controlled evidence for RQ1 and RQ2 without claiming real-world emotion understanding [BBL+08, LWC+22, GPSW17].

Its conversational component uses scenario-specific, deterministic stages and completion rules, while optional language-model generation provides bounded linguistic variation. Confidence, agreement, modality availability, and the user's stated pacing preference are interpreted by a versioned, inspectable adaptation policy. Missing or disputed emotion evidence leads to baseline interaction or an explicit user choice rather than an imposed psychological interpretation. Safety and feedback are independent of the affect classifier, and generation failures have deterministic fallbacks. Together, these choices turn RQ3 into a question about verifiable application behaviour rather than an untestable claim of machine empathy [BR09, LJG25, OFR+19].

Finally, AffectLab treats communication rehearsal as a useful, reflective activity delivered through a fully integrated application. Users practise workload negotia-

tion, boundary maintenance, and the expression of relationship needs, then inspect evidence-linked feedback, repeat or branch an exchange, and record an actionable takeaway. RQ4 is therefore addressed through developer-led verification of the complete product across functional, persistence, recovery, accessibility, responsive-interface, performance, and deployment boundaries. A small pilot contributes supporting perceptions of realism and usefulness but is not asked to prove product readiness or causal communication improvement [LKK+20, LSR+24].

2.16 Chapter Summary

Previous research provides substantial evidence for the value of structured communication practice, the usefulness of instructional feedback and reflection, the contribution of conversational context to emotion classification, the availability of strong pre-trained acoustic and textual representations, and the potential of multimodal fusion. It also reveals important limitations: benchmarks are domain-specific, labels do not directly measure private emotion, model confidence can be unreliable, generated dialogue can lose consistency, and apparently helpful simulations do not necessarily produce transferable learning [LKK+20, LSR+24, LWC+22, OFR+19, LJG25].

The work undertaken in this dissertation is consequently different from simply training another emotion classifier or constructing an unconstrained role-playing chatbot. AffectLab combines controlled context-aware text and speech experiments with a deliberately modest affect-adaptation policy, structured dialogue management, evidence-linked feedback, optional generation, reproducible fallbacks, and layered technical verification. Its contribution is to join these parts into a transparent, production-oriented application while keeping conclusions within the available evidence. Chapter 3 defines the proposed architecture, requirements, scope, ethical boundaries, and limitations; Chapter 4 documents the model and application implementation; and Chapter 5 presents the evaluation methodology and results.

Chapter 3

Proposed System Architecture

3.1 Architecture and Design Principles

AffectLab is a text-first application for rehearsing difficult interpersonal conversations. Its architecture combines deterministic application logic, trained affect-recognition models, optional generative services, and a browser interface, but it is not an end-to-end conversational model. The central design principle is separation of authority: probabilistic components may provide affective evidence or flexible wording, whereas explicit policies retain control of safety, dialogue progression, completion, and feedback evidence. This division makes consequential decisions inspectable and allows the core rehearsal to continue when a model or provider is unavailable.

Figure 3.1 shows the resulting information and authority flow. The interaction layer supports scenario selection, text and optional voice input, conversation display, and reflection. The application layer authenticates the request, validates ownership, coordinates the active session, and persists its state. Within the decision boundary, safety and dialogue policies are authoritative; affect inference supplies bounded evidence to the dialogue policy but cannot replace either authority. Response realisation converts a selected action into text using a constrained generator or its deterministic template. Feedback and reflection operate on recorded policy evidence rather than on unverified generated claims.

The architecture is deliberately text-first. Every conversational turn contains text, either typed by the user or reviewed after optional transcription. Text therefore remains available to the safety and dialogue components even when microphone permission is denied, audio is invalid, trained inference cannot be loaded, or an external provider fails. Raw audio is processed transiently and is not retained as part of conversation history. Optional services can improve modality coverage or linguistic variety, but their loss must not remove the core workflow.

The three standard scenarios—workload negotiation, personal boundary-setting, and expression of a relationship need—share a structural contract. Each defines its objective, stages, difficulty-dependent resistance, observable communication features, response actions, and possible outcomes. A deterministic policy evaluates the current message against that contract and records both the selected action and its evidence. The response-realisation layer receives the action only after this decision; consequently, generated wording cannot redefine success, remove intended resistance, alter the evidence record, or override a safety interruption.

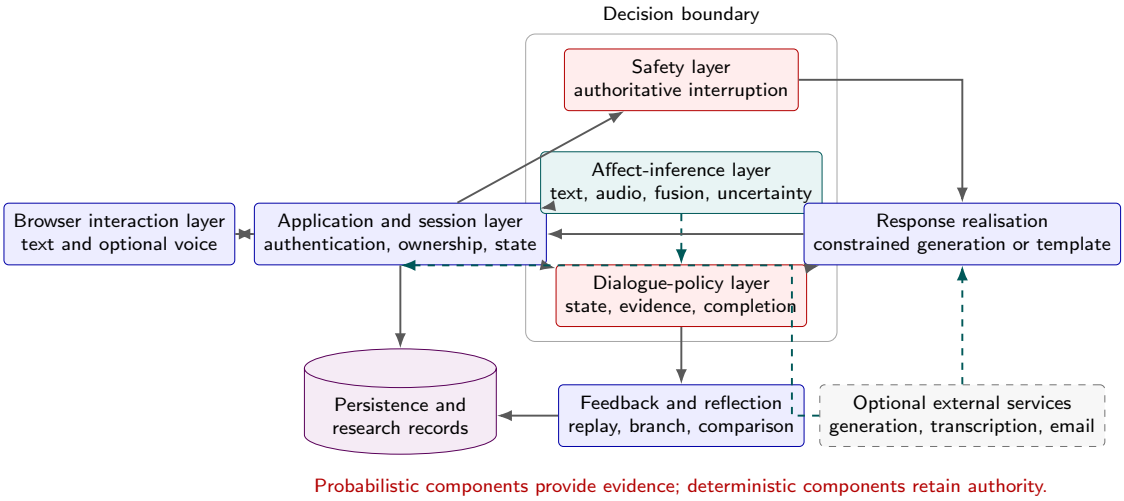


Figure 3.1: Logical architecture of AffectLab and separation between probabilistic evidence and authoritative application control.

Persistence stores the textual turns and enough decision metadata to reconstruct the interaction: state transitions, detected features, selected actions, reason codes, affect-policy decisions, component versions, and generation or fallback provenance. Completed sessions can then support evidence-linked feedback, replay, retry, branching, comparison, personal takeaways, and action cards. Research records form a separate, minimised projection rather than a copy of the participant-facing conversation.

Component boundaries are expressed through narrow contracts rather than shared access to model or provider internals. Inference returns probabilities and provenance; the affect policy returns a permitted adaptation decision; the dialogue policy returns state, evidence, and an action; and response realisation returns wording plus its source. Explicit unavailable and rejected states are part of these contracts. This makes failure behaviour testable, prevents an optional component from silently acquiring authority, and permits models, persistence implementations, or external providers to be replaced without rewriting the scenario policies.

These boundaries align the architecture with the research questions. Contextual text, speech, fusion, and calibration support RQ1 and RQ2; the separation of affect evidence from dialogue and safety authority addresses RQ3; and the integrated, recoverable application provides the object evaluated for RQ4. Model accuracy, policy conformance, application quality, and pilot feedback therefore remain distinct forms of evidence.

3.2 Grouped System Requirements

The original specification contains 29 functional requirements (FR1–FR29) and 14 non-functional requirements (NFR1–NFR14). Table 3.1 consolidates them into nine architectural groups while retaining every identifier used by the implementation and the Chapter 5 traceability analysis. This grouping is sufficient for the design argument; the complete atomic wording is reproducibility material designated for Appendix B during the appendix reorganisation.

Table 3.1: Grouped functional and non-functional requirements.

Group	**Identifiers**	**Required capability or quality**
Identity and data	FR1–FR3, FR26; NFR3–NFR5, NFR13	Authenticated ownership, clear disclosure and preferences, user-controlled optional features, minimised retention, deletion, security controls, and consistent stored state.
Preparation	FR4–FR8, FR19	Scenario preparation and configuration, text-first lifecycle control, and explicit successful, unresolved, manual, maximum-turn, and interrupted outcomes.
Voice and affect	FR9, FR11–FR14; NFR2, NFR14	Reviewed transcription, causal text context, separate audio evidence, declared late fusion, graceful missing-modality handling, and uncertainty-aware bounded adaptation.
Safety	FR10; NFR1	Textual safety checks interrupt ordinary role-play independently of affect classification and before generation.
Dialogue	FR15–FR18; NFR6, NFR8, NFR11–NFR12	Deterministic state transitions and evidence capture, constrained response wording, equivalent fallbacks, auditability, consistency, modularity, and testability.
Feedback	FR20–FR25	Evidence-grounded feedback, controlled phrasing, replay, retry, branching, compatible-attempt comparison, and user-authored reflection.
Research	FR27–FR28; NFR7	Versioned consent and eligibility, withdrawal-aware progress, pseudonymous minimal exports, and reproducible protocol and schema versions.
Support routing	FR29	Transparent lexical routing outside role-play, with tentative rule matches that have no authority over safety, eligibility, scoring, or scenario outcomes.
Operational quality	NFR9–NFR10	Bounded failures and recovery, measurable responsiveness, keyboard and assistive-technology support, visible status, reduced motion, and responsive layouts.

The groups also distinguish core from optional capability. Authentication, text interaction, scenario policy, safety handling, deterministic response templates, evidence-based feedback, and session management form the core. Voice capture, transcription, trained multimodal inference, generative wording, and research participation are optional. A valid implementation may expose an optional feature only when its configuration and failure path preserve the core requirements.

Consolidation does not weaken the atomic acceptance criteria. Each identifier still denotes a separately testable obligation, while a group states the common architectural boundary through which those obligations are implemented. Chapter 5 therefore traces identifiers to evidence rather than treating a group as a single pass/fail claim. This is particularly important for optional features: successful text fallback does not prove trained-inference quality, and successful provider generation does not replace verification of the underlying deterministic action.

3.3 Dialogue, Affect Adaptation, and Safety Boundaries

Each turn follows an authority-ordered pipeline. Authentication, ownership, input, and stale-state checks occur first. The safety layer then examines the text before affect adaptation or response generation. If a supported crisis-related expression

is detected, ordinary role-play stops, a predefined response is returned, and the interruption is recorded. Safety does not use the four-class emotion classifier: a neutral prediction cannot cancel explicit textual evidence, and a generator cannot suppress the interruption.

When no interruption is required, contextual text inference uses the current utterance and a fixed causal window of preceding turns. If valid speech is available, the audio model produces a separate distribution; calibrated modality outputs may then be combined through the declared late-fusion configuration. Availability, confidence, agreement, and component versions remain visible to the policy. Missing audio is absence of evidence, not evidence about the user's affect.

The affect policy maps this information to a small set of reversible actions. Depending on eligibility, user preference, dialogue state, confidence, and modality agreement, it preserves baseline behaviour, offers a pacing choice, or permits a tentative acknowledgement. It cannot determine crisis handling, scenario completion, feedback scores, study eligibility, or research inclusion. Thresholds are operational settings rather than psychological boundaries, and the interface must not present a predicted category as knowledge of an internal state.

In parallel, the scenario policy identifies observable features such as a specific request, maintained boundary, non-blaming statement, acknowledgement, or practical next step. It selects the next action and decides whether the exchange advances, continues under resistance, closes unresolved, reaches a defined outcome, or stops at the turn limit. A generator may phrase the selected action but cannot change it; invalid or unavailable generation invokes the template for the same action. Feedback uses the recorded features and supporting turns, with optional generation limited to restating that authoritative record.

Outside active role-play, deterministic lexical rules may select a bounded support strategy. Labels such as intent, cause, readiness, resistance, or cognitive pattern denote which authored rule fired, not validated psychological constructs. They remain contestable, have no authority over the rehearsal or research workflow, and are evaluated only for rule conformance.

The same authority order governs retries and concurrent requests. A turn is accepted only against the expected session state; a stale update cannot overwrite a newer transition. Retry and branch operations copy declared context but create distinct continuation records, so later feedback can separate inherited turns from new evidence. These constraints make replay reproducible and prevent a delayed provider response from retroactively changing a dialogue decision that has already been persisted.

3.4 Ethics, Privacy, and Design Limitations

AffectLab is intended for voluntary adult rehearsal of ordinary interpersonal conversations. It is not a diagnostic, therapeutic, crisis-monitoring, employment-assessment, educational-assessment, or surveillance system. Completing a scenario means satisfying its declared simulated conditions; it does not establish that the response is appropriate in every real relationship or that the user has improved a general communication skill.

The most important ethical constraint is that annotated expression is not di-

rect access to emotion. IEMOCAP contains ten actors in controlled English-language interactions, and its categories simplify ambiguous affective experience [BBL+08]. Speaker-independent evaluation limits identity leakage but does not establish demographic, cultural, device, or deployment validity. Confidence and text–audio agreement also do not prove correctness, especially under distribution shift [GPSW17, OFR+19]. The architecture therefore confines predictions to low-consequence choices, preserves a non-adaptive path, and allows the user to decline voice or adaptation without penalty.

Privacy constraints follow purpose limitation and data minimisation [Eur16]. Microphone use requires an explicit action; transcripts are reviewable before submission; raw audio is not intentionally retained; conversation storage is time-bounded and account-owned; and research exports exclude direct identifiers, raw conversations, audio, feedback prose, custom scenario wording, and personal takeaways. Pseudonymous records remain potentially linkable while withdrawal is supported and must not be described as anonymous. External transcription or generation also creates a provider boundary that requires disclosure, protected credentials, and a text-only fallback.

Safety remains deliberately limited. Conservative English lexical checks can miss indirect, coded, misspelled, sarcastic, or multilingual expressions and can interrupt quoted or hypothetical language. The application does not know the user's location, provide continuous monitoring, perform clinical triage, or contact emergency services. These limits must accompany the interruption response and prevent the application from being presented as a substitute for professional support.

Fairness and accessibility also shape the design. Text must remain a complete route through the application so that an accent, transcription error, unavailable microphone, or preference not to speak does not remove core functionality. Keyboard operation, visible focus, labelled controls, status announcements, reduced-motion support, and responsive layouts are architectural requirements rather than interface polish. Even so, aggregate model scores and automated interface checks cannot establish equitable performance or accessibility across cultures, communication styles, disabilities, and devices; representative evaluation remains necessary.

These safeguards are design constraints, not evidence of legal compliance. Processing purpose, institutional role, jurisdiction, provider terms, and recipients of affect outputs can change the applicable obligations. In particular, voluntary rehearsal does not authorise employers or educational institutions to use individual emotion inferences for assessment or discipline [Eur24]. Any such change would require a new ethical and legal analysis rather than reuse of the present evaluation.

Deterministic dialogue and feedback improve reproducibility but recognise only anticipated states and language patterns; valid paraphrases, indirect communication, and culturally specific wording may be missed. Generative wording remains probabilistic despite validation. Accessibility checks cannot establish usability for every disabled user, and technical safeguards cannot guarantee confidentiality in an incorrectly configured or compromised deployment. The principal empirical limitations—benchmark validity, authored test coverage, local performance scope, and the non-adaptive pilot—are consolidated with the results in Chapter 5. Accordingly, deployment still requires target-environment security, privacy, accessibility, provider, model, and legal review; the dissertation does not claim compliance certification.

Chapter 4

Implementation

4.1 Chapter Overview

This chapter describes the implementation of AffectLab, translating the architecture, requirements, and safeguards defined in Chapter 3 into a production-oriented application. The implementation combines a React and TypeScript browser interface, a Python and FastAPI backend, MongoDB persistence, trained text and speech emotion-recognition models, and optional external services for language generation, transcription, moderation, and email delivery. These elements are integrated as replaceable components rather than as a single end-to-end model, preserving the separation between affect inference, deterministic application control, response realisation, and presentation established in the system design.

The application follows a text-first and failure-tolerant implementation strategy. Structured role-play, dialogue progression, safety handling, evidence collection, feedback, and session management remain available without voice input or network-dependent artificial-intelligence services. When configured, the trained models add contextual textual and acoustic affect estimates, which are calibrated and combined through late fusion. Their outputs are interpreted by a bounded pacing policy and cannot determine crisis handling, dialogue success, or feedback scores. Similarly, a language model may improve the wording of a response or feedback item, but deterministic policies and recorded evidence remain authoritative. This arrangement makes component failures explicit and allows the core rehearsal workflow to fall back to reproducible local behaviour.

The chapter first presents the technology stack, repository organisation, and configuration approach. It then explains the emotion-recognition pipeline, including dataset preparation, contextual text modelling, speech modelling, multimodal fusion, calibration, and model provenance. The subsequent sections describe the backend services and persistence layer, the scenario-based role-play engine, affect-aware pacing, safety precedence, and the feedback and reflection mechanisms. The browser interface is then considered from the perspective of the participant and researcher workflows, including conversation preparation, voice interaction, uncertainty presentation, replay, branching, questionnaires, and research export.

The final sections describe the security and privacy controls, deployment modes, and verification strategy used during development. This chapter is concerned with how the system was constructed and how its implementation was checked through unit, integration, browser, accessibility, performance, end-to-end, and smoke tests.

Quantitative model performance, the consolidated application-verification results, and a short supplementary user-pilot summary are reported separately in Chapter 5.

4.2 Technology Stack and Project Organisation

AffectLab is implemented as a modular full-stack application rather than as a single machine-learning program. The implementation separates the participant-facing interface, application and dialogue logic, persistence, optional external integrations, and model-development code. This separation reflects the architectural boundaries introduced in Chapter 3: a trained classifier supplies evidence to the application, but it is not responsible for authentication, dialogue progression, safety handling, or feedback scoring. It also permits the core text-based rehearsal workflow to run when model artefacts, a graphics-processing unit, or a network-backed provider are unavailable.

Table 4.1 summarises the principal technologies in the implemented system. Exact Python application dependencies are pinned in the backend requirements file, while the frontend lock file records the resolved JavaScript dependency graph. Machine-learning dependencies use bounded version ranges because training and inference may be executed in different managed environments, including local installations and notebook runtimes.

Table 4.1: Principal technologies used in the AffectLab implementation.

Area	**Technologies**	**Implementation role**
Browser application	React 19, TypeScript, Vite, HTML, and CSS	Implements account, rehearsal, feedback, reflection, and research interfaces as a responsive single-page application.
Application programming interface	Python 3.12, FastAPI, Uvicorn, and Pydantic	Validates requests, coordinates services, exposes authenticated HTTP endpoints, and serialises typed response objects.
Persistence	MongoDB 8 and PyMongo	Stores users, account-owned sessions, research records, token digests, lifecycle state, and frozen research exports.
Machine learning	PyTorch, Hugging Face Transformers, DeBERTa-v3, and wav2vec 2.0	Supports text and speech model training, calibration, evaluation, and optional local multimodal inference.
Optional integrations	OpenAI Python client and SMTP	Provide constrained response wording, feedback phrasing, moderation, transcription, and transactional email when configured.
Verification	pytest, Ruff, Playwright, axe-core, ESLint, and the TypeScript compiler	Checks backend behaviour, code quality, browser workflows, accessibility, type consistency, and integrated operation.
Packaging and serving	Docker Compose, Docker, and Nginx	Builds the services, coordinates local deployment, serves the compiled frontend, and provides service health checks.

4.2.1 Frontend Technologies

The participant and researcher interfaces are implemented in React with TypeScript. At the time of the documented build, the lock file resolves React and

React DOM to version 19.2.8, TypeScript to version 6.0.3, and Vite to version 8.2.0. The TypeScript configuration enables strict checking and targets the ES2022 and browser document-object-model libraries. This provides compile-time checking across component properties, application state, and the HTTP data structures shared by the interface and backend schemas.

The frontend is a single-page application with a small application shell in `App.tsx`. It uses the browser History API to select public, authentication, onboarding, user, settings, practice, and optional researcher views. Feature code is divided into components such as the role-play workspace, conversation preparation, voice capture, emotion panel, replay view, branch comparison, study checklist, and researcher dashboard. State that belongs to an individual workflow remains close to the corresponding component, avoiding an unnecessary global state-management dependency.

Communication with the backend is centralised in a typed client under `frontend/src/services`. The client defines the API base address through `VITE_API_URL`, attaches the current access token, includes the refresh cookie, performs one controlled refresh after an authentication failure, and converts non-successful responses into interface errors. Request and response types are declared separately under `frontend/src/types`; this makes mismatches between components and backend response shapes visible during the TypeScript build. Browser session storage contains the short-lived access token, whereas refresh-token handling remains behind the backend-issued cookie boundary.

Vite provides the local development server and the production bundling step. The production container performs the TypeScript build before creating the optimised browser bundle, so type errors prevent an invalid image from being produced. The compiled static files are served by Nginx. Its configuration supports client-side routes by returning `index.html` when no static resource matches, prevents stale caching of the application entry point, and assigns long-lived immutable caching to content-hashed assets. Styling is implemented in the repository's CSS rather than through a component framework, allowing the responsive layouts, keyboard focus states, reduced-motion behaviour, status announcements, and touch-oriented controls to remain directly inspectable.

4.2.2 Backend Technologies

The backend is implemented in Python and is packaged with a Python 3.12 base image. FastAPI 0.141.1 provides the HTTP application and automatic request routing, Uvicorn 0.52.1 supplies the application server, and Pydantic-based schemas validate incoming data and serialise responses. The API is exposed below the `/api` prefix. Its endpoints cover authentication, user settings, session management, role-play actions, feedback, branching, multimodal inference, transcription, participant-study operations, and restricted researcher functions. FastAPI dependency injection supplies authenticated users and the configured services to these endpoint functions.

Backend code is divided by responsibility. The `api` package contains the HTTP boundary; `schemas` defines transport objects; `models` contains domain entities and enumerations; `services` implements application, dialogue, feedback, research, and external-provider behaviour; `repositories` defines persistence operations; `safety` contains the independent crisis-text checks; and `core` contains configuration, secu-

rity middleware, and service construction. Consequently, an HTTP route coordinates validated inputs and outputs without also containing the complete dialogue or persistence implementation.

Explicit interfaces further reduce coupling between these packages. The repository abstraction defines account, session, research-record, token, and dataset-freeze operations, with separate in-memory and MongoDB implementations. Interfaces for emotion analysis, cognitive analysis, strategy selection, and response generation similarly allow deterministic implementations, trained models, and provider-backed generators to be substituted without changing the conversation service's public contract. The application container selects the repository from configuration and constructs the authentication, conversation, multimodal, and transcription services once for the process.

The backend uses separate dependency files for the core runtime, optional local inference, and development tooling. The main `requirements.txt` file contains the web application, database, authentication, validation, and provider clients. The larger PyTorch, Transformers, audio, and numerical packages are added only through `requirements-inference.txt` when local trained-model inference is required. Test and static-analysis tools are isolated in `requirements-dev.txt`. This prevents a normal text-only backend installation from acquiring the substantially larger machine-learning runtime and makes the optional nature of multimodal inference concrete at the packaging level.

4.2.3 Persistence and External Services

MongoDB is the configured durable store for the integrated application. Its document representation accommodates a session containing ordered conversational turns together with dialogue states, policy decisions, feedback evidence, model metadata, and questionnaire records. The PyMongo-based repository adds ownership conditions to session operations so that a session identifier alone is insufficient to access another user's data. It also creates the indexes required for uniqueness, expiry, and research operations. Session and study-record retention are represented by separate expiry fields because the application workflow and the consented research dataset have different lifecycles.

An in-memory repository implements the same abstract interface. It is used by the normal automated backend suite and can support isolated development without a database process. The purpose of this implementation is behavioural substitution rather than durable storage: tests can exercise service logic quickly, while MongoDB-specific tests separately verify persistence across process restarts, index creation, expiry behaviour, cascade deletion, and concurrent-update protection. Selecting between the implementations through `PERSISTENCE_BACKEND` avoids conditional database logic throughout the services.

Network-backed functionality is accessed through narrow service boundaries. The OpenAI client may be configured for response realisation, feedback phrasing, moderation, and audio transcription. These calls are subject to timeouts and are not required for the deterministic role-play path. If credentials are absent, a service is disabled, a request fails, or an output does not satisfy the expected constraints, the relevant service reports the failure and the application uses its local response path. The email service follows the same configuration approach: SMTP enables ac-

count verification and password-recovery messages, while credentials and transport settings remain outside the source code.

The browser and API communicate through JSON over HTTP, with short voice samples represented only for the request that requires them. Local model inference loads private text and audio artefacts from configured directories rather than packaging those artefacts into the public repository. This arrangement keeps licensed data, model weights, credentials, and transient user audio outside version-controlled application code. The detailed persistence, privacy, and failure-handling controls are discussed later in this chapter.

4.2.4 Repository Structure and Configuration

The repository is organised around independently executable concerns. The `frontend` directory contains the React application, browser tests, and production web-server configuration. The `backend` directory contains the FastAPI application, dependency specifications, container definition, seed utilities, and backend tests. Model work is located under `ml`, with distinct packages for preprocessing, training, evaluation, and reproducible notebooks. Declarative experiment, fusion, baseline, and study-protocol settings are stored under `configs`. Supporting design records and data dictionaries are maintained under `docs`, repeatable command-line workflows are placed under `scripts`, and the dissertation sources are isolated under `thesis`.

This organisation separates source code from generated and restricted material. Raw datasets, checkpoints, experiment runs, local database material, virtual environments, Node modules, compiled frontend files, and environment files are excluded from version control. In contrast, model hyperparameters, label order, context length, audio limits, calibration parameters, protocol versions, and evaluation procedures are retained as configuration or documentation. A developer can therefore inspect the declared experimental and application settings without the repository distributing licensed datasets or private trained weights.

Application configuration is read through a typed Pydantic settings object. Values may be supplied through the process environment or through repository-level and backend-level `.env` files. The settings cover database selection, permitted origins and hosts, token lifetimes, retention periods, optional model paths, inference thresholds, provider timeouts, email delivery, study versions, researcher access, offline demonstration, and rate limits. Production-like environments additionally reject a weak signing secret, wildcard or non-HTTPS origins, wildcard hosts, disabled HTTPS enforcement, and disabled rate limiting. These checks cause unsafe deployment configuration to fail during application start-up rather than silently weakening the intended controls.

Two execution arrangements are supported. During local development, the frontend development server, backend server, and an existing MongoDB installation can be run independently. Alternatively, Docker Compose builds the frontend and backend and starts them with MongoDB 8, named database storage, dependency ordering, and health checks. The frontend image uses a Node.js 22 build stage followed by a small Nginx runtime image; the backend image exposes a readiness-based container health check. An offline demonstration configuration disables network-backed functions and provides deterministic behaviour for environments in which

external credentials or connectivity are inappropriate.

Together, the directory boundaries, dependency separation, lock files, declarative configuration, and container definitions make the implemented application inspectable and repeatable. They also preserve a practical distinction between the core application, optional probabilistic services, research-only model development, and deployment-specific secrets. The following section describes the emotion-recognition pipeline that produces the local model artefacts consumed through this structure.

4.3 Emotion-Recognition Pipeline

The emotion-recognition implementation transforms the private IEMOCAP release into separate text and audio classifiers. It calibrates their outputs without using held-out test labels and packages the final artefacts for optional local inference. The pipeline is isolated from the conversational application under the `ml` directory. Preprocessing, fold training, calibration, fusion, statistical comparison, final-model packaging, and integrity checks can therefore be executed without involving participant accounts or application sessions.

Two stages must be distinguished. First, model choices are assessed through five speaker-independent folds. Each fold produces validation and held-out test predictions, and only these fold results are suitable for reporting performance. Second, after the experimental configuration and calibration parameters have been frozen, one text model and one audio model are trained on all eligible examples for application inference. These final full-data artefacts have no independent test split and their training metrics are not treated as evaluation evidence.

Figure 4.1 makes this separation explicit by placing held-out evaluation and final-model packaging on distinct paths.

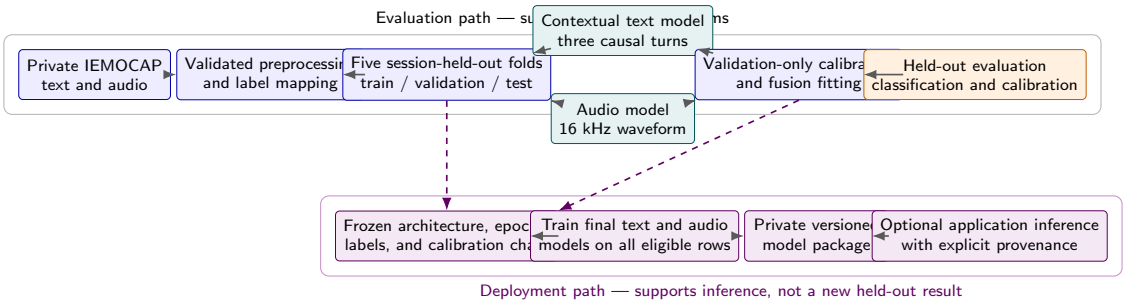


Figure 4.1: Emotion-recognition evaluation and deployment pipeline. The lower path packages a frozen configuration for inference but does not create a new held-out performance result.

4.3.1 Dataset Preparation and Label Mapping

IEMOCAP contains dyadic scripted and improvised sessions with aligned transcripts, sentence-level audio, categorical annotations, and dimensional ratings [BBL+08]. Access to the release is restricted, so the preprocessing code operates on a privately obtained archive and does not place raw media or row-level derived data

in the repository. The text preparation command can extract only annotation and transcription files from the archive. It rejects absolute paths and parent-directory traversal before extraction, and the audio preparation command applies equivalent path validation when creating the private task-specific audio bundle.

Annotation and transcript lines are joined by the IEMOCAP utterance identifier. For every usable row, preprocessing records the session, speaker, dialogue, timing, original categorical label, valence, arousal, dominance, transcript, and the expected location of the corresponding sentence-level waveform. Duplicate categorical annotations and missing transcripts cause preprocessing to stop. The resulting manifest records the source archive metadata, source-label counts, task definitions, split composition, per-class counts, and a SHA-256 digest of the manifest contents.

Table 4.2 shows the implemented label mappings. The deployed pipeline uses the conventional four-class task: happiness and excitement are merged, while anger, neutrality, and sadness remain separate. This produces 5,531 unique paired utterances. The code also retains an exploratory six-class task aligned more closely with the application vocabulary. That task preserves frustration, maps IEMOCAP fear to the operational label *anxiety*, and renames the merged happiness/excitement class as *joy*. It is not used by the deployed multimodal service, in part because the source corpus contains too few fear examples to support a stable application-facing anxiety model.

Table 4.2: IEMOCAP source-label mappings implemented for the two experimental tasks.

IEMOCAP label	**Four-class benchmark**	**Exploratory six-class task**
`ang` (anger)	anger	anger
`hap` (happiness)	happiness	joy
`exc` (excitement)	happiness	joy
`neu` (neutral)	neutral	neutral
`sad` (sadness)	sadness	sadness
`fea` (fear)	excluded	anxiety
`fru` (frustration)	excluded	frustration

The fold construction uses complete recording sessions rather than randomly partitioning utterances. For test session $s \in \{1, \ldots, 5\}$, the preceding session is used for validation, with Session 5 preceding Session 1 cyclically, and the remaining three sessions form the training split. Because each of the ten actors belongs to exactly one IEMOCAP session, this arrangement keeps speakers disjoint across training, validation, and test data. Preprocessing explicitly computes the speaker sets and aborts if any overlap is detected. Across the five folds, each session appears once as held-out test data.

The private audio bundle contains exactly the sentence-level waveforms referenced by the four-class rows and names them only by utterance identifier. During creation, every expected file must be found. The script audits sample rates, channel counts, sample widths, durations, byte totals, source-archive identity, and the final bundle hash. The recorded corpus audit confirms that all 5,531 packaged files are mono 16 kHz, 16-bit PCM. Raw and processed data remain outside Git and the OneDrive-backed repository, and the bundle manifest is retained beside the private archive for integrity checking.

4.3.2 Contextual Text-Emotion Model

The textual classifier uses `microsoft/deberta-v3-small`, a compact DeBERTaV3 encoder whose pre-training combines disentangled representations with replaced-token detection and gradient-disentangled embedding sharing [HGC21]. A classification head maps the encoded input to logits in the fixed order anger, happiness, neutral, and sadness. The same training program also supports the exploratory six-class label order through configuration rather than a separate model implementation.

Two input formats are implemented. The utterance-only baseline receives the target transcript with a maximum token length of 128. The contextual model receives at most the preceding three turns from the same dialogue followed by the target utterance, with a maximum length of 256 tokens. Previous turns are ordered by their annotated start time and are marked relative to the target speaker:

$$x_i = [r_{i-k}]\, u_{i-k} \parallel \cdots \parallel [r_{i-1}]\, u_{i-1} \parallel [\text{target}]\, u_i, \tag{4.1}$$

where $k = \min(3, i-1)$, u_i is the target utterance, r_j is either `same speaker` or `other speaker`, and $\parallel$ denotes concatenation with a line separator. Future turns, affect labels, dimensional annotations, and dialogue outcomes are never included. The backend reproduces this format from the last three available session turns so that training and deployment use the same causal information boundary.

For each fold, the encoder and classification head are fine-tuned with seed 42. The primary configuration uses a learning rate of 2×10^{-5}, weight decay of 0.01, a 10% warm-up period, training batches of 16, evaluation batches of 32, and at most eight epochs. Validation occurs after each epoch, the checkpoint with the highest validation macro F1 is restored, and training stops after two consecutive epochs without improvement. Trainable weights begin in 32-bit floating point; mixed 16-bit computation is enabled when a compatible graphics processor is available. The four-class task uses ordinary cross-entropy, whereas the optional six-class configuration supports inverse-frequency weights clipped to a declared maximum of 10 to constrain the effect of rare categories.

Each run stores the selected model and tokenizer together with its task, label order, seed, split sessions, class weights, training history, software environment, validation and test metrics, calibrated probabilities, per-class measurements, row-level prediction files, and confusion matrix. The utterance-only and contextual experiments use the same fold membership and identifiers, permitting paired comparison by complete dialogue rather than treating correlated turns as independent samples. Quantitative comparisons between these configurations are reported in Chapter 5.

After the fold experiments were frozen, the final contextual text artefact was trained for seven epochs on all 5,531 unique benchmark rows. The epoch count was derived from the completed cross-validation training behaviour rather than selected from the final full-data loss. Before training, the packaging code reconstructs the complete dataset from one fold, rejects duplicate identifiers, and requires the expected row count. The resulting artefact is intended only for inference; its training metrics do not replace the speaker-independent fold evaluation.

4.3.3 Speech-Emotion Model

The acoustic classifier fine-tunes `facebook/wav2vec2-base`, which learns speech representations from raw waveform input through self-supervised pre-training [BZMA20]. The same four-class output order is used as for text so that probability vectors can be paired without remapping. Training and inference load audio from memory or private local storage; the application database never provides audio as model-training data.

Waveforms are read as 32-bit floating-point samples. A multi-channel recording, if encountered, is averaged to mono. Audio with a different sampling rate is converted to 16 kHz by polyphase resampling, and only the first 20 seconds are supplied to the model. Shorter examples are padded dynamically within each batch by the model's feature extractor. Before a fold begins, an audio audit verifies that every metadata row has a matching waveform and records the observed sample rates, maximum duration, and number of examples requiring truncation.

The convolutional feature encoder is frozen during fine-tuning, while the transformer and classification layers remain trainable. The configured learning rate is 1×10^{-5} with weight decay 0.01, 10% warm-up, a per-device batch size of four, and four gradient-accumulation steps, producing an effective batch of 16 before distributed effects. Gradient checkpointing reduces memory use. Training runs for at most ten epochs, evaluates once per epoch, restores the checkpoint with the highest validation macro F1, and applies the same two-epoch early-stopping patience as the text experiment. GPU execution uses mixed precision when available.

Audio runs write the trained feature extractor and classifier along with the fold, label order, seed, split sessions, audio audit, environment versions, training history, validation-calibration parameters, test distributions, per-class measurements, and confusion matrix. Validation and test identifiers must match those of the contextual text run before fusion can proceed. Following the fold experiments, the deployable audio artefact is trained for ten epochs on the same 5,531 unique examples used by the final text model. As with the final text artefact, this is a packaging run rather than a new evaluation.

4.3.4 Multimodal Late Fusion

Text and speech are combined through decision-level late fusion rather than by joining their hidden representations. This choice keeps each modality independently trainable, inspectable, and usable in ablation analysis, and it avoids retraining a joint network when the fusion policy changes [BAM19a]. Before any distributions are combined, the fusion program verifies the label order, exact utterance-identifier set, expected label, recorded fold number, and reconstruction of the saved single-modality metrics. It also rejects an utterance that appears in the test output of more than one fold.

Let z_t and z_a denote text and audio logits, and let T_t and T_a be their validation-fitted temperatures. The calibrated modality distributions are

$$p_t = \operatorname{softmax}\left(\frac{z_t}{T_t}\right), \qquad p_a = \operatorname{softmax}\left(\frac{z_a}{T_a}\right). \tag{4.2}$$

Given text weight w, the intermediate fusion is a convex combination,

$$p_m = wp_t + (1 - w)p_a. \tag{4.3}$$

The research comparison uses a predeclared equal-weight setting, $w = 0.5$, so that the hypothesis that audio contributes complementary information is not assessed using a weight selected from the held-out test predictions. A separate deployment-calibration path fits w over the fixed grid $\{0.00, 0.01, \ldots, 1.00\}$ using paired validation predictions and validation negative log-likelihood only. After choosing the weight, it fits a final scalar temperature T_f and calculates

$$p_f = \mathrm{softmax}\left(\frac{\log(\max(p_m, 10^{-12}))}{T_f}\right). \tag{4.4}$$

For the packaged full-data models, the deployable parameters were fitted from 5,531 pooled validation-role predictions. Pooling changes the evidential status of the subsequent summary: across the five rotating folds, every utterance identity and label contributes once to this global fit and also once to the pooled test-role predictions used to describe the fitted chain, although the two predictions come from different trained models. The resulting metrics are therefore not an independent held-out evaluation. They are retained only as an operational packaging analysis for choosing one parameter set for the full-data models. The frozen configuration contains $T_t = 1.8268$, $T_a = 1.3357$, $w = 0.59$ for text, $1 - w = 0.41$ for audio, and $T_f = 0.5989$; these values are applied unchanged by the backend. Confirmatory performance claims instead use the predeclared equal-weight comparison and the fold-specific configuration in which each test session's labels did not participate in fitting that fold's parameters.

4.3.5 Confidence Calibration and Uncertainty Handling

For each single-modality fold, temperature scaling fits one positive scalar on validation logits by minimising negative log-likelihood [GPSW17]. The implementation searches in log-temperature space and never uses the test session during fitting. Dividing every class logit by the same temperature changes the probability concentration without changing the class with maximum logit. Calibration is consequently evaluated separately from classification performance.

The evaluation utilities compute accuracy, macro F1, weighted F1, per-class precision, recall and F1, negative log-likelihood, multiclass Brier score, and expected calibration error. Expected calibration error partitions maximum confidence into fixed bins and accumulates the sample-weighted difference between mean confidence and empirical accuracy. Row-level outputs retain the complete probability vector rather than only the winning label, enabling later reliability analysis and exact pairing between modalities. Statistical comparisons use paired percentile bootstrap samples at dialogue level so that multiple turns from one conversation remain in the same resampled unit; those results belong to the evaluation chapter rather than the implementation account.

At inference time, the backend returns the calibrated text, audio, and fused distributions, each modality's selected label and confidence, and an explicit indicator of whether the two modalities agree. Fused confidence below 0.55 is labelled *low*, confidence from 0.55 up to 0.75 is labelled *moderate*, and confidence at or above 0.75 is labelled *high*. The adaptation policy uses the configurable 0.55 low-confidence

boundary for the fused, textual, and acoustic confidences; the 0.75 boundary names the high presentation band but does not impose a further adaptation gate. These boundaries do not modify the model probabilities, turn a prediction into a verified emotion, or establish that estimates above them are correct.

Missing, invalid, conflicting, or low-confidence evidence is represented explicitly rather than converted into a stronger claim. The role-play workflow remains usable without audio or trained inference, and the dialogue policy and crisis checks do not depend on the predicted category. Calibration measured on IEMOCAP may deteriorate under changes in speakers, microphones, language, interaction style, or context [OFR+19]. The application therefore treats confidence and cross-modal agreement as bounded operational signals, not as guarantees of correctness.

4.3.6 Model Artefacts, Provenance, and Inference Loading

The final private package contains one contextual-text model directory and one audio-model directory. Each directory includes the saved classifier and the associated tokenizer or feature extractor. The versioned multimodal JSON configuration stores the label order, input field, maximum text length, sample rate, waveform-duration limit, model names, optimisation settings, and complete calibration chain. Model weights and licensed data are excluded from Git; a synchronisation script retrieves the final directories from private cloud storage into a user-selected local destination and prints the corresponding environment settings.

Provenance is captured after the private models are downloaded. The provenance command records the current source-control commit, SHA-256 of the multimodal configuration, Python platform, relevant library versions, and, for every file in both model directories, its relative path, size, and SHA-256 digest. It also calculates an aggregate directory digest from the ordered file entries. This manifest binds an inference result to specific model contents and code configuration without publishing the restricted artefacts themselves.

The backend keeps multimodal inference disabled by default. When enabled, it checks that both model directories and the configuration exist, then loads the tokenizer, feature extractor, and classifiers lazily on the first request. Device selection uses an available CUDA device when configured as automatic and otherwise uses the requested device or CPU. Models are placed in evaluation mode and executed under the framework's inference-only context. A process-level asynchronous lock serialises multimodal requests so that simultaneous users do not compete unpredictably for model memory; model execution is moved to a worker thread so that the asynchronous web loop remains responsive. Queue and inference times are returned separately.

Before inference, the service checks the configured byte limit, parses the in-memory WAV payload, requires a duration between 0.25 and 35 seconds, converts multi-channel audio to mono, resamples when required, and truncates model input to the configured 20-second maximum. Text context is constructed from the latest three causal turns using the same speaker-relative markers as training. The service then applies the stored modality temperatures, weighted fusion, and fusion temperature exactly as defined in Equations 4.2–4.4. Its response includes the distributions, agreement, confidence level, model version, latency, queue time, and an explicit statement that audio was not persisted.

Two forms of validation protect the packaged artefacts. Unit tests check causal context construction, confidence boundaries, disabled-service behaviour, and serialisation of concurrent requests without importing machine-learning libraries when inference is disabled. A private integrity smoke test selects an equal number of examples from each class, loads both final models, verifies finite logits and probabilities, runs the frozen fusion chain, and requires each component to predict more than one class. Any accuracy calculated on this balanced training sample is labelled as an integrity diagnostic and is not reported as evaluation performance. If loading or inference fails in the application, the failure remains contained within the optional service and the text-first rehearsal workflow continues through its deterministic path.

4.4 Backend Application

The backend application is the authoritative coordination layer between the browser, persistence store, dialogue policies, safety checks, trained models, and optional external providers. It exposes a JSON API below the `/api` prefix, but keeps transport concerns separate from the rules that govern a rehearsal. Consequently, neither the frontend nor a language-model response can directly alter dialogue state, feedback evidence, study status, or resource ownership. These changes are performed by typed backend services and persisted through a repository interface.

This section describes the general backend infrastructure and the lifecycle of a conversation request. The implementation of the role-play policy, feedback calculations, research workflow, and detailed security controls is developed in the subsequent sections.

4.4.1 Application and Service Architecture

The FastAPI application is assembled from four principal layers. API routes form the HTTP boundary and translate expected service failures into suitable status codes. Pydantic schemas validate transport data and constrain the response shape. Application services implement authentication, conversation coordination, transcription, multimodal analysis, preparation, branching, and research operations. Finally, repositories encapsulate all mutable storage. The domain models are shared between the service and repository layers, while external-provider clients remain behind dedicated service classes. This arrangement prevents route functions from becoming an alternative location for dialogue or persistence logic.

Service construction is centralised in a small application container. At process start, configuration selects either the MongoDB repository or its in-memory substitute, after which one authentication service, conversation service, multimodal service, transcription service, and email service is constructed. FastAPI dependencies provide these instances and the authenticated user to individual routes. The conversation service itself depends on interfaces for emotion analysis, cognitive analysis, strategy selection, response generation, and persistence. Default deterministic implementations can therefore be replaced without changing the route contract or the stored session representation.

The application lifespan initialises repository indexes before accepting normal work. In offline demonstration mode it also creates the local demonstration account if required; this mode is rejected by production configuration. Existing eligible sessions are then used to backfill any missing study records. A separate liveness endpoint confirms that the process is running, whereas the readiness endpoint validates configuration, checks the persistence connection and required indexes, and reports the availability of optional transcription and multimodal services. A missing optional model does not make the text-first application unready.

Cross-cutting HTTP behaviour is installed as middleware. Configured origins control credentialled cross-origin requests, trusted-host validation rejects unexpected host headers, and production deployments redirect HTTP traffic to HTTPS. Response middleware adds restrictive framing, content-type, referrer, permissions, caching, and content-security headers, together with HTTP Strict Transport Security in production. A configurable rate limiter applies separate windows to authentication, email, transcription, and model-backed operations. It is intentionally a single-instance implementation; a horizontally scaled deployment would require a shared limiter at a gateway or distributed store.

4.4.2 Domain Models and API Schemas

The domain layer represents application state as nested Pydantic models rather than unstructured dictionaries. Its enumerations constrain concepts such as speaker role, difficulty, practice goal, role-play status, support strategy, and user intent. A `Session` owns its ordered turns, current emotion state, optional role-play state, feedback, action card, questionnaires, research events, version, and expiry time. Each `ConversationTurn` may retain the emotion state observed at that point, selected strategy, response-generation metadata, and the bounded affect-pacing decision. More detailed role-play models preserve dialogue snapshots, evidence linked to source turns, reason codes, policy versions, branch lineage, and feedback comparisons. This makes the recorded session an auditable account of state transitions rather than only a transcript.

Transport schemas are deliberately distinct from these persistent entities. Request models impose limits before data reaches a service: messages are non-empty and limited to 5,000 characters, session titles to 80 characters, takeaways to 500 characters, and encoded audio to 7,000,000 characters. Enumerated fields constrain pacing preferences and difficulty settings, strict Boolean fields prevent ambiguous coercion for decisions such as eligibility confirmation, and selected request objects reject unknown properties. Responses expose only the fields needed by the relevant interface; for example, the public user response omits the password hash and token records held by the domain and repository layers.

Validation also protects relationships between fields. An enhanced chat exchange that requests audio analysis, affect adaptation, or non-default pacing must supply both a unique request identifier and the version of the session displayed by the client. Questionnaire schemas restrict ratings to their declared scales, and service validation checks that all required answers are submitted together or explicitly skipped. A multimodal estimate is accepted only when every probability is finite and within the unit interval, each distribution sums to approximately one, and the declared label and confidence agree with its maximum entry. Agreement and

confidence-band fields are then derived again on the backend instead of being trusted from an external payload.

The domain models also distinguish operational metadata from user-facing content. Generated responses record their source, model, latency, token counts, and any fallback reason. Affect decisions record availability, user preference, inference timing, confidence, and the selected pacing action. Study records contain pseudonymous participant and protocol identifiers rather than copying the complete account. These explicit fields allow later analysis to separate deterministic and provider-backed behaviour, while preventing absent measurements from being interpreted as observed zero values.

4.4.3 Authentication and Account Management

Registration requires acceptance of the current privacy terms. Email addresses are normalised before storage and a unique database index prevents duplicate accounts. Passwords are processed by the password library's recommended one-way hashing scheme; plaintext passwords are never stored. A cryptographically random verification value is sent by email, while only its SHA-256 digest and expiry time are retained. Normal login and protected routes require a verified email address. Registration, verification resend, and password-recovery endpoints return deliberately generic messages so that their responses do not reveal whether a supplied address belongs to an account.

Successful authentication issues two tokens with different handling. The short-lived JSON Web Token (JWT) used for API authorisation is returned to the browser and presented as a bearer credential. The longer-lived refresh token is placed in an HTTP-only, same-site cookie scoped to the authentication routes and marked secure in production. Its identifier and digest are stored server-side. Refreshing atomically removes the presented record and creates a replacement, so a token cannot be successfully rotated twice. Logout, password change, password reset, and account deletion revoke the user's outstanding refresh tokens; password-reset links are likewise digest-only, expiring, and consumed atomically after the replacement password has passed validation.

The dependency that resolves the current user decodes the access token, reloads the account, and checks email verification on every protected request. This means deletion or loss of eligibility is observed by later calls rather than being determined solely by claims in an old access token. Researcher routes add a second dependency that compares the verified account email with the configured researcher allow-list. Participant study enrolment is a separate state transition and does not implicitly grant researcher privileges.

Profile and lifecycle operations are handled through the same authenticated service boundary. Users can update the constrained profile fields, choose between one and three distinct practice goals during onboarding, change their password, sign out, or delete their account. Account deletion cascades through sessions, study records, refresh tokens, and outstanding verification and recovery records in both repository implementations. Study withdrawal has a narrower, protocol-specific meaning and is therefore implemented independently from deletion; the later research-workflow section describes this distinction in detail.

4.4.4 Session Persistence and Ownership Controls

The repository contract supplies identical account, session, token, study-record, lifecycle, and frozen-export operations for the in-memory and MongoDB implementations. The in-memory implementation supports isolated development and fast unit tests; the MongoDB implementation supplies durable storage for the integrated application. Services depend only on this contract, so choosing a persistence backend does not introduce database-specific branches into conversation processing.

Session ownership is enforced as part of repository queries. Reading or deleting a session requires both its identifier and the authenticated user's identifier, and session listings are filtered by the same user key. Expired sessions are excluded from reads, while a MongoDB time-to-live (TTL) index removes them according to their `expires_at` value. Thus, knowing another session's universally unique identifier is insufficient to retrieve or mutate it. Service methods translate a failed owned lookup into the same not-found result, avoiding a distinction between a missing resource and one owned by somebody else.

Every session also carries a monotonically increasing version. When the enhanced chat interface submits an exchange, it includes the displayed version and a request identifier. The conversation service rejects a repeated request identifier or stale version before modifying a deep copy of the session. At the storage boundary, MongoDB replaces a document only when its stored version equals the service's expected version, then increments it; an insert collision or failed conditional replacement becomes a concurrent-update error. The in-memory repository implements the same comparison. The global exception handler returns HTTP status 409 with instructions to reload, preventing two tabs or overlapping requests from silently overwriting one another.

MongoDB indexes additionally enforce unique email, session, study-record, protocol, and frozen-export identifiers, support per-user chronological queries, and expire session, token, and study records at their respective deadlines. Readiness checks verify these indexes rather than assuming they were created. Conversation and consented research data remain separate: a session contains the complete interaction needed by the application, whereas the synchronised study record contains protocol-bounded events, measures, generation counts, and feedback metrics under its own retention rule. The later research-workflow and data-protection sections describe the detailed minimisation and withdrawal behaviour.

4.4.5 Conversation Request Processing

The `/api/chat` endpoint is intentionally a thin coordinator. After schema validation and authentication, it delegates the message, account identity, optional audio, affect preference, request identifier, and expected session version to the conversation service. The service first performs the owned session lookup and the duplicate and stale-version checks described above. If necessary, the first message supplies a shortened default title; beginning a new exchange also closes an unused immediate post-questionnaire opportunity so that questionnaire timing cannot drift into later conversation.

Safety evaluation precedes affect adaptation and response generation. A local crisis-language check always runs, and provider moderation may add a flag when the provider is configured. If a crisis is detected, optional multimodal inference

is skipped, the fixed safety response takes precedence, and an active or paused rehearsal is marked as interrupted. The trained affect model is therefore neither a prerequisite nor an authority for crisis handling.

For a non-crisis exchange, multimodal inference is attempted only when the request is bound to a version and request identifier, audio is present, a configured model is available, and the rehearsal is eligible for experimental adaptation. It is deliberately excluded from required study tasks so that the required protocol path is not altered by this optional condition. Decoding, validation, or inference failure is caught and recorded as unavailable rather than aborting the message. Independently, the local text analyser updates the smoothed emotion state, the cognitive analyser derives tentative intent and readiness features, and the scored selector chooses a support strategy with its component scores and reason codes.

The user turn is appended before reply selection. During an active rehearsal, the deterministic role-play engine advances the dialogue state, links observed evidence to that turn, and supplies both a required character action and safe fallback wording. Outside role-play, the selected support strategy guides the general response generator. An affect-pacing prefix may be added only through the bounded pacing policy and only under its eligibility conditions. The resulting assistant turn stores generation and affect-decision metadata; role-play decisions additionally store before-and-after snapshots, evidence identifiers, policy versions, and reason codes.

Finally, the service records a minimised completion event, updates measurement boundaries when a rehearsal ends, persists the incremented session, and synchronises an eligible study record. The response returns the committed version together with the user and assistant turns, decision summary, current role-play state, any completed feedback, and the one-use opportunity token required for an immediate post-rehearsal questionnaire. This single service path keeps the dialogue shown by the client aligned with the state and evidence retained by the backend.

The ordering and alternative paths are summarised in Figure 4.2. In particular, the safety branch terminates ordinary processing before optional affect inference and response generation.

4.4.6 Optional Services and Deterministic Fallbacks

Network-dependent and model-dependent functions are implemented as enhancements rather than foundations of the rehearsal workflow. When an OpenAI API key is present, the response generator can realise a selected support strategy in natural language, moderate text, phrase evidence-based feedback, or render a role-play action. Requests use a configured timeout and disable provider-side response storage. If credentials are absent, the service is disabled, a call raises an exception, or a response is empty, refused, or structurally invalid, generation returns the corresponding deterministic template and records the fallback reason.

Provider output is constrained differently according to its purpose. General support generation receives the authoritative emotion state and selected strategy together with instructions against diagnosis, medication advice, certainty claims, and dependency. Role-play realisation receives the authoritative scenario, difficulty behaviour, state, and required action, and must return a typed action and a short in-character utterance. Outputs that expose coaching language or disagree with the required action are discarded. Where the enhanced dialogue policy contains a

concrete negotiated constraint or proposal, exact deterministic wording remains in force until semantic validation can reliably establish that a rephrasing preserves it. Feedback generation is limited to rewording strengths and suggestions that have already been calculated from recorded evidence; it cannot create new observations or scores.

Transcription and trained multimodal inference use separate authenticated endpoints and report explicit unavailability when invoked directly without configuration. In the integrated chat path, multimodal failure is contained and the deterministic exchange continues. Audio is size-checked and decoded in memory for the immediate operation, and service responses state that it was not persisted. Email delivery is similarly isolated behind its service boundary; generic account responses remain stable when delivery fails, without falsely granting verification or recovery access.

Each generated turn records whether its content came from a template, safety response, deterministic role-play path, or configured provider, together with latency and fallback information where available. Health and model-information endpoints expose capability status without treating an optional dependency as a core outage. These choices preserve an operational baseline in offline demonstrations, tests, and provider failures, while making the provenance of enhanced behaviour visible for debugging and later evaluation.

4.4.7 Rule-Based Cognitive Features and Support Routing

The class named `RuleBasedCognitiveAnalyzer` is a deterministic phrase matcher applied to the current text; "cognitive" is an implementation name rather than a claim that the component performs cognitive or clinical assessment. It checks ten authored regular-expression families labelled mind-reading, catastrophising, overgeneralisation, all-or-nothing thinking, should statements, emotional reasoning, fortune-telling, personalisation, discounting positives, and labelling. A matching family receives a fixed rule weight and the highest-weight family is returned as a "possible" pattern. The number is neither a learned probability nor a severity score. The rules have no access to clinical history, speaker identity, longitudinal belief patterns, or professional annotations.

The remaining fields are equally mechanical. Intent follows a fixed precedence: explicit rehearsal terms, practical-help terms, emotion or validation terms, a question mark, and otherwise unclear. "Readiness for advice" is assigned one of four constants according to those same matches; it is not estimated from observed acceptance of advice. "Resistance" is 0.7 for four explicit pushback phrases, 0.15 for any other occurrence of "but", and zero otherwise. "Possible cause" copies up to 100 characters following *because*, *after*, *when*, or *since*. Boolean request flags repeat the rehearsal, practical-help, and validation matches. These definitions make the output reproducible but also expose obvious failure modes involving negation, quotation, sarcasm, polysemy, indirect requests, multiple clauses, and dialectal or multilingual wording.

Outside active role-play, `ScoredStrategySelector` converts the heuristic record and lexical affect state into additive priorities for validation, reflection, clarification, validate–reframe, practical suggestion, rehearsal invitation, de-escalation, and safety escalation. For example, an explicit rehearsal phrase favours an invitation

to practise; a practical-help match with its associated fixed readiness value favours a suggestion; a cognitive-pattern match favours validate–reframe; and one of the four strong pushback phrases favours reflection while reducing the suggestion score. The maximum total selects the strategy supplied to the response generator. These strategy scores express rule priority, not the probability that a response is suitable or effective.

The subsystem has deliberately limited authority. Crisis handling is evaluated first and forces the safety strategy. During active role-play, the scenario controller selects the character action, so cognitive tags cannot advance a stage, declare success, or alter feedback scores. The subsystem does affect general supportive responses and its intent, possible pattern, possible context, selected strategy, and reasons can be displayed in the "Why this response?" panel. That visibility is why the feature is documented here even though it is secondary to the dissertation's emotion-recognition and rehearsal contributions. Its evidence base is limited to rule and integration tests reported in Section 5.6.2; no corpus-level predictive validity, clinical validity, or outcome benefit has been established.

4.5 Role-Play and Adaptation Engine

The role-play engine implements the central distinction established in the system architecture: deterministic code selects the permitted conversational action, while a generator may only realise that action in words. It therefore treats scenario progress, evidence, completion, safety, and feedback as application state rather than delegating them to a language model. The same state can be serialised, restored, replayed, rewound, and tested without a network connection.

4.5.1 Scenario Representation and Configuration

A role-play scenario is represented by an identifier, title, character, situation, user objective, opening line, expected skills, difficulty-specific behaviour, success conditions, and a maximum number of turns. The repository contains six standard scenarios: workload prioritisation, maintaining a personal boundary, expressing a relationship need, giving feedback to a colleague, negotiating a deadline, and sharing household responsibilities. The first three form the standardised study sequence; the remaining scenarios broaden ordinary practice without changing the study protocol.

Difficulty is an explicit enumeration with beginner, intermediate, and difficult levels. At initialisation these levels map to increasing challenge and decreasing cooperation values, but the selected level does not change the meaning of a recorded communication feature. For standard unprofiled practice, difficulty primarily controls how much resistance the character provides and, for the boundary scenario, how many clear refusals must be maintained. Every new attempt stores its scenario, policy, and scoring versions so that later feedback and comparisons are interpreted under the rules that produced them.

Additional practice in the workload, boundary, and relationship scenarios may also use one of three authored character profiles. The cooperative profile asks for a clear practical response, the rushed profile prioritises concise decisions and a workable time, and the sceptical profile questions the proposal and may repeat pressure.

These profiles select versioned deterministic policies rather than psychological personalities. They are unavailable for required study tasks and custom scenarios, preventing an optional experimental path from modifying the frozen protocol.

Users can create account-owned custom scenarios directly or through the preparation workflow. Custom inputs are normalised and limited in length, and their target skills must come from the supported observable set: clear request, specific detail, boundary maintenance, I-statements, and non-blaming language. A prepared scenario additionally stores the difficult part identified by the user and a tentative likely objection produced by a bounded local template. The template is presented for review and editing; it is not claimed to predict another person's response.

4.5.2 Dialogue State and Transition Logic

The role-play state stores status, current stage, turn count, difficulty, cooperation, accumulated evidence, decisions, outcome, completion reason, and timestamps. It supports active, paused, completed, and interrupted states. Pausing permits reflection without advancing the rehearsal, resuming returns to the existing controller state, and manual completion remains distinct from automatic success. The maximum of eight user turns prevents an unresolved simulation from continuing indefinitely.

The enhanced scenario policy represents each supported conversation as a small state graph. Workload practice advances from explaining the problem, through addressing constraints, to agreeing a plan. Boundary practice progresses from an initial refusal, through pressure, to respectful closure. Relationship practice moves from describing the situation and need, through acknowledging the other perspective, to selecting a practical next step. A transition occurs only when the observable evidence required at the current stage has appeared; unsupported or contradictory input leaves the controller in place and produces a targeted follow-up.

The three graphs contain scenario-specific checks. For example, the workload path requires the user to acknowledge the stated delivery constraint while proposing a supported trade-off before agreement can be confirmed. The boundary path distinguishes a maintained refusal from a reversal or conditional acceptance and varies the permitted pressure rounds by difficulty and profile. The relationship path requires a specific personal need, acknowledgement of perspective, and a feasible request; unsupported or explicitly unresolved endings remain recorded as such. These rules model the authored practice exercise, not the probable behaviour of a real manager, friend, or partner.

Figure 4.3 presents the principal state progression for the three standardised scenarios and the explicit non-success exits shared by the controllers.

Each processed rehearsal message produces a decision record containing the before-and-after dialogue snapshots, selected character action, reason codes, linked user and assistant turns, supporting evidence identifiers, generation metadata, character profile, and scenario, policy, and scoring versions. Completion reasons distinguish success, the turn limit, manual completion, and safety interruption. Persisting these transitions makes recovery exact: a restored session does not need to infer its stage again from the transcript.

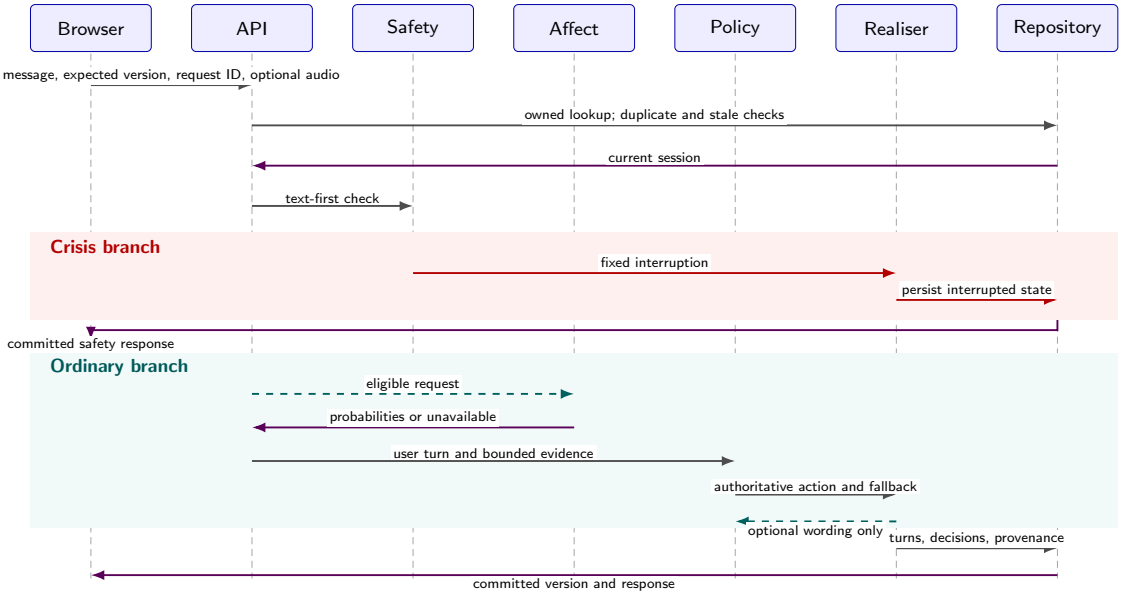


Figure 4.2: Sequence of a version-bound conversation request, including safety precedence and failure-tolerant optional processing.

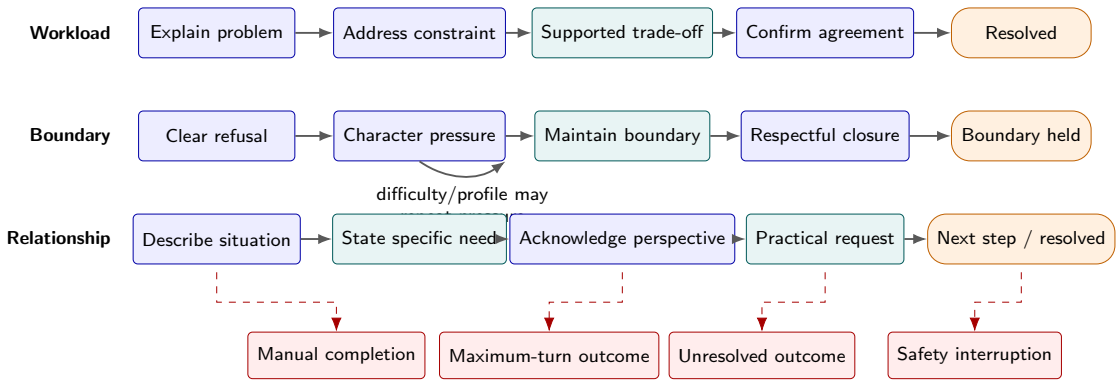


Figure 4.3: Principal dialogue-state progression for workload, boundary, and relationship rehearsal.

4.5.3 Communication-Evidence Extraction

Evidence extraction operates on the submitted text and records bounded, inspectable language features. The common extractor identifies patterns associated with a concrete request, repeated apology, maintained boundary, blame phrasing, specific detail, and I-statements. Versioned scenario extractors add features such as a problem description, workable option, check-in, clear refusal after pressure, respectful closure, acknowledgement of perspective, and practical request. Negation, conditional language, reported speech, and contradictory clauses are handled for the specific patterns required by the authored dialogue paths.

Each evidence item records its rehearsal number and source-turn identifier. It also contains the extracted feature names and the text analyser's arousal value. It does not store an invented quotation span. The original utterance remains the authoritative source, and replay highlights the complete linked turn when exact spans were not recorded. Features may accumulate across exchanges where a scenario permits this, but completion is still controlled by stage order; mentioning an agreement before the character has raised its constraint cannot skip the intervening dialogue.

The rules intentionally favour auditability over unrestricted semantic coverage. Indirect refusals, sarcasm, unusual paraphrases, and complex multi-clause statements may be missed. A detected phrase is therefore reported as observable evidence in this rehearsal, not as proof of communication competence, intention, personality, or likely real-world success. The deterministic test fixtures include supported, negated, reversed, irrelevant, and ambiguous examples to keep this boundary visible.

4.5.4 Affect-Aware Pacing Policy

The affect-pacing policy is a separate versioned layer applied after the dialogue controller has selected its response. It never changes the scenario stage, profile, difficulty, success conditions, evidence, or feedback score. Automatic use is disabled by default, and the participant may instead request that the exchange keep going unchanged, use a gentler pace, or add more challenge. An explicit choice takes precedence over automatic inference and can operate without audio.

Automatic adaptation is restricted to active, enhanced, non-required rehearsals. Required study tasks, associated retries, ordinary reflection, legacy sessions, paused or completed attempts, and ineligible scenarios use the baseline response. Safety and completed-exchange conditions take precedence. The policy also refuses to translate model categories into anxiety, readiness, resistance, or other psychological constructs; the existing lexical state tracker remains independent.

When automatic adaptation is eligible, missing audio, unavailable models, inference failure, or unsupported categories produce no addition. If any fused or component confidence is below the stored low-confidence threshold, or if the fused, text, and audio labels do not all agree, the response neutrally offers a pacing choice. When all three confidences meet that threshold and all three labels agree, anger or sadness adds only the phrase "I hear you."; agreement on happiness or neutral produces no addition. Thus, a matching estimate in either the moderate or high presentation band can reach the category branch. The user-selected gentler and more-challenge modes use fixed prompts. The full decision, reason, source, model

version, timings, preference, and probability summaries are stored with the assistant turn, allowing the interface to explain the exact action without reconstructing it later.

Figure 4.4 expresses the same policy as a decision tree and highlights the precedence of safety, eligibility, and explicit user choice over automatic inference.

4.5.5 Response Realisation

Response realisation begins with the authoritative action and deterministic wording returned by the controller. In offline operation this wording is sent directly. When provider generation is enabled for a supported unbranched practice flow, the language model receives the saved scenario, difficulty behaviour, current role-play state, affect estimate, required action, and fallback wording. Its structured response must repeat the required action exactly, remain in character, avoid coaching and hidden-system language, and stay within a short response limit.

Failure to parse the response, an action mismatch, disallowed coaching language, an exception, or unavailable credentials returns the deterministic wording. For enhanced dialogue states containing a concrete constraint or proposal, even a plausible paraphrase is rejected unless it matches the authoritative wording; this conservative rule avoids treating a correct action label as evidence that a generated sentence preserved the negotiation facts. Branches likewise use deterministic wording so their differences arise from the participant's alternative response rather than an uncontrolled regeneration.

Outside role-play, the support-routing component described in Subsection 4.4.7 supplies one of the permitted general-response strategies. The response generator receives the selected strategy but cannot select it. Generation metadata records whether wording came from a template, deterministic role play, the provider, or a safety response, ensuring that later study summaries can count response sources and fallbacks without retaining provider internals.

4.5.6 Safety Precedence and Crisis Interruption

A local crisis-language detector executes before model inference and response generation. Its bounded patterns cover direct self-harm and immediate-danger statements while applying exclusions for common negations, hypothetical or quoted material, and known false-positive phrases. When an external moderation service is configured, it can add a safety flag but cannot suppress a local match. This remains a conservative engineering safeguard rather than a validated clinical risk assessment.

On a detected safety condition, the backend skips multimodal inference and pacing, emits the fixed crisis response, and changes an active or paused role play to interrupted with the completion reason `safety_interruption`. The decision trail preserves the preceding dialogue state and records the interruption action. Safety-interrupted wording is never reused as a suggested action-card phrase, and the research workflow does not count the attempt as a completed protocol task.

The response states the application's limitations and directs the user to emergency or crisis support without claiming to know the user's location or contacting services on their behalf. Because the deterministic safety path precedes provider and classifier output, failure of either optional component cannot remove the inter-

ruption. False negatives and ambiguous statements remain possible, which is why the interface consistently states that AffectLab is not monitored care, diagnosis, or emergency support.

4.6 Feedback and Reflection Features

Feedback features are constructed from the evidence and decisions already stored during a rehearsal. They do not rerun an affect model, infer hidden reasoning, or ask a generator to judge the participant after completion. This design permits the user to move from a summary into the exact conversation evidence, try a bounded alternative, and prepare personal wording while preserving the original attempt.

4.6.1 Evidence-Grounded Feedback Generation

At completion, the role-play service calculates scenario-specific metrics from Boolean evidence attached to the participant's turns. Examples include clear request, specific evidence, maintained boundary, concise delivery, I-statements, and non-blaming language. A metric score is the proportion of relevant turns in which its condition was observed, and its evidence list contains the corresponding turn numbers. Versioned dialogue policies add stage-specific feedback while retaining the same link to stored evidence.

Deterministic templates turn these measurements into observed statements, strengths, and suggestions. A feature at or above the configured descriptive boundary becomes a strength; an absent or infrequent feature becomes a next-attempt suggestion. The completion reason and controller outcome remain separate from individual language metrics: reaching the turn limit is not relabelled as success, and a safety interruption is not scored as a skill failure. Custom scenarios receive metrics only for their selected supported skills.

If provider phrasing is enabled, it receives only the completed evidence-based feedback object and may reword its strengths and suggestions. The numeric scores, evidence links, and observations remain authoritative, and generation metadata distinguishes deterministic feedback from provider wording. Comparative feedback is added only when an earlier ordinary attempt uses the same scenario and scoring version; changes are described as differences in observed language rather than personal or clinical improvement.

4.6.2 Conversation Replay

Replay is a read-only projection of the stored session rather than a second analysis pass. Its timeline is bounded by the role-play start and measurement-end timestamps, with a compatibility rule for older sessions that recorded only completion time. Later reflection is excluded, while reflection entered during a pause remains visible but has no controller decision and is not shown as a scored exchange. Missing boundaries, legacy decisions, unavailable feedback, and inference failures are displayed as gaps instead of being reconstructed.

Selecting an exchange highlights the paired user and assistant turns. Separate panels show observable language features, controller action and stage transition, generation source, and any saved affect-pacing decision. Technical details expose

reason codes, before-and-after state, policy versions, modality distributions, confidence and agreement, availability, and timings. Feedback evidence buttons resolve stored evidence numbers back to their conversation-turn identifiers. Since exact phrase spans are unavailable, the complete source utterance is highlighted rather than presenting a guessed substring.

The browser constructs this view from the normal ownership-protected session response; replay has no new endpoint and makes no writes. It does not reload audio, regenerate explanations, or mutate recorded decisions. Navigation away from an unanswered immediate post-questionnaire may close that questionnaire window under the study rules, but inspection of the replay itself remains read-only.

4.6.3 Rewind, Branching, and Comparison

Rewind removes only the most recent linked user–assistant rehearsal exchange. For versioned dialogue it restores the exact preceding snapshot, truncates the matching evidence and decision lists, clears completion and feedback, restores the previous affect state, and removes later research events. It is refused when the latest messages cannot be linked safely, when they belong to copied branch context, or after post-ratings or an explicit skip have fixed the measured attempt. The removed user text is returned to the composer for editing.

Branching provides a copy-on-write alternative without modifying the original. It is available only for completed or interrupted attempts whose stored policy and pre-turn snapshot are supported. The server validates ownership, source version, branch-point turn, linked decision, policy versions, and intact evidence. It then creates a deterministic child identifier from the user and request identifiers, copies only the context before the selected response, restores the saved pre-turn state, and records lineage including the parent version, copied turn identifiers, branch group, and SHA-256 digest of the source rehearsal.

The child is labelled as a retry with no required-study association. Its feedback covers only newly authored continuation evidence; copied context is not counted again in turn totals, events, or generation statistics. Comparison presents the shared context once and the original and alternative continuations side by side, including actions, evidence, sources, and outcomes. If the parent has been deleted, expired, or changed since the branch was created, the saved alternative remains usable but the original continuation is explicitly marked unavailable. This prevents a silently altered source from being presented as a controlled comparison.

Figure 4.5 illustrates the copy-on-write branch and the distinction between shared context and newly assessed continuation turns.

4.6.4 Takeaways and Action Cards

The feedback screen provides a 500-character personal takeaway that is stored with the session and surfaced on the home dashboard. It is entirely user-authored and remains distinct from measured feedback. The dashboard also aggregates prior feedback metrics as a practice history, accompanied by language that these are averages of observed rehearsal features rather than a clinical assessment.

After a completed or interrupted rehearsal, the application can draft an action card containing an opening sentence, main request, boundary or fallback, and re-

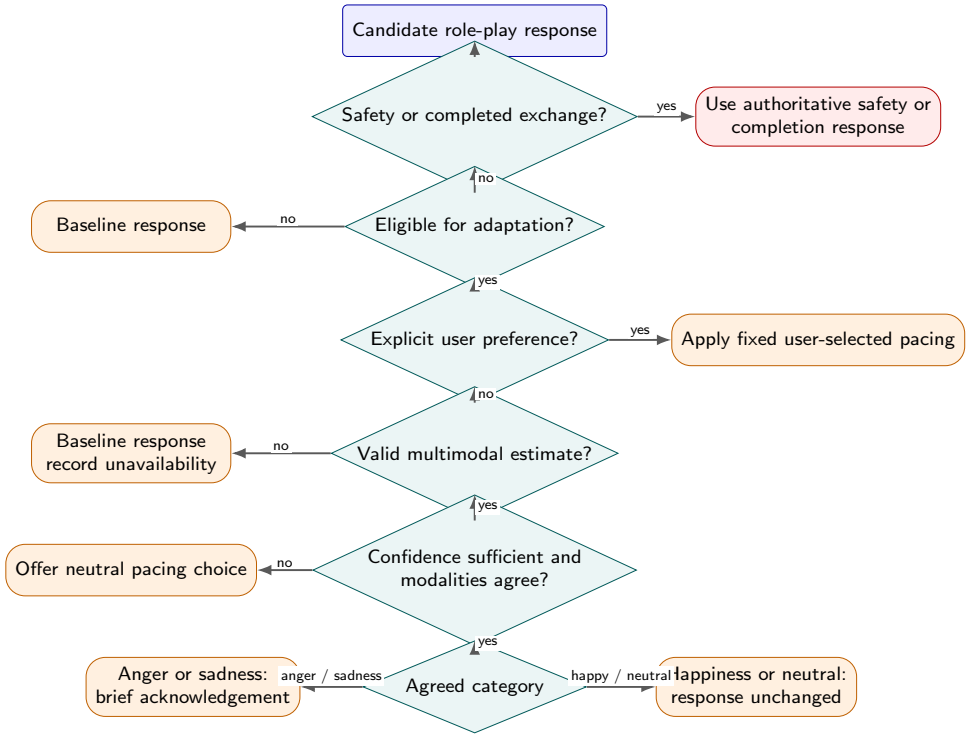


Figure 4.4: Decision tree for bounded affect-aware pacing. Every terminal action preserves the authoritative dialogue state and evidence.

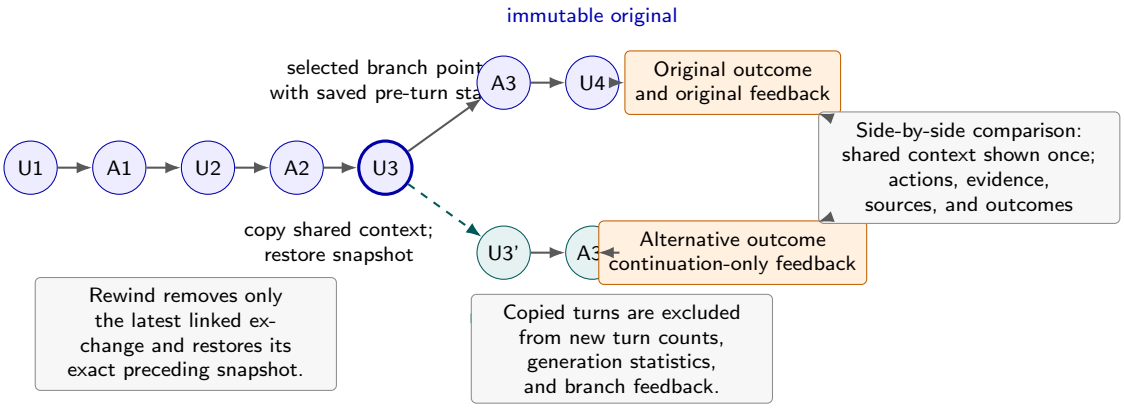


Figure 4.5: Replay, rewind, and copy-on-write branching from a stored pre-turn dialogue snapshot.

minder. The draft draws only from eligible participant turns within the rehearsal measurement boundary and stores the identifiers of any reused source turns. It uses a maintained-boundary utterance where available, otherwise supplies neutral fallback wording. Safety-interrupted utterances and later reflection are excluded from suggested real-world phrasing.

The user can edit every card field before saving. Saving requires the expected session version, marks the card as user-edited, and updates neither feedback, questionnaires, research events, nor measurement boundaries. Cards can be copied or downloaded by the browser and survive normal session reloads. In a branch they use new continuation turns rather than silently recycling shared context, preserving the distinction between the original and alternative plans.

4.7 Frontend Application

The frontend presents the backend's typed state as a sequence of participant and researcher workflows. It does not calculate authoritative dialogue transitions, affect decisions, or study completion in the browser. Instead, components retain editable local state, submit constrained requests, and replace their view with the version returned by the server. This keeps recovery behaviour and uncertainty disclosures as important as the successful path.

4.7.1 Application Interface Overview

Figures 4.6, 4.7, and 4.8 provide a compact visual overview of the participant-facing application. They follow the main user journey from choosing a practice activity, through an active rehearsal, to reviewing the resulting feedback. The screenshots should be captured with synthetic demonstration content and without personal or participant data. Only these three representative views are included because the purpose is to show the appearance and principal interactions of the implemented system, while the following subsections describe its behaviour in detail.

4.7.2 Application Navigation and User Workflow

The React application uses the browser History API to provide public landing and information pages, registration, sign-in, email verification and recovery, onboarding, the authenticated home dashboard, the practice workspace, settings, and the restricted researcher dashboard. On initial load it attempts to restore the access session and then performs one refresh-token exchange if necessary. Unauthenticated access to a protected route returns to sign-in, while a verified user who has not completed the current onboarding version is directed through onboarding before reaching the workspace.

Onboarding explains that the application is not therapy or diagnosis, that affect estimates can be wrong, that voice is optional, and that emergency support is outside the service. The user selects between one and three practice goals, which are stored on the account and used to recommend a starting scenario without restricting the available scenarios. The home dashboard then offers reflection, guided role play, recent sessions, saved takeaways, and a descriptive history of observed skills. Sessions can be resumed, renamed, or deleted.

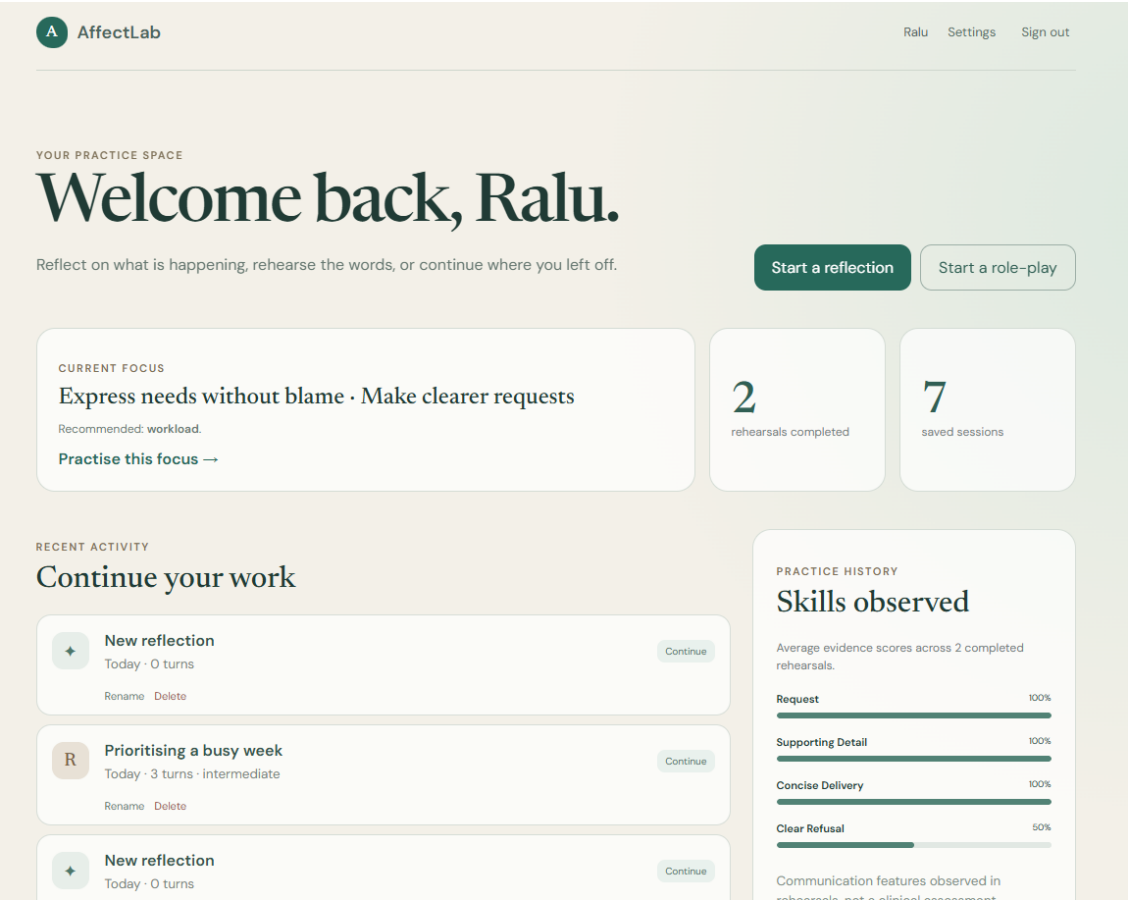


Figure 4.6: AffectLab home dashboard, showing the available practice paths, recommended starting point, and access to recent sessions and saved reflections.

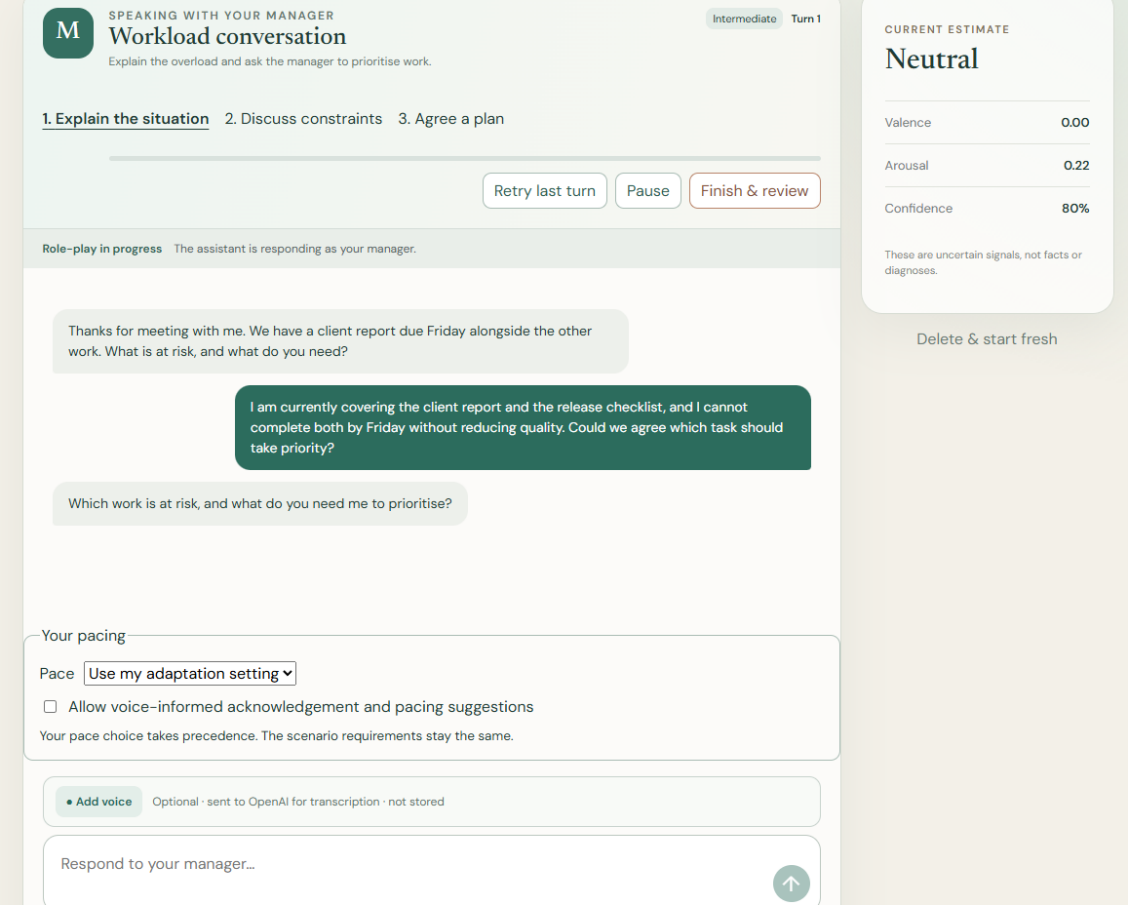


Figure 4.7: Active role-play workspace, combining the conversation transcript and message composer with scenario progress, session controls, and optional voice and affect-aware functions. The standard affect panel shown here is the lexical estimate; a trained voice–text result, when available, appears in a separate panel.

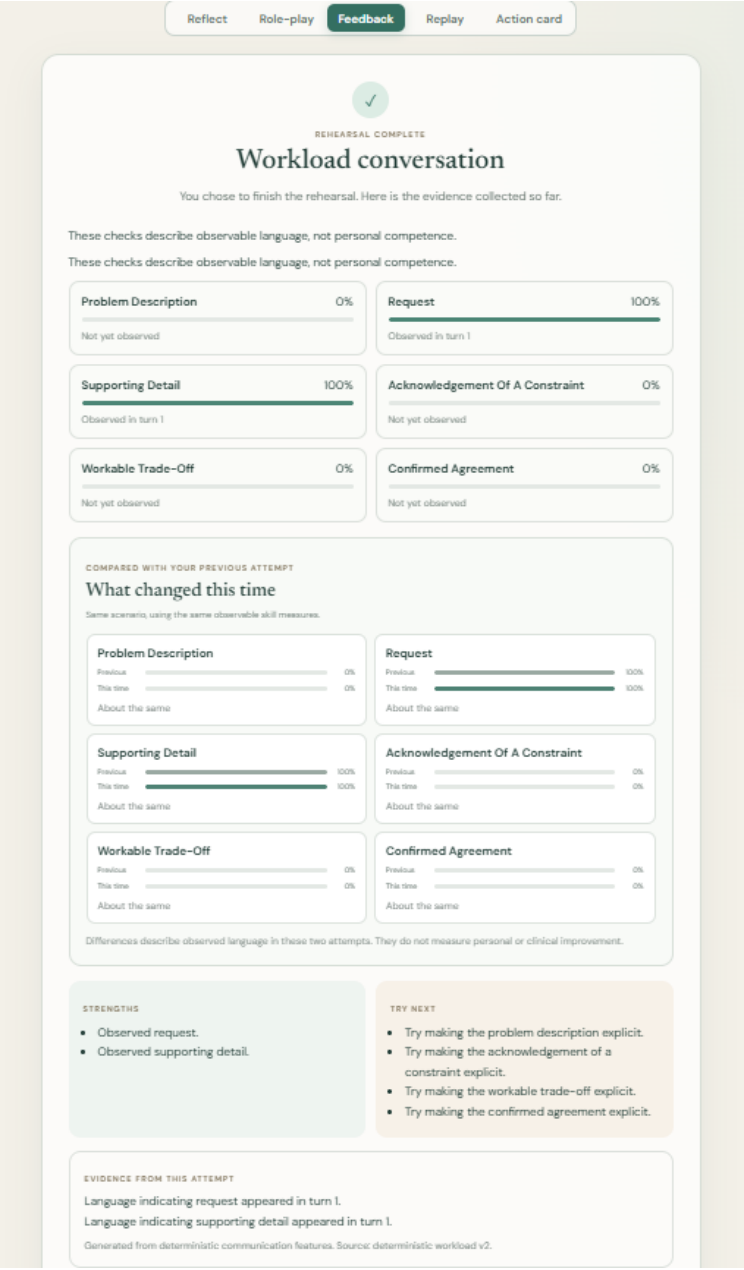


Figure 4.8: Post-rehearsal feedback and review interface, presenting evidence-linked observations and access to replay, alternative branches, personal takeaways, and the editable action card.

The application shell includes explicit loading, empty, success, and failure states. An error boundary provides a recoverable sign-in path if a component cannot render. Network operations disable only the relevant control, expose errors through alert or status regions, and generally retain the user's input for retry. Responsive CSS changes the multi-column workspace and comparisons into single-column layouts, while visible keyboard focus, semantic controls, reduced-motion rules, live regions, and touch-sized actions support accessible desktop and mobile use.

4.7.3 Conversation Preparation

Participants can begin with a standard scenario or prepare a conversation in their own terms. The preparation form asks who the conversation is with, what happened, the desired outcome, and the difficult part. These answers are sent to the bounded preparation endpoint, which returns an editable scenario draft with an opening line, supported skills, and a tentative objection. The interface labels this output as a rehearsal plan rather than a prediction.

The participant reviews and may edit the title, character, situation, objective, opening line, expected skills, difficult part, and likely objection before saving the custom scenario. Validation errors remain attached to the draft so an outage or rejected save does not discard their work. Previously stored custom scenarios can be edited or deleted only through the account that owns them; older scenarios without preparation metadata remain compatible with the editor.

Before every attempt, the setup screen displays the selected character, objective, supported skill targets, difficulty description, and, where eligible, character profile. Pre-rehearsal confidence and anxiety ratings require an explicit complete response or an explicit skip. Starting from a populated session creates a new attempt rather than replacing prior reflection, feedback, or research history.

4.7.4 Role-Play Workspace

The main workspace contains modes for reflection, active role play, feedback, replay, comparison, and the action card. A transcript panel presents ordered user and assistant turns, while the composer holds typed or reviewed transcribed text. During role play, a header shows the scenario, character, stage, progress, difficulty or profile, and controls for pause, resume, finish, and eligible rewind. Moving to reflection pauses an active rehearsal after confirmation; text entered while paused does not advance the role-play controller.

The browser includes the current session version and a newly generated request identifier with enhanced chat submissions. It clears the composer only after the server confirms the saved exchange. If a request fails, the typed draft and any voice attachment remain available; if the server reports a conflict, the interface offers to reload the committed conversation while retaining the draft. This avoids displaying a message bubble that was never persisted and lets the participant decide whether an uncertain submission should be sent again.

The workspace replaces its state from each response, including the saved user turn, assistant turn, emotion state, role-play status, feedback, and post-questionnaire opportunity. Busy indicators and an `aria-live` transcript communicate progress without inventing an optimistic response. Deleting and starting afresh

requires a deliberate action and uses the backend deletion path rather than only clearing local display state.

4.7.5 Voice Capture and Multimodal Interaction

Voice input is opt-in at both the settings and exchange levels. The browser requests microphone permission only after the participant selects the voice control, records for at most 20 seconds, and exposes recording and stop states. The captured signal is decoded through the browser audio context, mixed to mono, resampled to 16 kHz, encoded as PCM WAV, and kept only in component memory. Denied permission, unsupported recording, and conversion failure leave the text composer available.

When transcription is configured, stopping a recording sends the transient WAV to the authenticated transcription endpoint. The returned text is inserted into the composer for explicit review and editing; it does not become a conversation turn until Send is selected. A new recording or text edit clears any stale display associated with the previous draft. The reviewed text and optional WAV are then submitted together in one version-bound chat request so the affect estimate, pacing decision, and stored wording refer to the same exchange.

The interface permits a user to disable the microphone preference locally, enable automatic voice-informed pacing, or select keep going, gentler pace, or more challenge. It does not calculate predictions or send client-authored confidence values. After completion or failure, the component releases its media stream and object references. Raw audio is not added to browser session history or backend session objects.

4.7.6 Presentation of Affect and Uncertainty

The standard emotion panel displays the backend's tentative lexical state using a distribution, valence, arousal, intensity, trend, and confidence. This is the panel shown in Figure 4.7; it is not an output of the trained multimodal classifier. Its confidence is a deterministic heuristic derived from lexical evidence and temporal smoothing, so, for example, a displayed value of 80% does not mean that the estimate has an empirically calibrated 80% probability of being correct. Its language consistently presents these values as estimates. The panel is secondary to the conversation and is accompanied by the uncertainty disclaimer rather than being used as an authoritative description of how the user feels.

When a valid multimodal result is available, a separate panel presents the fused confidence, text and voice leaders, modality agreement, full distribution, latency, queue time, and the statement that audio was not stored. A fused label below the configured display threshold is headed as an uncertain estimate rather than as the predicted category. Disagreement receives an explicit explanation, and the full probabilities remain visible so that the leading label is not separated from its uncertainty.

For exchange-linked pacing, the "Why this response?" view uses the persisted `AffectDecision`. It shows whether analysis was requested and available, the selected action and reason, user preference, fallback code, model and policy versions, timings, and distributions. Reloading displays the same record and does not run

inference again. Missing or failed inference is represented as an availability gap, not silently replaced by the lexical estimate.

Table 4.3 makes explicit the different inputs, vocabularies, score semantics, operational effects, and supporting evidence of the four analysis and decision components. In particular, only the trained multimodal classifier has corpus-level classification and calibration evidence; the remaining scores expose deterministic rule strength or priority and must not be interpreted as probabilities.

4.7.7 Feedback, Replay, and Comparison Interfaces

The feedback screen distinguishes the controller outcome from the individual metrics. It renders evidence scores, supporting turn numbers, strengths, suggestions, observed features, generation source, and any same-version comparison with a prior attempt. Difference labels explicitly refer to observed language in two rehearsals and avoid claims of personal improvement. The same screen hosts the personal takeaway and the immediate post-rehearsal ratings when their server-issued opportunity token is still valid.

Replay presents a selectable exchange timeline with linked evidence, controller state, generation provenance, and affect-pacing details. Buttons move focus to the source turn for keyboard as well as pointer users. Missing legacy data and unavailable measurement boundaries have dedicated explanations. From a supported user turn, the participant can create a branch; the original remains unchanged and the new workspace opens with copied context and the old response as an editable draft.

The comparison view separates shared context from two labelled continuation regions and stacks them on narrow screens. It reports utterances, evidence, actions, sources, outcomes, and agreements without ranking one branch as globally better. The action-card view provides editable, character-limited fields with save, retry, copy, and download controls. Save failures preserve edits, and version conflicts ask the user to refresh rather than overwriting a newer card.

4.7.8 Optional Research Interfaces

Participant research controls live in settings and the authenticated dashboard rather than in public registration. The settings page fetches the current server-provided information sheet, displays eligibility criteria, data categories, processors, audio handling, retention, risks, withdrawal terms, and contacts, and requires separate confirmations plus the access code. Enrolled participants can see their pseudonymous identifier and consent versions, download a text-free personal research JSON file, or enter a two-step withdrawal flow without deleting the ordinary account.

An enrolled participant also receives a three-task checklist. It obtains progress from durable study records, labels tasks as not started, in progress, awaiting ratings, incomplete, or complete, and directs the next eligible action to workload, boundary, and relationship practice in order at intermediate difficulty. Required-task selection is fixed by the server; additional practice remains visibly separate. Closing or skipping a post-questionnaire advances the interface without calling the task complete.

The allow-listed researcher dashboard presents descriptive recruitment and completion counts, protocol status, scenario and difficulty summaries, questionnaire av-

Table 4.3: Distinction between the trained affect model and the application's deterministic analysis and decision components.

Component	**Inputs**	**Labels or outputs**	**Meaning of scores**	**Affected behaviour**	**Evaluation evidence**
Trained multimodal classifier	Target text, up to three preceding causal turns, and up to 20 seconds of 16 kHz speech	Anger, happiness, neutral, and sadness	Temperature-scaled text and audio class probabilities followed by calibrated late fusion; confidence is the maximum fused probability	Separate voice–text panel and optional bounded pacing. It does not determine crisis handling and is excluded from required study tasks	Five speaker-independent IEMOCAP folds with classification, calibration, confusion-matrix, and dialogue-bootstrap analyses (Section 5.5)
Lexical affect estimator (`lexical-v2`)	Current user text and the preceding lexical state used for smoothing	Anxiety, sadness, anger, frustration, shame, guilt, joy, and neutral; also valence, arousal, intensity, and trend	Normalised term weights, a neutral prior, punctuation adjustments, and temporal smoothing. Confidence is heuristic rule strength, not a calibrated probability	Standard affect panel; input to support-strategy selection and to bounded role-play state and evidence handling	Unit and authored-case tests of rule behaviour only; no labelled-corpus accuracy or calibration evaluation
Rule-based cognitive analyser	Current user text and the independently determined crisis flag	Five intent categories, ten possible cognitive-pattern matches, a possible cause, readiness, resistance, and request flags	Fixed values and pattern-derived weights indicating which rules matched; neither probabilities nor validated psychological measurements	Supplies tentative features to the strategy selector; it does not override crisis precedence or the role-play controller	Unit and integration cases described in Section 5.6.2; no predictive- or construct-validity evaluation
Scored strategy selector	Smoothed lexical state and cognitive-analysis output	Validation, reflection, clarification, validate–reframe, practical suggestion, rehearsal invitation, de-escalation, or safety escalation	Additive rule priorities used to rank strategies; scores are not probabilities	Selects the general-response strategy outside active role play. Safety and active role-play actions remain independently authoritative	Deterministic policy and integration tests (Section 5.6.3); no evidence that the selected strategy is clinically effective or optimal

erages, skill summaries, generation sources, and pseudonymous participant activity. Researchers may set the collection dates, record a bounded exclusion reason and data-quality note, download the versioned CSV, and initiate the final dataset freeze when eligible. Names, email addresses, transcripts, audio, custom wording, and takeaways are not rendered. A non-researcher cannot reach this view merely by navigating to its route because the API applies the independent researcher dependency.

4.8 Optional Research Support and Data Export

The research workflow implements the frozen feasibility protocol as versioned application state. Account privacy acceptance and study participation are separate, and ordinary use of AffectLab does not imply research consent. The protocol identifier, consent version, eligibility version, attempt purpose, task association, and measurement timestamps are carried into the records needed to reconstruct the study flow.

4.8.1 Participant Enrolment and Consent

The participant information endpoint supplies the current study title, protocol and consent versions, eligibility scope, collected data, external processors, audio handling, retention statement, risks, limitations, withdrawal terms, and research contacts. Enrolment requires an authenticated, verified account, a configured access code, an open collection window, and explicit confirmation that the information was read, participation is voluntary, and the stated processing is accepted. The access code is compared using a constant-time operation.

Eligibility is confirmed separately for minimum age, approved geography, English use, and the additional frozen inclusion and exclusion criteria. The backend stores the criteria fingerprint, protocol version, and confirmation time, but not date of birth, precise address, health details, or a reason for ineligibility. If the configured criteria, consent version, or protocol changes, the old confirmation is not silently treated as current. Withdrawn or researcher-excluded accounts cannot re-enrol, and enrolment closes once dataset freezing begins.

Successful enrolment creates a pseudonymous participant identifier independent of email and records the exact consent and eligibility versions. These fields are evaluated again when starting a required task or accessing progress; possession of an access token from an earlier state is insufficient. The protocol is a single-arm feasibility and usability pilot, and the application does not expose efficacy or clinical-effectiveness claims.

Pilot evaluation boundary. Required study tasks are deliberately ineligible for multimodal inference and affect-aware pacing, as specified by the request-processing eligibility check and the adaptation policy in Subsection 4.5.4. The pilot therefore evaluates completion and perceptions of the baseline text-based rehearsal and deterministic feedback workflow. Its post-task realism and usefulness ratings do not evaluate, and cannot support claims about, the acceptability or benefit of the emotion-aware adaptation itself. Multimodal model quality and adaptation-policy

behaviour are instead evaluated through the separate benchmark experiments and engineering tests reported in Chapter 5.

4.8.2 Study Progress and Questionnaires

The required procedure contains workload, boundary, and relationship scenarios in that order, each at intermediate difficulty and on the non-adaptive baseline path. Only an attempt explicitly marked required and associated with the matching scenario can become the primary attempt for a task. The first such attempt is selected deterministically by start time and session identifier; an additional practice, retry, custom scenario, branch, or later successful attempt cannot replace it. This prevents optional affect-aware practice from entering the pilot outcomes and prevents outcome-driven reassignment of the protocol record.

Before an attempt begins, confidence and anxiety use integer ratings from 1 to 7. After an eligible completed rehearsal, confidence, realism, and usefulness use the same scale. Each phase requires either all declared items or an explicit skip. Pre-ratings cannot be added after the role play starts, and post-ratings require the one-use token issued at completion. Leaving the immediate feedback opportunity closes it; a saved response or skip cannot be edited, and the attempt cannot then be rewound.

A required task is complete only when the required scenario and difficulty match, its completion reason is success, maximum turns, or manual finish, and a complete post-questionnaire exists. A safety interruption, omitted post-ratings, closed window, or abandoned task remains incomplete. Progress nevertheless permits the participant to continue to later tasks, preserving observable missingness rather than forcing or fabricating a score. Collection dates and dataset-freeze state can disable new required activity while leaving ordinary practice available.

4.8.3 Pseudonymous Research Records

Eligible post-consent session activity is projected into a separate `StudyRecord`. It contains participant and session pseudonyms, protocol and consent versions, enrolment and activity times, retention deadline, turn count, attempt and branch classification, scenario, difficulty, completion reason, measurement times, numeric feedback metrics, questionnaire values, generation and fallback counts, and text-minimised event records. Pre-consent events are excluded, and branch copies are not counted as new participant activity.

The record excludes account names, email, password material, conversation text, raw audio, custom scenario wording, feedback prose, saved takeaways, action cards, and dialogue snapshots. Event export further reduces properties to counts because even a nominally structured property could contain custom wording. A participant with no retained sessions still receives a participant-level export row so the enrolment denominator is not silently reduced.

Study-record retention is independent of the shorter application-session lifecycle. Each eligible activity refreshes the configured research deadline, and MongoDB applies a separate time-to-live index. Startup backfilling projects missing records only for currently eligible, post-consent material. A protocol freeze blocks further durable study-record changes, preserving the final analysis boundary.

4.8.4 Research Dashboard and Data Export

The researcher dashboard constructs aggregates from the current protocol cohort while excluding withdrawn and researcher-excluded participants from analysis. It reports enrolment and completer targets, protocol completion, completed rehearsals, scenario and difficulty counts, questionnaire averages, deterministic skill averages, generation sources, and participant-level lifecycle status. These are descriptive monitoring values; the inferential analyses and substantive results belong to Chapter 5.

The live CSV uses the versioned `affectlab-frozen-dataset-v4` schema. Its rows include pseudonymous identifiers, consent and eligibility versions, timestamps, task classification, branch and feedback scope, questionnaire values, numeric feedback, generation and fallback counts, and event counts. Missing values remain empty rather than imputed. Branch rows explicitly report copied-context counts and continuation-only feedback so that they are not pooled unknowingly with full attempts. The participant's personal JSON export follows the same exclusion of identity and conversation text while retaining their own participant identifier.

Once the configured study end date is reached, an authorised researcher may start the freeze operation. The repository uses a freeze token to prevent concurrent freezes, generates a de-identified CSV with new sequential participant and session labels, stores its schema, protocol, time, row and participant counts, and SHA-256 digest, and then marks the lifecycle frozen. Subsequent CSV downloads return this immutable stored content with its checksum. Date changes, participant-review edits, enrolment, and research-record updates are blocked after freezing; withdrawal remains available, although an irreversibly de-identified frozen row can no longer be linked for removal.

Figure 4.9 summarises the minimising projection, distinct retention domains, and boundary introduced by irreversible dataset freezing.

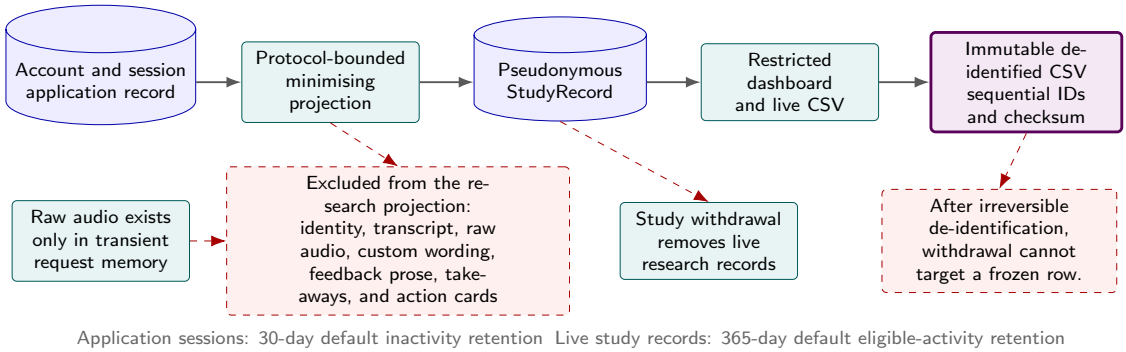


Figure 4.9: Research-data lifecycle from the richer account-owned session to the pseudonymous live record and immutable de-identified export.

4.9 Security, Privacy, and Data Protection

Security and privacy controls are distributed across configuration validation, middleware, authentication services, ownership-aware repositories, data minimisation, and user-facing lifecycle operations. They establish a production-oriented security

baseline rather than a guarantee of confidentiality. The repository includes a separate remote-deployment baseline because a local demonstration configuration is not sufficient for internet-facing use.

4.9.1 Authentication and Token Management

Account passwords are accepted only over the API's protected transport boundary in a deployed environment and are converted immediately to recommended adaptive hashes. Verification and password-reset values are generated with a cryptographic random source; the database receives only SHA-256 digests and expiry times. Password reset validates the new password before atomically consuming the stored record, so an invalid new password does not destroy the recovery opportunity while two successful requests cannot reuse it.

Access tokens are signed, typed JSON Web Tokens with configurable short lifetimes. Refresh tokens have separate type claims and unique identifiers, and their digests permit server-side rotation and revocation. The browser keeps the access token in session storage, whereas the refresh credential is an HTTP-only, same-site cookie restricted to the authentication path and marked secure in production. Changing or resetting a password revokes all refresh records and signs the current browser out.

Generic registration, verification-resend, and password-recovery responses reduce account enumeration. Email verification is checked after reloading the user on protected requests. Authentication and email endpoints have dedicated rate-limit categories, and production start-up rejects a disabled limiter. These controls reduce common automated misuse but do not replace monitoring, secret rotation, or infrastructure-level abuse protection.

4.9.2 Authorisation and Resource Isolation

Authorisation begins with the authenticated account identifier and is repeated at the persistence boundary. Session read, list, update, delete, multimodal analysis, branching, comparison, questionnaire, feedback, and action-card operations all obtain the session through an identifier-and-owner query. An unauthorised identifier produces the same not-found behaviour as a missing session. Custom scenarios are nested in the owning user, and branch creation verifies both parent and child lineage against that owner.

Researcher authorisation is not inferred from a route, client flag, or pilot enrolment. It requires a verified account whose normalised email is present in the server-side allow-list. Study operations additionally check current consent, eligibility version, protocol version, withdrawal and exclusion state, collection dates, and freeze state. Dataset lifecycle mutations, participant review, and exports therefore remain protected even if a participant manually invokes their URLs.

Optimistic versions and idempotency identifiers add integrity protection to authorisation. They do not merely identify who may write, but also which state that write was based on. Conditional MongoDB replacements and deterministic branch identifiers prevent lost updates and duplicate alternatives across processes. The API returns conflicts for recovery instead of choosing a winner silently.

4.9.3 Audio and Conversation Data Handling

Conversation text is stored because session continuation, replay, evidence links, and deletion controls require it. When provider-backed generation is enabled, the relevant session context may be sent to the configured provider with response storage disabled. The application does not write raw prompts or generated replies to its own operational logs. The participant information and settings interfaces disclose these boundaries rather than treating local and provider processing as equivalent.

Audio requires an explicit recording action for every exchange. The browser holds the WAV transiently, and the backend checks encoded and decoded size, file structure, duration, channels, and sample rate before transcription or local inference. Transcription may send it to the configured provider, whereas multimodal inference processes it in memory. Neither path adds the recording to the session, study record, or export. Only an approved transcript, bounded prediction summary, provenance, and pacing decision may persist.

Research projections minimise the richer session document. They omit identity, transcripts, free-text feedback, custom scenario content, takeaways, action cards, and raw model inputs. Exported event information is counted by event name rather than exposing arbitrary properties. Source-control exclusions separately keep environment files, local databases, raw datasets, checkpoints, model artefacts, generated runs, and backup material outside the repository.

4.9.4 Retention, Withdrawal, and Account Deletion

Application sessions and research records have distinct retention fields. By default, an active session expires after 30 days of inactivity, while a pseudonymous study record expires 365 days after its latest eligible activity. MongoDB time-to-live indexes perform eventual physical deletion, and repository reads also apply the deadlines logically so expired material is not returned while background deletion is pending. Token collections use their own expiry indexes.

A participant may delete an individual session without deleting the account. Account deletion removes the user, owned sessions, live study records, refresh tokens, and outstanding verification and reset records. The settings interface requires browser confirmation and retains the signed-in state if the request fails, allowing a safe retry. Backups remain a separate operational responsibility and must follow the approved encrypted retention schedule.

Study withdrawal is intentionally narrower. It removes questionnaires, questionnaire skips, post-rating tokens, research-event telemetry, and live study records, and marks the account as withdrawn so it is excluded from later collection and export. Ordinary account access, conversations, feedback, and takeaways remain available. The interface explains that rows already irreversibly de-identified in a frozen export or incorporated into aggregate analysis may no longer be linkable for removal. Re-enrolment after withdrawal is blocked.

4.9.5 Deployment Security Controls

Setting the environment to production, pilot, or staging activates fail-fast configuration validation. The signing secret must be unique and at least 32 bytes; allowed origins must be explicit HTTPS addresses; trusted hosts cannot contain a wildcard;

HTTPS enforcement and rate limiting must be enabled; and offline demonstration mode is prohibited. Refresh cookies become secure and responses add HTTP Strict Transport Security. CORS still permits credentials only from the declared origins.

The API and Nginx layers add complementary content-security, framing, referrer, content-type, permissions, and caching headers. Host validation and HTTPS redirection occur in the backend, while the frontend server restricts executable and embedded sources and grants media access only to local blob content. The intended remote arrangement terminates TLS at a maintained reverse proxy and prevents direct public access to MongoDB and the application server.

The bundled limiter is explicitly scoped to a single backend process and does not trust forwarded client-address headers. A multi-worker deployment must use a shared gateway or distributed limiter. The operational baseline also requires patched hosts, dependency auditing, protected secrets, encrypted access-controlled backups, and a restore drill. The backup utility preserves BSON types and indexes, records per-collection checksums and counts, and will restore by default only into an empty database whose name ends in `_restore_test`, reducing the chance of overwriting the live store.

4.10 Deployment and Configuration

The implementation supports three related execution arrangements: direct local development, a containerised integrated stack, and a deliberately offline demonstration. These arrangements reuse the same application code but select different repositories and optional services through configuration. None embeds production credentials or private model files in an image.

4.10.1 Local and Containerised Deployment

For local development, MongoDB can run as an existing service while Uvicorn serves the FastAPI application and Vite serves the frontend with hot reloading. The API address is provided to the frontend at build or development time. Developers may instead select the in-memory repository for isolated service work, accepting that its contents disappear when the process stops.

Docker Compose defines MongoDB 8, the backend, and the production frontend. A named volume stores database contents, the backend waits for the MongoDB health check, and the frontend waits for backend readiness. The backend image installs the core requirements into a Python 3.12 slim base and runs Uvicorn. Optional PyTorch and Transformers packages and private model directories are deliberately absent unless a separate inference-capable deployment supplies them.

The frontend uses a Node.js 22 build stage to run the TypeScript and Vite production build, then copies only compiled assets into an Nginx image. Nginx serves the single-page-application fallback, prevents caching of `index.html`, applies immutable caching to content-hashed assets, and exposes a small health endpoint. Compose's default values are for local use; a remote deployment must replace the development signing secret and satisfy all production validation.

Figure 4.10 shows the resulting service topology and distinguishes locally controlled components from optional external processing.

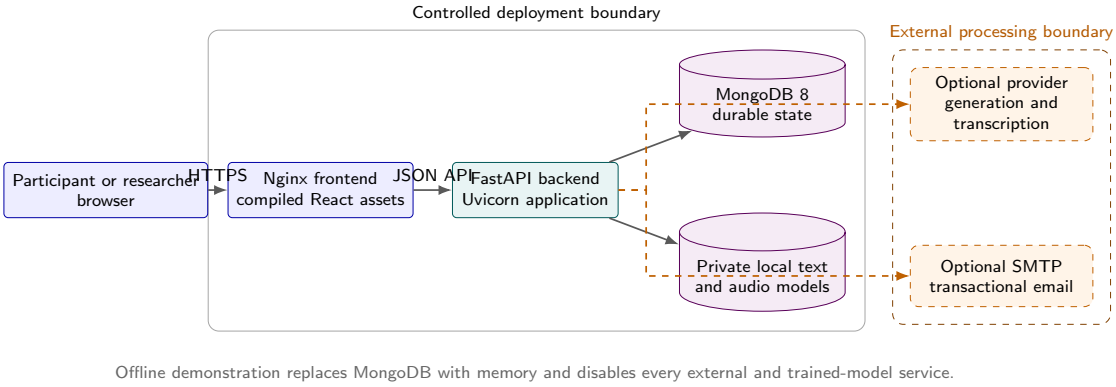


Figure 4.10: Containerised deployment topology and external-service boundary.

4.10.2 Offline Demonstration Mode

The offline configuration selects the in-memory repository, creates a verified demonstration account at application startup, and disables OpenAI generation, role-play generation, transcription, multimodal inference, SMTP, and rate limiting. Reflection and rehearsal therefore use only deterministic local templates. No MongoDB, cloud storage, private model, email server, or internet connection is required.

This mode makes the application's graceful-degradation claims demonstrable without pretending that unavailable capabilities ran. Model and transcription status remain unavailable, and data disappear on backend restart. The configuration validator rejects offline demonstration in production-like environments, so convenience credentials and automatic account creation cannot be enabled accidentally for a remote pilot.

A synthetic demonstration dataset and runbook keep example conversation content separate from participant data. A stronger engineering demonstration uses real MongoDB and a compiled frontend but still disables external providers and trained inference. The two modes serve different purposes: the memory-backed mode is portable for presentation, while the isolated MongoDB smoke run verifies persistence and restart behaviour.

4.10.3 Environment-Based Configuration

Settings can be supplied through process variables or repository- and backend-level environment files. They cover API origins and hosts, repository choice and database name, token and retention lifetimes, provider models and timeouts, SMTP, multimodal paths and thresholds, consent and protocol versions, study targets and contacts, eligibility scope, researcher allow-list, access code, offline credentials, and rate limits. Defaults support development but do not silently qualify as production settings.

The checked-in example file documents required names without containing live secrets. The real `.env` file is ignored by version control. Private model weights are synchronised into a user-selected local directory and referenced by path; raw IEMOCAP data and derived row-level artefacts remain outside the application repository. Separate dependency sets allow a deployment to omit local machine-learning libraries entirely.

Configuration values with scientific meaning are also retained in versioned JSON rather than only in mutable environment variables. These include the final multimodal label order, input limits and calibration chain, as well as the study protocol task order, measures, missing-data policy, target sizes, and prohibition on efficacy claims. This separates deployer-specific secrets from inspectable experimental decisions.

4.10.4 Health Checks and Operational Failure Handling

The liveness endpoint reports only that the API process can respond. Readiness revalidates settings, pings the repository, and verifies every required index; configuration, persistence, and index failures receive distinct states and HTTP 503. Optional transcription and multimodal capabilities are reported separately and do not make the text-first service unready. Container health checks consume these endpoints for dependency ordering and restart decisions.

Expected failures are translated into bounded client responses: unauthenticated and forbidden requests, missing resources, validation failures, unavailable optional services, rate limits with `Retry-After`, and concurrency conflicts have distinct status codes. Provider exceptions and inference failures do not expose stack traces or raw exception content in conversation records. The frontend preserves drafts and presents retry or reload actions according to whether the operation may have committed.

The system is designed to fail closed where authority is involved and fail soft where an enhancement is involved. Missing ownership, stale state, invalid consent, a closed study window, or a dataset freeze prevents the write. Missing generation, transcription, or emotion-model capability preserves the deterministic text workflow. This distinction keeps availability from weakening access, protocol, or safety constraints.

4.11 Implementation Verification

Verification is organised by boundary rather than by a single undifferentiated test suite. Deterministic unit tests check policies and transformations, API tests check contracts and ownership, MongoDB tests check real persistence semantics, browser tests check user workflows and accessibility, performance tests detect latency regressions, and smoke tests join the compiled components. These developer-led checks provide the principal evidence for application quality; model accuracy and supplementary user perceptions remain separate questions.

4.11.1 Backend Unit and Integration Testing

The default pytest suite forces the in-memory repository and disables SMTP, provider, transcription, and trained-model inference. It tests authentication and recovery, generic anti-enumeration responses, token rotation and revocation, ownership, optimistic versions, session lifecycle, study consent and questionnaires, exports and freezing, preparation, feedback, replay boundaries, rewind, branching, crisis precedence, dialogue transitions, affect pacing, and optional-service failure. Pa-

rameterised cases exercise profiles, difficulty levels, negations, reversals, confidence combinations, missing modalities, duplicate requests, and legacy compatibility.

Machine-learning support tests separately validate IEMOCAP parsing, safe archive paths, causal context, speaker-independent folds, class weights, calibration, probability metrics, utterance pairing, fusion, bootstrap procedures, and balanced final-model smoke selection. The disabled multimodal service is tested without importing heavy machine-learning dependencies; concurrency tests verify serial inference access when enabled. These fixtures confirm implementation contracts, not the accuracy of a trained model on new users.

MongoDB integration tests are opt-in and create uniquely named disposable databases. They recreate repository and service clients to check persistence across restart, required indexes and logical expiry, separate study retention, cascade deletion, optimistic conflict detection, single-use recovery, versioned dialogue restoration, branch idempotency, and saved affect provenance. Backup tests create an archive, restore it into a protected disposable target, and compare checksums and document counts.

Ruff checks imports, syntax-level errors, modern Python usage, and selected bug patterns across application, test, and script code. Dependency security is checked separately because style and behavioural tests cannot identify published package vulnerabilities.

4.11.2 Frontend End-to-End Testing

The regular Playwright suite runs the browser against a deterministic mocked API boundary. This makes UI state and failure injection repeatable while testing registration and onboarding, required and ordinary practice, preparation, voice-review submission, feedback, replay, branching, comparison, action cards, research enrolment, questionnaire timing, researcher access, settings, and account recovery. Dedicated cases abort or reject network calls to verify that drafts, recordings, selections, and stored rows are not falsely discarded.

Projects run in desktop Chromium and a mobile Chromium profile. Representative pages are scanned with axe-core against WCAG 2.0 and 2.1 A/AA rule sets, alongside assertions for keyboard focus, accessible names, live status messages, and responsive layouts. Production compilation runs the strict TypeScript project build before Vite bundling, and ESLint checks React hook and TypeScript rules.

Because the normal browser suite mocks the backend, its passing result demonstrates frontend behaviour against declared contracts rather than integrated persistence. Assertions that depend on real authentication, FastAPI serialisation, MongoDB indexes, or process restart are reserved for the full-stack suite.

4.11.3 Performance Regression Testing

The repeatable performance script exercises the deterministic text-first workflow through FastAPI's in-process HTTP test transport. After warming the application, it records repeated liveness, authenticated scenario-catalogue, session-creation, role-play-start, and conversation-turn requests. Each summary includes the sample count, median, 95th percentile, and maximum, while the evidence file records

the execution environment and SHA-256 hashes of the relevant application and benchmark sources.

The benchmark deliberately uses the memory repository and disables transcription, trained multimodal inference, and provider generation. This isolates locally controlled routing, authentication, validation, dialogue, persistence-interface, and serialisation overhead and makes the check stable enough for regression use. Its scope statement explicitly excludes network transit, MongoDB, external providers, concurrent users, and deployment capacity. The performance results are reported in Section 5.6.5; production staging must repeat the measurement with the selected database, models, providers, network, and expected concurrency.

4.11.4 Full-Stack Smoke Testing

The container smoke runner overlays a dedicated Compose configuration on the normal stack. It builds the production frontend and backend, starts an isolated MongoDB database, creates a verified offline researcher, and waits for service health. Playwright then signs in through the rendered application, creates and reloads a pre-enrolment reflection, enrols with versioned eligibility and consent, completes a required rehearsal and questionnaire, verifies progress and researcher aggregates, checks that the official export excludes the pre-consent session, restarts the backend, and confirms that the completed session persists.

A second journey exercises an enhanced deterministic rehearsal, simulated network failure and draft recovery, replay, branch creation, comparison, action-card persistence, another backend restart, and preservation of the original required-task progress. The runner always removes its isolated containers and volume, and uploads traces or screenshots on continuous-integration failure. A GitHub Actions workflow runs this smoke path on pushes and pull requests.

The native engineering smoke script provides a complementary local path. It builds the compiled frontend, serves it from a small local server, launches and restarts actual Uvicorn child processes, uses a uniquely named real MongoDB database, and writes a synthetic evidence manifest with environment details, source hashes, elapsed time, and restart identifiers before deleting the database. Neither smoke arrangement calls a live language provider or claims trained-model performance.

4.11.5 Reproducibility and Model-Provenance Checks

Reproducibility checks combine declarative configuration, frozen folds, stored predictions, code-level evaluation scripts, and immutable hashes. Model preprocessing records source and manifest digests; training configurations fix seeds and hyperparameters; calibration consumes validation predictions; fusion verifies exact utterance identifiers; and dialogue-clustered bootstrap procedures retain speaker and conversation dependence. Full-data packaging is kept distinct from held-out evaluation.

After private final models are downloaded, the provenance script records the Git commit, multimodal-configuration hash, platform and relevant package versions, and the size and SHA-256 digest of every text and audio model file. Ordered file entries produce aggregate directory digests. The private manifest can therefore bind

an inference installation to exact artefacts without publishing licensed weights or local storage paths in the repository.

The deterministic engineering evaluator uses authored synthetic conversations to compare expected state trajectories, evidence decisions, and affect-pacing actions. Its JSON output records scope, source hashes, environment, individual cases, and in-process timings, with an explicit statement that it is neither participant nor model evaluation. The evidence package similarly distinguishes reproducible engineering demonstrations from the cross-validated results reported in Chapter 5.

Continuous integration (CI) supplements behavioural checks with weekly and change-triggered Python and npm dependency audits, while Dependabot proposes updates for Python, npm, and workflow dependencies. Reproducing a result still requires recording the exact commit, lock files, private dataset and model provenance, export checksum, and applicable protocol version; a successful build alone is not treated as sufficient provenance.

4.12 Chapter Summary

This chapter described the implementation of AffectLab as a modular, text-first, production-oriented application. The React and TypeScript frontend communicates with a typed FastAPI backend whose services separate authentication, persistence, dialogue control, affect analysis, response realisation, research collection, and optional external providers. MongoDB stores ownership-bound, versioned sessions and separately retained pseudonymous study records, while an in-memory repository and deterministic templates support isolated testing and offline demonstration.

The emotion-recognition subsystem prepares speaker-independent IEMOCAP folds, trains contextual DeBERTa-v3 text and wav2vec 2.0 speech classifiers, calibrates their distributions, and combines them through late fusion. Private final artefacts are loaded only when configured and are tied to explicit calibration and provenance manifests. Within the application, these predictions remain optional uncertainty-bearing inputs to a bounded pacing policy; they do not control crisis handling, dialogue success, study completion, or feedback scores.

Role-play is governed by versioned scenarios, observable evidence rules, deterministic state transitions, and stored before-and-after decisions. Provider generation may phrase an authorised action but cannot select it. Feedback remains linked to source turns, and replay, rewind, branching, comparison, takeaways, and action cards preserve the distinction between recorded evidence and user-authored reflection. Safety checks precede inference and generation and interrupt a rehearsal through a fixed local path.

The optional research workflow separates account use from versioned study consent and eligibility. Required tasks, immediate questionnaires, pseudonymous research projection, text-free exports, lifecycle controls, and immutable dataset freezing implement the frozen pilot protocol without affecting ordinary practice. Authentication, ownership filtering, optimistic concurrency, retention, withdrawal, account deletion, transient audio handling, production configuration checks, and backup verification provide layered security and privacy controls as a foundation for controlled deployment, subject to target-environment validation and independent review.

Finally, unit, API, database, browser, accessibility, performance, full-stack, re-

producibility, and provenance checks verify the implementation at complementary boundaries. These checks establish that the coded components follow their declared contracts and that the local deterministic path remains responsive under the measured conditions. Model validity, deployment-scale capacity, and the limited supporting user evidence are reported with their separate boundaries in Chapter 5.

Chapter 5

Evaluation and Results

5.1 Chapter Overview

This chapter evaluates AffectLab primarily as a complete technical product. It first reports the speaker-independent emotion-recognition experiments concerning conversational context, acoustic information, multimodal fusion, and calibration. It then evaluates the integrated application through authored engineering cases, automated unit and integration suites, production builds, browser and accessibility checks, persistence and recovery exercises, full-stack smoke journeys, and a reproducible response-time benchmark. A short final subsection reports the supplementary user-pilot results without treating that small study as the principal validation of the application.

These sources support different claims. IEMOCAP results measure classification and probability quality on a controlled corpus. Engineering evidence establishes that the application follows its specified transitions, fallbacks, ownership rules, safety precedence, persistence contracts, responsive layouts, and recovery paths under the tested conditions. These benchmark and engineering evaluations are the principal evidence for the emotion-recognition, adaptation-policy, and application-quality claims. Local timings characterise the deterministic core but are not a substitute for deployment-specific load testing. The small user pilot contributes limited contextual evidence about perceived realism, feedback usefulness, and completion of the baseline rehearsal workflow; because required tasks bypassed multimodal inference and adaptation, it supplies no evidence about the acceptability or benefit of emotion-aware adaptation.

The chapter first maps the research questions to their appropriate evidence. It then presents the common model-evaluation protocol, the contextual-text comparison, the audio and multimodal results, and the classification–calibration trade-off. Application verification, performance results, and the concise pilot summary follow. The final discussion answers each research question only to the extent supported by the available evidence and identifies threats to validity.

5.2 Evaluation Scope and Evidence Hierarchy

5.2.1 Research Questions and Evaluation Levels

Table 5.1 maps the four research questions introduced in Section 1.3 to an evaluation unit and an admissible conclusion. This prevents model accuracy, policy conformance, and integrated application quality from being combined into one undifferentiated claim.

Table 5.1: Mapping between research questions, evidence, and permitted conclusions.

RQ	Evaluation unit	Principal evidence	Permitted conclusion
RQ1	IEMOCAP utterance with causal dialogue history	Utterance-only and three-turn contextual text predictions on identical held-out sessions	Whether causal context improves benchmark text classification.
RQ2	Paired textual and acoustic IEMOCAP samples	Audio baseline, equal-weight fusion, validation-fitted fusion, and calibration metrics	Whether audio adds benchmark classification information and what calibration trade-off follows.
RQ3	Versioned application decision and complete workflow	Authored policy fixtures, automated tests, saved provenance, and full-stack smoke journeys	Whether uncertain estimates are translated into the specified bounded, explainable, and failure-tolerant actions.
RQ4	Integrated application and its principal workflows	Automated suites, desktop/mobile browser checks, production builds, persistence and recovery exercises, full-stack smoke journeys, and local response-time measurements	Whether the completed system satisfies its declared functional and non-functional requirements under reproducible developer-led verification.

RQ1 and RQ2 are empirical machine-learning questions with completed results. RQ3 concerns the bounded integration policy: because the policy is deterministic, controlled input cases can establish whether it implements its specification. RQ4 concerns the quality of the complete implementation and is evaluated through complementary developer-led and automated checks at service, API, persistence, browser, accessibility, recovery, and full-stack boundaries. The small user pilot is reported separately as supporting product feedback rather than as the main answer to RQ4.

5.2.2 Separation of Development, Evaluation, and Deployment Evidence

The model study separates training, validation, and test roles by complete IEMOCAP sessions. Validation predictions select temperatures and, where applicable, fusion parameters; the held-out session supplies evaluation predictions. The predeclared equal-weight fusion is retained as the principal classification comparison because its weight was not selected from the fused test results. Fold-specific validation-

fitted fusion is reported as a calibration analysis rather than substituted retrospectively as the headline classifier.

The final text and audio packages were trained on all 5,531 eligible utterances after the cross-validation design and epoch counts were frozen. These packages have no independent test partition of their own. Their training behaviour and balanced-sample smoke checks are therefore integrity evidence, not model-performance results. Performance claims in this chapter come from the five session-held-out experiments.

Engineering test cases use controlled utterances and probability distributions to exercise known policy branches. Browser and full-stack tests automate test accounts and ratings. Their purpose is to demonstrate repeatability, persistence, recovery, and guard conditions, not to estimate human behaviour or the deployed model's accuracy. The supplementary user pilot is therefore interpreted independently of these tests.

5.3 Emotion-Recognition Evaluation Methodology

5.3.1 Benchmark Task and Speaker-Independent Splits

The primary benchmark uses the four IEMOCAP categories anger, happiness, neutral, and sadness. Excitement is merged with happiness, and utterances carrying other source labels are excluded. The resulting task contains 5,531 matched text–audio examples drawn from 151 dialogues across the five dyadic sessions of IEMOCAP [BBL+08]. The same utterance identifiers and label order are used for the text, audio, and fusion experiments.

Five folds provide speaker-independent coverage. In each fold, one complete session is reserved for testing and a different complete session for validation; the remaining three sessions are used for training. Every session therefore appears exactly once as test data. Keeping both speakers from a dyad within one session prevents an individual's utterances from appearing in the training portion of the fold in which that person is tested.

The utterance-only and contextual text conditions use the same DeBERTa-v3-small architecture, training settings, labels, seed, and folds. Their only intended difference is the input: the contextual condition prepends the previous three causal dialogue turns with speaker-relative markers. The audio condition uses wav2vec 2.0 on the paired 16 kHz waveform. This design supports paired comparisons because competing configurations produce predictions for the same examples.

5.3.2 Classification and Calibration Measures

Classification is reported through accuracy, macro-averaged F1, weighted F1, and per-class F1. Macro F1 gives each class equal weight and is the primary aggregate measure because the class frequencies are unequal. Weighted F1 complements it by weighting each class according to its support. Per-class results are retained so that an aggregate gain cannot conceal deterioration in one category.

Probability quality is considered separately. Negative log-likelihood and the multiclass Brier score are proper scoring rules over the complete predicted distribution. Expected calibration error (ECE) partitions examples by maximum confidence and accumulates the absolute difference between the mean confidence and empirical accuracy within each bin. Because ECE depends on the chosen bins and does not fully describe multiclass calibration, it is interpreted alongside the proper scores and reliability outputs rather than as a complete measure [GPSW17, KPNK+19].

Temperature scaling is fitted on validation logits only. For a logit vector $\mathbf{z}$ and positive temperature T, the calibrated distribution is

$$\hat{\mathbf{p}} = \mathrm{softmax}\left(\frac{\mathbf{z}}{T}\right). \tag{5.1}$$

A single temperature changes confidence concentration but not the predicted class. Fusion weights and a final fusion temperature are also fitted only from validation predictions in the deployment-oriented configurations.

5.3.3 Paired Dialogue-Level Statistical Comparison

Utterances from one conversation are not independent observations. Statistical comparisons therefore use a paired percentile bootstrap over the 151 complete dialogues rather than sampling individual rows. For each of 10,000 replicates, dialogues are sampled with replacement and the same sampled dialogue indices are applied to both compared systems. The difference in accuracy, macro F1, or weighted F1 is then recomputed. The 2.5th and 97.5th percentiles form the reported 95% interval. The procedure uses seed 20260808.

This interval describes uncertainty under resampling of the observed dialogue clusters. It is not a confidence interval over all cultures, microphones, languages, or spontaneous emotional settings. No participant or speaker subgroup inference is attempted because IEMOCAP contains only ten actors and was not designed to support such claims.

5.4 Contextual Text-Emotion Results

5.4.1 Primary Four-Class Comparison

Table 5.2 compares utterance-only text with the three-turn causal context condition. Context improved accuracy from 0.6624 to 0.7304 and macro F1 from 0.6651 to 0.7332. The absolute macro-F1 gain was 0.0681, with a dialogue-bootstrap 95% interval from 0.0504 to 0.0857. All three aggregate intervals excluded zero.

Table 5.2: Four-class contextual-text results on five held-out IEMOCAP sessions.

Metric	Utterance only	Three-turn context	Difference	95% interval
Accuracy	0.6624	0.7304	+0.0680	[0.0496, 0.0864]
Macro F1	0.6651	0.7332	+0.0681	[0.0504, 0.0857]
Weighted F1	0.6640	0.7300	+0.0661	[0.0477, 0.0842]

Table 5.3 reports the complete pooled class-level results underlying the aggregate scores.

Table 5.3: Per-class precision, recall, F1, and support for the four-class text configurations.

Configuration	Class	Precision	Recall	F1	Support
Utterance-only text	Anger	0.7465	0.6863	0.7152	1,103
	Happiness	0.7392	0.6980	0.7180	1,636
	Neutral	0.6134	0.6001	0.6067	1,708
	Sadness	0.5688	0.6827	0.6205	1,084
Three-turn context text	Anger	0.7781	0.6963	0.7349	1,103
	Happiness	0.7989	0.8136	0.8062	1,636
	Neutral	0.6421	0.6388	0.6404	1,708
	Sadness	0.7209	0.7841	0.7512	1,084

Support is the number of held-out examples after the five non-overlapping test sessions are concatenated and is therefore identical for configurations evaluated on the same rows. The per-class differences were not uniform. Happiness F1 improved by 0.0882, neutral by 0.0338, and sadness by 0.1307, with dialogue-bootstrap intervals above zero. Anger F1 improved by 0.0198, but its interval $[-0.0073, 0.0449]$ included zero. The result therefore supports an aggregate benefit from context but does not establish an anger-specific improvement.

The calibrated contextual model obtained pooled ECE 0.0311. This value describes the benchmark distribution after validation-fitted temperature scaling and does not validate the 0.55 application threshold on real AffectLab conversations.

5.4.2 Exploratory Six-Class Comparison

An exploratory task aligned additional source labels to anger, anxiety, frustration, joy, neutral, and sadness. The five held-out sessions contained 7,420 examples: 1,103 anger, 40 anxiety, 1,849 frustration, 1,636 joy, 1,708 neutral, and 1,084 sadness. Thus, anxiety represented only 0.54% of this exploratory dataset, whereas the other classes each contributed more than 1,000 examples. Context again improved all aggregate measures, as shown in Table 5.4. The pooled macro-F1 gain was 0.0732 with an interval from 0.0485 to 0.1009.

Table 5.4: Exploratory six-class contextual-text comparison.

Metric	Utterance only	Three-turn context	Difference	95% interval
Accuracy	0.5674	0.6228	+0.0554	[0.0382, 0.0734]
Macro F1	0.5490	0.6222	+0.0732	[0.0485, 0.1009]
Weighted F1	0.5675	0.6205	+0.0529	[0.0359, 0.0702]

Table 5.5 makes the session-to-session variation explicit. Context produced higher macro F1 in every fold, with fold-specific gains from 0.0475 to 0.0972. The unweighted fold means were 0.5456 and 0.6157; their standard deviations were 0.0330 and 0.0304, respectively. These unweighted means differ slightly from the pooled values in Table 5.4, which recompute macro F1 after concatenating all held-out predictions. The fold counts also expose the fragility of the anxiety result: individual test sessions contained only 2–12 anxiety examples.

Table 5.6 reports the complete class-level metrics for this exploratory task.

Table 5.5: Fold variation in macro F1 for the exploratory six-class comparison. Each fold holds out one complete IEMOCAP session.

Fold	Test n	Anxiety n	Utterance only	Three-turn context	Difference
1	1,377	12	0.5690	0.6180	+0.0490
2	1,357	9	0.5683	0.6655	+0.0972
3	1,535	2	0.4992	0.5813	+0.0820
4	1,519	7	0.5792	0.6268	+0.0475
5	1,632	10	0.5122	0.5870	+0.0748
Mean ± SD	–	–	0.5456 ± 0.0330	0.6157 ± 0.0304	+0.0701

Table 5.6: Per-class precision, recall, F1, and support for the exploratory six-class text configurations.

Configuration	Class	Precision	Recall	F1	Support
Utterance-only text	Anger	0.5857	0.5576	0.5713	1,103
	Anxiety	0.3889	0.5250	0.4468	40
	Frustration	0.5420	0.5165	0.5289	1,849
	Joy	0.7073	0.6779	0.6923	1,636
	Neutral	0.5102	0.4813	0.4953	1,708
	Sadness	0.5004	0.6347	0.5596	1,084
Three-turn context text	Anger	0.6089	0.5476	0.5766	1,103
	Anxiety	0.5778	0.6500	0.6118	40
	Frustration	0.5579	0.5809	0.5692	1,849
	Joy	0.7996	0.8026	0.8011	1,636
	Neutral	0.5174	0.4690	0.4920	1,708
	Sadness	0.6333	0.7408	0.6828	1,084

The class-level effects were heterogeneous. Frustration improved by 0.0402 ($[0.0158, 0.0647]$), joy by 0.1088 ($[0.0804, 0.1371]$), and sadness by 0.1232 ($[0.0926, 0.1557]$). The anxiety F1 difference was comparatively large at 0.1650, but it had a wide interval $[0.0476, 0.3144]$ and arose from only 40 IEMOCAP fear annotations mapped to anxiety. The anger difference of 0.0053 ($[-0.0288, 0.0339]$) and neutral difference of -0.0033 ($[-0.0316, 0.0241]$) both had intervals spanning zero. Consequently, the six-class experiment is not used as the deployed classifier and does not support a strong conclusion about anxiety recognition.

5.4.3 Answer to RQ1

Within the declared IEMOCAP protocol, RQ1 is answered positively. Adding the three preceding causal turns improved the principal text model by approximately 6.8 percentage points in accuracy and macro F1 compared with the utterance-only condition. Pairing, folds, architecture, and optimisation settings were held constant, and the dialogue-level intervals for all aggregate measures excluded zero.

The result is evidence for contextual benchmark classification, not proof that context reveals a user's true emotion. It also does not isolate which part of the representation produced the gain: additional lexical content, speaker markers, or interaction history may each contribute. External validation would be required to determine whether the improvement persists for spontaneous microphone speech and user-authored rehearsal conversations.

5.5 Speech and Multimodal Results

5.5.1 Audio-Only Baseline

The wav2vec 2.0 audio classifier obtained accuracy 0.6279, macro F1 0.6349, weighted F1 0.6229, and calibrated ECE 0.0468. Its complete class-level precision, recall, F1, and support are reported with the fusion configurations in Table 5.9. Audio was weaker than contextual text in aggregate but stronger for anger than the overall pattern might suggest, indicating that the modalities contained different useful information.

The comparison does not imply that the acoustic model measured emotion independently of linguistic content: speech representations can encode phonetic, lexical, speaker, and recording characteristics as well as prosody. It establishes only that the audio branch produced non-redundant predictive information on the matched benchmark task.

5.5.2 Predeclared Equal-Weight Late Fusion

The principal fusion experiment averaged validation-calibrated text and audio posterior distributions with weights of 0.5. Table 5.7 shows that fusion improved accuracy, macro F1, and weighted F1 over contextual text on the same rows. The macro-F1 difference was 0.0551 with a 95% interval from 0.0407 to 0.0713. Every one of the 10,000 dialogue-bootstrap replicates favoured fusion for each aggregate metric.

Table 5.7: Predeclared equal-weight fusion compared with contextual text.

Metric	Context text	Equal fusion	Difference	95% interval
Accuracy	0.7304	0.7832	+0.0528	[0.0381, 0.0683]
Macro F1	0.7332	0.7883	+0.0551	[0.0407, 0.0713]
Weighted F1	0.7300	0.7815	+0.0514	[0.0366, 0.0671]

The complete fusion class-level metrics are reported in Table 5.9. Unlike the contextual-text comparison, the combined system showed substantial performance across all four categories. Because the fusion rule was fixed before inspection of the combined test outcome, this result is retained as the primary evidence that audio contributed complementary benchmark information.

5.5.3 Qualitative Error Analysis

Inspection of matched held-out predictions illustrates both the value and the limits of context and multimodal fusion. IEMOCAP dialogue is not reproduced because of its access conditions, so the examples below are identified by utterance ID. They were selected after evaluation to explain recurring behaviours and are illustrative cases, not estimates of their prevalence.

Context sometimes corrected a locally plausible but wrong reading: for `Ses03M_script01_3_M046` (gold happiness), the utterance-only model predicted neutral, whereas the contextual model predicted happiness. The reverse occurred for

Ses03F_script02_2_M008 (gold neutral), which changed from a correct utterance-only neutral prediction to a contextual anger prediction. Context could also expose rather than resolve uncertainty. For Ses05F_impro08_F028 (gold neutral), the utterance-only model predicted neutral, but the contextual posterior assigned exactly 0.4407 to both happiness and neutral; the stored argmax was happiness. Thus, a changed top label need not represent a decisive contextual interpretation.

Text–audio disagreement was similarly bidirectional. For Ses03M_script02_2_F009 (gold anger), contextual text predicted neutral and audio predicted anger; equal fusion recovered anger. For Ses03F_script02_1_M020 (gold neutral), contextual text was correct but audio predicted anger, and equal fusion narrowly selected anger (0.4555 versus 0.4501 for neutral). Fusion therefore repaired some unimodal errors but could also let a strong wrong modality overturn a correct one. Finally, Ses03F_script01_2_M009 shows a shared failure: both modalities confidently predicted anger for a neutral annotation, so equal fusion reinforced the error. Together with neutral's lowest equal-fusion recall (66.6%), these cases indicate representative failure modes of contextual carry-over, near-tied interpretations, modality dominance, and correlated errors that late averaging cannot correct.

5.5.4 Classification–Calibration Trade-Off

Equal averaging improved classification but produced ECE 0.1619. Separately calibrated modality probabilities therefore did not remain calibrated after fusion. This result is important for AffectLab because confidence, not only the top class, is consumed by the pacing policy. Treating the equal-fusion probability as reliable would have contradicted the evaluation evidence.

Fold-specific validation fitting selected text weights between 0.54 and 0.63 and then fitted a final temperature. This reduced ECE to 0.0283, while macro F1 decreased from 0.7883 to 0.7758. Table 5.8 reports ECE together with negative log-likelihood (NLL) and the multiclass Brier score for every major calibrated configuration whose reported test session was excluded from fitting its parameters. Lower values are better for all three probability-quality measures.

Table 5.8: Classification and probability-quality trade-off for independently evaluated fold configurations.

Configuration	Accuracy	Macro F1	Weighted F1	ECE	NLL	Brier
Contextual text	0.7304	0.7332	0.7300	0.0311	0.7061	0.3845
Audio only	0.6279	0.6349	0.6229	0.0468	0.9441	0.5140
Equal-weight fusion	0.7832	0.7883	0.7815	0.1619	0.6843	0.3619
Fold validation-fitted fusion	0.7717	0.7758	0.7707	0.0283	0.6050	0.3273

The proper scores qualify the ECE-only view. Equal fusion was substantially under-confident, yet its NLL and Brier score were both better than those of either unimodal branch, indicating improvement in the full predictive distribution despite the large confidence–accuracy gap. The validation-fitted fusion achieved the best held-out NLL and Brier score as well as the lowest ECE, despite its modest reduction in classification F1. This agreement across the two proper scores supports its use when probabilities are consumed operationally.

Table 5.9 gives the complete pooled class-level results for the acoustic and multimodal configurations. The repeated support values confirm that all three configurations were evaluated on the same 5,531 held-out utterances.

Table 5.9: Per-class precision, recall, F1, and support for the audio and fusion configurations.

Configuration	Class	Precision	Recall	F1	Support
Audio only	Anger	0.6583	0.7806	0.7142	1,103
	Happiness	0.6343	0.5312	0.5782	1,636
	Neutral	0.6214	0.5363	0.5757	1,708
	Sadness	0.5997	0.7629	0.6715	1,084
Equal-weight fusion	Anger	0.8053	0.8513	0.8277	1,103
	Happiness	0.8221	0.8105	0.8163	1,636
	Neutral	0.7412	0.6657	0.7014	1,708
	Sadness	0.7635	0.8579	0.8080	1,084
Fold validation-fitted fusion	Anger	0.8024	0.7879	0.7951	1,103
	Happiness	0.8208	0.8233	0.8221	1,636
	Neutral	0.7096	0.6710	0.6897	1,708
	Sadness	0.7601	0.8358	0.7961	1,084

The pooled values above are computed after concatenating the utterances from all five sessions, so they are not arithmetic means of the fold scores and sessions with more eligible utterances contribute more observations. Table 5.10 complements them with the primary metric for each held-out session and an unweighted descriptive summary across the five folds. Thus, each session contributes equally to the reported mean and population standard deviation (SD); the range gives the observed minimum and maximum rather than an uncertainty interval.

Table 5.10: Macro F1 by held-out IEMOCAP session, with unweighted variation across the five folds.

Configuration	S1	S2	S3	S4	S5	Mean	SD	Range
Utterance-only text	0.6376	0.7125	0.6443	0.6575	0.6625	0.6629	0.0264	0.6376–0.7125
Three-turn context text	0.6918	0.7556	0.7235	0.7310	0.7505	0.7305	0.0227	0.6918–0.7556
Audio only	0.6450	0.6974	0.5870	0.6017	0.6224	0.6307	0.0386	0.5870–0.6974
Equal-weight fusion	0.7517	0.8108	0.7708	0.8090	0.7922	0.7869	0.0227	0.7517–0.8108
Fold validation-fitted fusion	0.7294	0.8070	0.7542	0.7932	0.7852	0.7738	0.0281	0.7294–0.8070

The session-level values preserve the aggregate ordering without implying uniform difficulty. Three-turn context exceeded utterance-only text in all five sessions, and equal-weight fusion achieved the highest macro F1 in every session. Audio varied most across sessions (SD 0.0386; range 0.5870–0.6974), whereas contextual text and equal fusion each had an SD of approximately 0.0227. These are descriptive results for five actor dyads, not estimates of population-level speaker variability; the dialogue-cluster bootstrap intervals remain the inferential comparison reported for RQ1 and RQ2.

Figure 5.1 places the classification results and calibration trade-off on common visual scales. The utterance-only condition is omitted from the calibration panel because no canonical ECE is reported for that configuration.

The row-normalised confusion matrices in Figure 5.2 expose the class-specific complementarity behind the aggregate result. Audio recognised anger more often than contextual text (78.1% versus 69.6% recall), whereas contextual text was substantially stronger for happiness and neutral. Equal fusion raised recall to 85.1% for anger, 81.1% for happiness, 66.6% for neutral, and 85.8% for sadness. Fold-specific validation fitting retained a more balanced pattern while producing probabilities suitable for operational confidence.

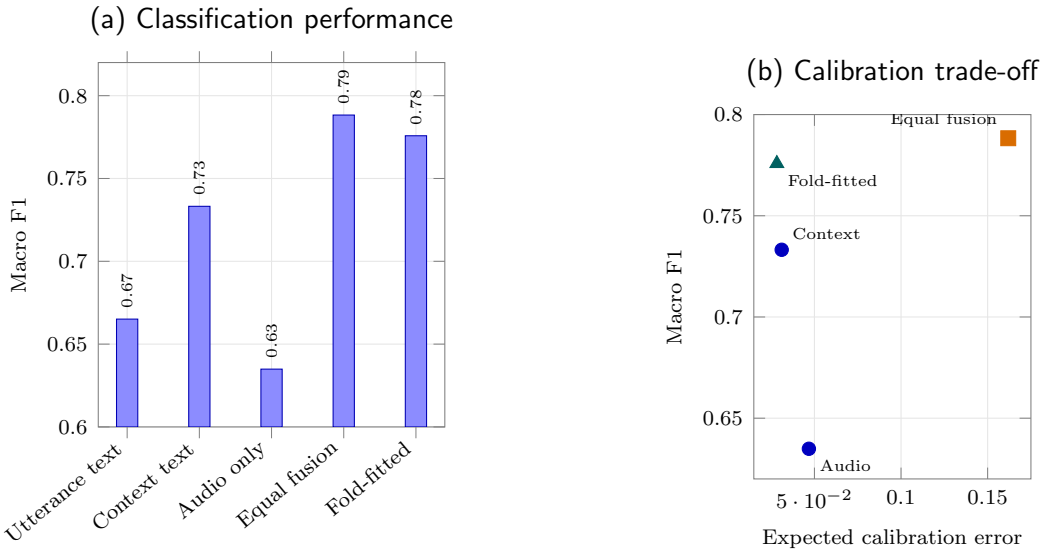


Figure 5.1: Aggregate four-class results from independently evaluated fold configurations. Panel (a) compares macro F1, while Panel (b) shows why the highest-classification equal-weight fusion is not the preferred configuration when confidence is consumed operationally.



Figure 5.2: Pooled row-normalised confusion matrices over the five held-out IEMOCAP sessions ($n = 5{,}531$). Rows are true labels and columns are predicted labels; cell values are percentages of the corresponding true-label row. All panels use the same 0–100% colour scale. Every fitted parameter represented here was estimated without the corresponding test session.

Figure 5.3 shows the corresponding bin-level calibration behaviour. Equal fusion lies systematically above the ideal diagonal through its populated mid-confidence region, indicating under-confidence despite its superior classification performance. Fold-specific validation fitting moves the fusion curve much closer to the diagonal. The isolated low-confidence points contain very few observations and should not be interpreted as stable local estimates.

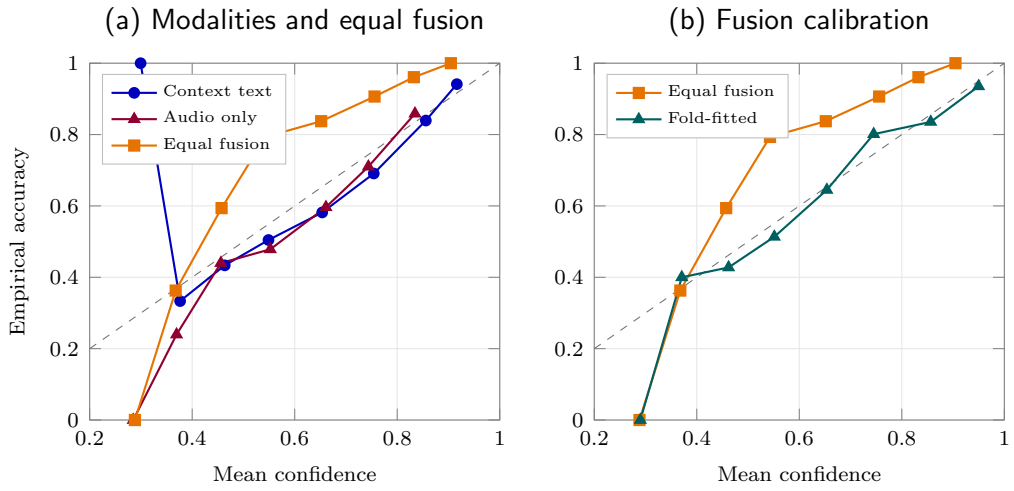


Figure 5.3: Reliability curves from pooled held-out predictions. Each point represents a non-empty equal-width bin of maximum class probability; the dashed diagonal denotes perfect calibration. Panel (a) compares the unimodal systems with predeclared equal fusion, and Panel (b) shows the effect of validation-derived fusion calibration. ECE values in Table 5.8 use the prespecified 15-bin calculation, while the curves use ten bins for legibility.

Operational full-data packaging analysis

The application requires one calibration chain for the final text and audio models rather than five fold-specific chains. For that purpose, the complete global chain fitted text and audio temperatures and then fusion parameters on 5,531 pooled validation-role predictions. It selected text weight 0.59, audio weight 0.41, and final temperature 0.5989.

Table 5.11: Operational global-chain summary; these descriptive values are not an independent held-out performance estimate.

Quantity	**Value**
Accuracy on pooled test-role predictions	0.7736
Macro F1 on pooled test-role predictions	0.7776
Weighted F1 on pooled test-role predictions	0.7728
ECE on pooled test-role predictions	0.0242
Negative log-likelihood	0.6075
Multiclass Brier score	0.3280

These values describe the packaged transformation but are label-reused. Across the rotating folds, the global fit used a validation-role prediction and label for every one of the 5,531 utterance identities; the summary then used a test-role prediction and the same label for every one of those identities, generated by a different trained model. Changing the model that produced the prediction does not make the label independent of calibration fitting. The global row is therefore excluded from Table 5.8 and from the held-out diagnostic figures. The equal-weight result remains the classification headline, while the fold-specific validation-fitted result supplies the held-out calibration evidence.

5.5.5 Comparison with Published IEMOCAP Results

Published IEMOCAP scores are comparable only when the target labels, example selection, speaker separation, test folds, modalities, and metric definitions are aligned. Li et al. provide the closest published reference for the present experiment: their study uses the same four labels, merges excitement with happiness, reports 5,531 utterances, and evaluates audio, text, and audio–text systems by leave-one-session-out five-fold cross-validation [LWC+22]. In their terminology, weighted accuracy (WA) is overall accuracy and unweighted accuracy (UA) is the unweighted mean of the four class recalls. For Table 5.12, the AffectLab UA values were therefore computed from the pooled confusion matrices; UA is not macro F1 and should not be substituted for the dissertation's primary metric.

Table 5.12: Descriptive comparison with protocol-compatible four-class, session-held-out IEMOCAP results reported by Li et al. [LWC+22]. Values are percentages.

System	Modality	Accuracy/WA	UA
AffectLab wav2vec 2.0 baseline (pooled held-out predictions)	Audio	62.79	65.27
Li et al., fine-tuned WavLM+Transformer encoder (five-fold mean)	Audio	67.99	68.24
AffectLab three-turn causal DeBERTa-v3 (pooled held-out predictions)	Text	73.04	73.32
Li et al., fine-tuned BERT+fully connected classifier (five-fold mean)	Text	77.67	78.76
AffectLab predeclared equal-weight late fusion (pooled held-out predictions)	Audio+text	78.32	79.64
Li et al., fine-tuned multimodal Transformer (five-fold mean)	Audio+text	80.36	81.70

The shared mapping and session-held-out test design make this a substantially closer comparison than results obtained from random utterance splits. The same broad pattern appears in both studies: text exceeds audio, and multimodal fusion gives the strongest classification result. AffectLab's equal-weight fusion falls below the published multimodal Transformer on both displayed measures, while remaining in the same performance range. This is benchmark placement, not a controlled head-to-head ranking. Li et al. average the five fold scores, whereas the AffectLab entries concatenate the five non-overlapping test sessions before computing the metrics; exact percentage-point differences between rows should therefore not be interpreted as like-for-like effect estimates. Their development set is a randomly selected speaker from the four non-test sessions, while AffectLab reserves a complete session for validation and trains on the other three. Their text context also includes four future as well as four preceding utterances, whereas AffectLab uses only the previous three turns so that inference is causal. Architectures, audio truncation, optimisation, and random variation differ as well; consequently, the table does not isolate the effect of any one modelling choice and no significance claim is made between studies.

Several commonly cited IEMOCAP results are deliberately excluded. The 84.45% WA and 84.39% UA reported by Li et al. for Session 5 use a single test session rather than the full five-fold evaluation, so direct numerical comparison with the pooled five-session AffectLab result is invalid [LWC+22]. DialogueGCN, COSMIC, and CoMPM retain six separate IEMOCAP labels, including excitement and frustration, and use a fixed training/development/test partition rather than the four-class rotating-session task [GMP+19, GMG+20, LL22]. Because both their

label mapping and their split differ, direct numerical comparison with the four-class results in this dissertation is invalid. The same restriction applies to random utterance-level cross-validation, which can place speakers from one session on both sides of a split. These exclusions prevent superficially similar IEMOCAP percentages from being presented as though they measured the same prediction problem.

5.5.6 Answer to RQ2

RQ2 is also answered positively within IEMOCAP. Equal-weight late fusion improved macro F1 by 0.0551 over contextual text and by 0.1534 over audio alone, showing complementary information in the textual and acoustic branches. The paired interval against contextual text excluded zero.

The answer includes a material trade-off: the strongest classifier was poorly calibrated, whereas fold-specific validation fitting sacrificed approximately one percentage point of macro F1 while reducing ECE substantially on held-out sessions. AffectLab uses the separately reported global chain as an operational packaging choice and retains conservative presentation rules; its label-reused summary is not additional held-out evidence. Calibration under IEMOCAP does not guarantee calibration after distribution shift [OFR+19].

5.6 Application Verification and Readiness

Table 5.13 provides a compact trace from the requirements in Chapter 3 to their implementation and verification evidence. Requirements are grouped only where they share the same implementation boundary, verification method, outcome, and principal limitation. Here, "passed" means conformance under the cited developer-led checks, not universal validation beyond the tested configuration. For application-readiness claims, that configuration is the offline deterministic core: the default suite uses the memory repository, while the full-stack smoke journey adds a compiled frontend and disposable MongoDB instance; both disable live providers and trained inference. It is a verification configuration, not a validated production environment.

5.6.1 Deterministic Dialogue Cases

The versioned engineering evaluator contains five authored trajectories for enhanced workload, boundary, and relationship practice. Each specifies the user turns and expected sequence of controller stages. All five enhanced trajectories matched their expected sequence, as shown in Table 5.14. The fixed controller is included only to expose the change in policy granularity; it stops according to its own earlier completion conditions and is not treated as a human-performance baseline.

These cases show that the enhanced controller requires intermediate evidence rather than completing as soon as one generic success phrase is detected. They do not show that its dialogue is more persuasive, realistic, or educational for a participant. Those properties require human evaluation.

Table 5.13: Compact requirement-to-evidence traceability matrix.

Requirement IDs	**Implementation location**	**Verification method**	**Outcome**	**Principal limitation**
FR1, FR26; NFR5, NFR13	Authentication, persistence, isolation, and deletion (Sections 4.4.3, 4.4.4, and 4.9)	API, security, ownership, stale-write, deletion, and MongoDB recovery tests	Passed	No independent penetration test or deployed backup exercise
FR2, FR3, FR25; NFR3	Onboarding, preferences, takeaways, and action cards (Sections 4.7.2 and 4.6.4)	API and browser onboarding, preference, reflection, and recovery journeys	Passed	No independent usability assessment of these controls
FR4–FR8, FR15, FR19	Preparation, scenario configuration, lifecycle, and dialogue policies (Sections 4.7.3, 4.5.1, and 4.5.2)	Policy/unit, API, desktop/mobile browser, authored-trajectory, and smoke tests	Passed	Three standard scenarios and controlled inputs
FR9, FR11–FR13; NFR2, NFR4	Voice, causal-context inference, fusion, and fallback paths (Sections 4.3, 4.7.5, and 4.9.3)	Context, multimodal, transcription, transient-audio, missing-service, and browser tests	Passed in tested configuration	Limited live-device, provider, and private-model execution
FR10; NFR1	Crisis interruption and safety precedence (Section 4.5.6)	Crisis paraphrase, false-positive, precedence, API, and conversation tests	Passed	A bounded lexical detector cannot cover every expression
FR29	Cognitive heuristics and strategy selection (Subsection 4.4.7)	Authored unit and service cases for pattern, intent, clause extraction, pushback, strategy, and safety precedence	Passed as rule conformance only	No construct, predictive, clinical, comparative-helpfulness, or harm evaluation
FR14; NFR14	Bounded affect pacing and uncertainty presentation (Sections 4.5.4 and 4.7.6)	Seven authored policy cases plus preference, disagreement, failure, and replay tests	Passed	Controlled probabilities; no evidence of user benefit
FR16–FR18, FR20–FR21; NFR6, NFR8	Evidence, response realisation, fallback, and feedback (Sections 4.5.3, 4.5.5, and 4.6.1)	Determinism, evidence-linking, constrained-generation, fallback, feedback, and concurrency tests	Passed	Designed cases do not exhaust language variation
FR22–FR24	Replay, retry, branching, and comparison (Sections 4.6.2 and 4.6.3)	Backend, browser, stale-state, restart-persistence, and full-stack smoke tests	Passed	Synthetic histories rather than natural long-term use
FR27–FR28; NFR7	Consent, study progress, pseudonymous export, and versioning (Section 4.8 and Subsection 4.11.5)	Protocol, eligibility, withdrawal, export-exclusion, dataset-freeze, provenance, and smoke tests	Passed	No independent privacy or reproduction audit
NFR9	Health, bounded failures, recovery, and local responsiveness (Subsections 4.10.4 and 4.11.3)	Recovery journeys, health checks, full-stack restart, and performance regression artefact	Passed locally	No networked concurrent-load or provider-latency test
NFR10	Accessible and responsive interface (Subsections 4.7.2 and 4.11.2)	axe-core scans, keyboard assertions, and desktop/mobile browser runs	Passed tested views	No evaluation with users with disabilities
NFR11–NFR12	Modular and testable architecture (Sections 4.4.1 and 4.11)	Interface inspection, provider substitution, and automated suites across component boundaries	Shown	Maintainability was not measured longitudinally

Table 5.14: Authored enhanced-dialogue trajectory results.

Case	Expected and observed enhanced stages	Final state
Workload agreement	constraints → agree → resolved	Completed: success
Workload non-confirmation	constraints → agree → agree	Active; no false agreement
Boundary maintained	pressure → resolved	Completed: success
Relationship next step	perspective → agree → resolved	Completed: success
Relationship unresolved	perspective → unresolved	Completed: unresolved

5.6.2 Cognitive-Heuristic and Support-Routing Cases

The cognitive and strategy tests are small, implementation-authored examples rather than a labelled evaluation dataset. Table 5.15 states exactly what they establish. The combined sentence deliberately activates several rules at once; the test verifies that the documented precedence and extraction code are stable, not that a human assessor would assign the same psychological labels.

Table 5.15: Implemented cognitive-heuristic and support-routing checks.

Authored input or condition	Verified implementation result	What the check does not establish
"They will definitely think I am a failure because I missed one deadline. What should I do?"	A cognitive-pattern rule matches; practical-help wins the intent precedence; the clause after *because* is copied as possible context	That the speaker has a cognitive distortion, that the copied clause is the true cause, or that practical advice is wanted
"But that won't work. This is pointless."	The explicit pushback vocabulary produces a resistance value above the selector threshold	Psychological resistance, its degree, persistence, or cause
"How do I handle this?" with the default lexical affect state	Practical-help routing makes practical suggestion the highest-scoring strategy	That the suggestion is helpful, well timed, or superior to clarification or reflection
Crisis flag true with high arousal	Safety escalation wins over all ordinary strategy scores	Sensitivity or specificity of crisis detection, or adequacy of the safety response
Two authored emotional-support messages	The analyser, selector, and offline template compose a non-generic emotion-specific response	Accuracy of the affect or intent label, user preference, therapeutic value, or absence of harm

These checks cover selected positive paths and one precedence condition. They do not systematically test all ten pattern families, all combinations and tie cases, negated and quoted pattern language, malformed causal clauses, demographic or dialectal variation, or user correction of a wrong inference. The browser suite verifies rendering of deterministic decision data but does not independently validate the semantics of the labels. Consequently, the appropriate result is "conforms to the tested rules", not an accuracy percentage. The subsystem did not contribute an outcome measure to the pilot, and no claim is made that its labels or selected strategies improved the participant experience.

5.6.3 Evidence Extraction and Affect-Pacing Cases

Twelve human-authored feature judgements were compared with the deterministic extractor. The fixture produced five true positives, four true negatives, one false positive, and two false negatives. Reported speech containing "I need" was incorrectly attributed to the current speaker; an indirect refusal and the synonym "shift" were missed. With only twelve designed cases, these counts are diagnostic examples rather than a held-out accuracy estimate. Importantly, the errors expose the intended limitation of a bounded lexical policy and justify replaying the complete source utterance.

Seven controlled affect cases exercised matching confident estimates, modality disagreement, low confidence, missing audio, an unavailable model, disabled adaptation, and an explicit user choice. All seven produced the declared action. Matching high-confidence sadness allowed a brief acknowledgement; disagreement and low confidence offered a pacing choice; unavailable evidence and disabled adaptation preserved the baseline; and the explicit gentler preference took precedence. Separate tests verify that safety, completion, ineligible sessions, and required study tasks override adaptation.

The deterministic evaluator recorded median in-process service latency of approximately 0.64 ms and a lower-order-statistic 95th-percentile value of approximately 3.46 ms across 19 calls. These figures exclude HTTP, MongoDB, provider generation, and trained inference and are not a load test or production latency claim.

5.6.4 Automated and Full-Stack Verification Results

The final application verification was repeated on 19 October 2026 against commit `2e275767f412ea858d54ac2c507a189815581785`. The worktree was clean when testing began; the subsequent changes in this subsection alter reporting only. The machine ran Windows 11 build 26300 on an AMD Ryzen 5 3500U with eight logical processors and 5.9 GiB of physical memory. The relevant toolchain was Python 3.14.3, pytest 8.4.2, Ruff 0.14.10, Node.js 24.14.1, npm 11.12.1, Playwright 1.63.0, axe-core 4.13.0, and bundled Chromium 153.0.8010.12. No GPU-dependent path was measured. The separately saved full-stack manifest records MongoDB 8.0.1.

The exact final verification commands, issued from the repository root unless the listing changes directory, were:

```
.\.venv\Scripts\python.exe -m pytest backend\tests -rs
.\.venv\Scripts\ruff.exe check backend scripts
Set-Location frontend
npm.cmd run build
npm.cmd run lint
$vite = Start-Process -FilePath node.exe -ArgumentList '.\node_modules\vite\bin\vite.js','--host
    ','127.0.0.1','--port','4173' -WorkingDirectory (Get-Location) -WindowStyle Hidden -PassThru
npx.cmd playwright test replay.spec.ts branching.spec.ts preparation.spec.ts practice.spec.ts
    workspace-recovery.spec.ts --workers=2
Stop-Process -Id $vite.Id
```

Starting Vite separately avoided a Windows teardown wait observed when Playwright owned the web-server process; it did not change the tests or frontend source. The production TypeScript/Vite build, ESLint, and Ruff checks exited successfully. Pytest collected 258 backend cases: 245 passed and 13 were skipped in 88.07

seconds. The focused browser command expanded 38 logical cases across desktop Chromium and the Pixel 7 mobile profile, so 76 browser cases passed in 3.3 minutes. These totals count parameterisations and browser projects as executed test cases; skipped backend cases are not counted as passes.

All 13 skips were the following real-MongoDB node IDs from `backend/tests/test_mongo_integration.py`; each had the explicit reason `Set TEST_MONGODB_URI to run real MongoDB tests`:

```
test_prepared_brief_and_card_survive_restart_without_changing_feedback
test_branch_restart_concurrent_creation_and_parent_deletion
test_data_persists_when_repository_client_restarts
test_ttl_index_and_logical_expiry_do_not_remove_research_record
test_account_deletion_cascades_across_personal_collections
test_concurrent_session_updates_reject_a_lost_update
test_backup_can_be_restored_and_verified
test_password_reset_rejection_preserves_link_and_concurrent_success_is_single_use
test_enrolled_rehearsal_round_trip_and_restart
test_enhanced_workload_state_survives_restart
test_profiled_dialogue_reload_continue_and_rewind[boundary-I cannot take this on.-continuation0-
    boundary_held]
test_profiled_dialogue_reload_continue_and_rewind[relationship-When we spend evenings apart, I feel
     lonely and need time together.-continuation1-next_step]
test_exchange_bound_predictions_survive_restart_without_audio
```

The skips therefore represent one declared environmental boundary rather than failures distributed across unrelated functionality. Real-database behaviour is instead evidenced by the separately recorded smoke journeys and the earlier opt-in MongoDB run; the 19 October default-suite total does not silently treat those 13 cases as successful.

Accessibility assertions used axe-core 4.13.0 through `@axe-core/playwright` 4.13.0. The selected tags were `wcag2a`, `wcag2aa`, `wcag21a`, and `wcag21aa`, corresponding to automated rules mapped to WCAG 2.0 and WCAG 2.1 Levels A and AA. Ten distinct interface states were scanned in both browser projects, giving 20 scan executions: the restored reflection after voice submission; paused enhanced-workload progress; paused boundary-pressure progress; reviewed voice text with pacing; stored prediction provenance; editable preparation brief; saved action card; evidence-linked replay; uncertain multimodal replay; and branch comparison. These states covered `/app`, preparation and active-session variants of `/practice`, Replay, Compare, and Action Card views. Each assertion required an empty axe violation array, and all 20 returned zero violations. This result is limited to the rules, DOM states, fixtures, browsers, and viewports exercised; it is not a WCAG conformance certification and does not replace screen-reader, assistive-technology, or participatory testing.

The saved full-stack smoke manifest was recorded on 4 October 2026 and incorporated with its source-hashed inputs at commit `00a7f54aecefec1d8bb6f53235ddc7ab8f4f0773`. It was generated from the repository root by the exact command `.\.venv\Scripts\python.exescripts\engineering_smoke.py`. The manifest reports two passing browser journeys against a compiled frontend, actual FastAPI process, and disposable MongoDB 8.0.1 database. The journeys included account access, pre-consent exclusion, enrolment, a required task and questionnaire, researcher export, enhanced dialogue, network-failure recovery, replay, branching, comparison, and action-card persistence. The backend was stopped and relaunched twice while MongoDB remained active, and persisted state was verified after each relevant restart. The recorded total run time was 95.72 seconds; this is demonstration timing, not a

capacity benchmark.

Together, these complementary checks support the requirement outcomes summarised in Table 5.13; their principal scope limitations are reported in the same table.

5.6.5 Performance and Responsive-Interface Checks

A reproducible local benchmark exercised the HTTP boundary of the deterministic core using FastAPI's in-process transport and the memory repository. It included routing, bearer-token authentication, request validation, application services, and response serialisation. The evidence was recorded on 11 October 2026, incorporated with its source-hashed inputs at commit `230073c6026f73b24c9162787f3ff493b7ed8a81`, and generated from the repository root by `.\.venv\Scripts\python.exescripts\evaluate_performance.py`. After warm-up, the benchmark collected 200 samples for read-only endpoints and 100 samples for each state-changing workflow operation. Table 5.16 reports the results from the recorded Windows 11 and Python 3.14.3 run.

Table 5.16: Local single-client application latency results.

Operation	**Samples**	**Median (ms)**	**P95 (ms)**	**P95 budget (ms)**
Liveness endpoint	200	1.332	1.866	25
Authenticated scenario catalogue	200	2.942	3.922	50
Session creation	100	2.988	3.961	50
Role-play start	100	3.396	4.437	75
Deterministic conversation turn	100	4.659	5.836	100

Each sample was a sequential, single-client request timed with Python's `time.perf_counter` around FastAPI `TestClient.request`; response status validation remained inside the measured operation. One liveness and one scenario-catalogue request warmed routing, schema, and service paths before collection. The median is Python's conventional sample median. P95 uses the nearest-rank rule: after ascending sort, the observation at one-indexed rank $\lceil 0.95n \rceil$ is selected (rank 190 of 200 or 95 of 100). A check passes when this P95 is less than or equal to its budget. The five budgets in Table 5.16 were fixed in the benchmark script before execution as deliberately generous regression tripwires; they are not production service-level objectives derived from users or deployed traffic.

The benchmark is a fast regression check for local application overhead, not a production-capacity test. It excludes network latency, MongoDB, external generation and transcription providers, trained multimodal inference, and concurrent clients. Those variables depend on the deployed environment and require a separate staged load test. Within its declared scope, however, the result shows that the locally controlled text-first path adds only a few milliseconds per request and that optional services are not on the critical path when deterministic fallbacks are used. All five measurements passed their P95 regression budgets. The JSON evidence artefact records the environment, sample counts, budgets, source hashes, individual scope exclusions, and summary statistics.

Perceived responsiveness also depends on the browser rather than API latency alone. The frontend therefore runs as a compiled production build in both desktop Chromium and a Pixel 7 mobile profile. The browser suite verifies responsive layouts, keyboard interaction, focus behaviour, live status messages, preservation of unsent input after failure, and immediate recovery controls. These checks support the claim that the interface is responsive across the tested viewports; they do not replace field measurements of Core Web Vitals on the eventual hosting infrastructure.

5.6.6 Answer to RQ3

RQ3 asks how uncertain multimodal predictions can be integrated while keeping behaviour explainable, controllable, safe, and failure-tolerant. The implementation and engineering evidence provide a constructive answer: the classifier output is validated and stored as evidence; a deterministic policy maps confidence, agreement, availability, eligibility, and user preference to a small action set; the dialogue controller remains authoritative; and each action and fallback reason is persisted for replay.

The authored cases and automated tests show that low confidence and disagreement produce a user choice, missing inference preserves the baseline, explicit preference takes precedence, and safety bypasses model adaptation. Version-bound requests and optimistic persistence attach a prediction to its exact exchange. This establishes specified system behaviour under the tested conditions. It does not establish that the emotion label itself is correct for a real participant or that pacing improves their outcome.

5.6.7 Answer to RQ4: Integrated Application Quality

RQ4 is answered through the combined evidence traced in Table 5.13. The automated, browser, full-stack, recovery, and authored-policy checks passed for the offline deterministic core defined above, while its locally controlled text-first operations remained below 7 ms at the 95th percentile. These results establish implementation completeness for the tested paths and make the system suitable for controlled deployment only after environment-specific validation.

That validation must exercise the intended MongoDB deployment, trained inference and external providers, real network transit, expected concurrency, backup and recovery, observability, and adversarial and independent security testing. It should establish service-level objectives and close the private model-file provenance gaps before deployment. The present evidence therefore does not claim production latency, internet-scale capacity, or security assurance.

5.6.8 Supplementary User-Pilot Results

Scope of the pilot result. The pilot did not evaluate multimodal affect adaptation. Every required task was ineligible for multimodal inference and affect-aware pacing, so participants completed the baseline text-based rehearsal with deterministic dialogue and feedback. Accordingly, the ratings below describe that baseline

experience only: they cannot establish whether emotion-aware adaptation is acceptable, useful, or beneficial. Evidence concerning the classifier and the implemented adaptation policy instead comes from the IEMOCAP experiments in Section 5.5 and the authored, automated, and full-stack tests reported above; those tests establish benchmark performance and policy conformance within their stated scopes, not human benefit.

Within that boundary, the supplementary single-arm pilot examined completion of the baseline rehearsal workflow and collected brief usability ratings. The analysis uses the AffectLab feasibility protocol v1.1 export, with the first attempt at each required intermediate-difficulty scenario selected for the protocol outcomes. The analytic export includes 15 participants and 46 rehearsal sessions, of which 42 were first required attempts and four were additional or retry sessions. Of the required attempts, 39 reached task completion (14 workload, 13 boundary, and 12 relationship). Twelve participants completed all three required scenarios and submitted the corresponding post-rehearsal questionnaires, giving a protocol-completion rate of 80.0% (12/15; two-sided 95% Wilson interval 54.8%–93.0%). The point estimate met the protocol's 80% feasibility target, although the interval remained wide at this sample size.

Table 5.17 summarises the available study-specific ratings. Feedback usefulness and scenario realism for the non-adaptive required tasks had means of 5.33 and 4.85, respectively, on seven-point scales. These scores concern the deterministic feedback and scenario experience, not multimodal inference or affect-aware pacing. Among 38 observations with both confidence ratings available, representing 14 participants, the mean within-task confidence change was +0.97 points (median +1; participant-cluster bootstrap 95% interval +0.67 to +1.26). The mean paired changes were +0.85 for workload ($n = 13$), +0.85 for boundary ($n = 13$), and +1.25 for relationship ($n = 12$). These within-task differences reflect immediate self-report, not independently verified communication improvement or an effect attributable to the application.

Table 5.17: Supplementary pilot questionnaire results from available first required attempts. Ratings use seven-point scales; paired change is post-confidence minus pre-confidence.

Measure	n	**Mean**	**SD**	**Median**	**Q1–Q3**
Pre-rehearsal confidence	40	3.80	1.04	4.0	3.0–4.0
Pre-rehearsal anxiety	40	4.58	1.28	5.0	4.0–5.25
Post-rehearsal confidence	40	4.75	1.37	4.5	4.0–6.0
Scenario realism	40	4.85	0.98	5.0	4.0–5.25
Feedback usefulness	40	5.33	0.92	5.0	5.0–6.0
Paired confidence change	38	+0.97	0.97	+1.0	0.25–1.75

The deterministic feedback also produced scenario-specific lexical indicators on the 39 completed required tasks. Mean scores included 42.3% for clear requests in workload rehearsal ($n = 14$), 59.0% for clear refusal or maintained boundaries ($n = 13$), and 44.4% for use of an I-statement in relationship rehearsal ($n = 12$). The configured absence-of-blame indicators reached 100% in workload and relationship tasks, and the no-repeated-apology indicator reached 100% in boundary tasks. These are averages of rule-defined scores rather than percentages of participants demonstrating validated skills; the indicators were not independently annotated or

established as measures of real-world communication ability. They provide limited behavioural context to the stronger engineering evidence for the feedback implementation.

Two required attempts lacked a post-questionnaire, and only 38 of the 42 required attempts contained a valid pair of confidence ratings. Consequently, questionnaire summaries use available observations and report their denominators rather than imputing responses. The pilot offers preliminary evidence of workflow uptake and perceived usefulness, but the small convenience sample and uncontrolled design preclude causal, clinical, or generalisable effectiveness claims. It is therefore treated as a supplement to the model and system-verification results, not as the principal basis for deployment readiness.

5.7 Threats to Validity and Limitations

5.7.1 Model and Statistical Validity

The five test folds cover all IEMOCAP sessions, but the corpus contains only ten actors. Fold results are therefore dependent on a small set of speakers, acted or improvised tasks, recording conditions, and annotation practices. The dialogue bootstrap reflects the 151 observed conversations but cannot compensate for missing demographic, cultural, linguistic, and environmental diversity. Only one primary training seed was used after the pipeline check, so optimisation variance is not fully characterised.

Repeated GPU execution can produce small numerical variation. The reported paired comparisons use the latest internally consistent private prediction files, each checked against its neighbouring metrics record. This preserves internal pairing but illustrates why commit, configuration, and file hashes are necessary. The final full-data packages cannot be evaluated on their training rows, and their file-level provenance remains incomplete until the private directories are downloaded and hashed beside the application.

ECE is bin-dependent, and calibration can deteriorate under shift. The global deployment chain is fitted from pooled validation roles across the corpus; it is useful for choosing one full-data configuration but is not an independent external calibration study. The 0.55 and 0.75 presentation bands are operational thresholds rather than validated psychological boundaries.

5.7.2 External and Construct Validity

IEMOCAP labels describe annotated expressive categories, not direct measurements of internal emotion. Mapping excitement to happiness and fear to exploratory anxiety further simplifies the constructs. Model performance must not be interpreted as diagnosis, emotional truth, or equitable performance across accents, languages, devices, ages, disabilities, or cultures.

The role-play features are likewise operational constructs. A phrase detector can identify authored patterns such as a clear refusal or practical request, but it does not measure general communication ability or transfer to a real conversation. The diagnostic fixture already demonstrates missed synonyms, indirect language,

and reported-speech error. Deterministic feedback is more auditable than an unconstrained judgement, but it remains limited by these definitions.

The rule-based cognitive fields have an even narrower evidential basis. Familiar labels such as cognitive distortion, intent, cause, readiness, and resistance may suggest assessment of an internal state, but the implementation observes only the current message and assigns fixed values when short patterns occur. No clinician, validated questionnaire, independent annotator, held-out corpus, or participant self-report was used as a reference standard. The tests reported in Subsection 5.6.2 establish software conformance, not accuracy, construct validity, or clinical validity. Because the pilot neither isolated this subsystem nor measured its helpfulness or harms, the selected support strategy cannot be claimed to be optimal or beneficial. False and overly authoritative labels remain a risk, especially for negation, quotation, indirect language, cultural variation, and messages containing several possible requests.

The engineering evaluation largely uses deterministic templates and test accounts. It does not evaluate live provider variability, private-model latency under concurrent participant use, real microphone noise, or long-duration deployment. The successful smoke journey establishes integration and restart persistence only for its tested path.

5.7.3 Supplementary User-Evidence Limitations

The supplementary pilot was single-arm and included only 15 participants in the analytic export, limiting precision and generalisability. More fundamentally for the present dissertation, required tasks bypassed multimodal inference and affect-aware pacing. The pilot therefore provides no direct user evidence about whether the emotion-aware adaptation was acceptable, accurate, intrusive, helpful, or harmful; its realism and usefulness scores apply only to the baseline rehearsal and deterministic feedback that participants encountered. The wide confidence interval around protocol completion illustrates additional uncertainty. Missing ratings, incomplete rehearsals, fixed scenario order, repeated exposure, self-selection, and demand characteristics may influence the descriptive findings. The study-specific ratings are not validated clinical scales, and the deterministic lexical scores were not validated against independent assessments of communication quality. Without a control condition or follow-up assessment, the observed immediate confidence change cannot establish causation, sustained skill development, or transfer to real conversations. These limitations are why the pilot contributes contextual usability evidence rather than the principal validation of the application or evidence for affect-aware adaptation.

5.8 Chapter Summary

The evaluation provides completed answers to the two machine-learning questions. Three-turn causal context improved the four-class text model from 0.6651 to 0.7332 macro F1, with a paired dialogue-bootstrap interval excluding zero. Equal-weight text–audio fusion further improved macro F1 to 0.7883, demonstrating complementary benchmark information. However, its ECE of 0.1619 exposed a classification–calibration trade-off. Validation-derived calibration reduced ECE substantially at a

modest classification cost, motivating the global text-dominant configuration used for final packaging.

The application evaluation answers RQ3 and RQ4 at the level of specified, reproducible system behaviour. All five authored enhanced dialogue trajectories and all seven affect-pacing decisions matched expectations. Diagnostic evidence cases exposed one false positive and two false negatives rather than concealing lexical limitations. A backend run with 245 passes and 13 explicitly identified environmental skips, 76 passing desktop/mobile browser cases, 20 zero-violation automated accessibility scans, production frontend checks, two full-stack journeys, persistence across backend restarts, and local HTTP timings verify complementary functional and non-functional boundaries. The deterministic conversation path recorded a median of 4.659 ms and a 95th percentile of 5.836 ms in the scoped local benchmark.

The small pilot remains supporting evidence for the non-adaptive workflow: 12 of 15 participants completed the three required rehearsals and questionnaires, while mean scenario-realism and feedback-usefulness ratings were 4.85 and 5.33 out of seven. Because those required rehearsals bypassed multimodal inference and affect-aware pacing, these observations say nothing about the acceptability or benefit of emotion-aware adaptation. The evaluation therefore rests principally on the separate model benchmark and the authored, automated, browser, persistence, recovery, full-stack, accessibility, and performance tests. Overall, AffectLab is a fully implemented communication-practice system whose model behaviour, core policies, interfaces, persistence, accessibility, recovery, and local responsiveness have been tested extensively by the developer and through automation, with human evaluation of adaptation and deployment-scale performance appropriately reserved for future controlled and staging studies.

Chapter 6

Conclusions and Future Work

6.1 Chapter Overview

This dissertation investigated how multimodal emotion recognition can be incorporated into a helpful application for rehearsing difficult interpersonal conversations without allowing uncertain predictions to control consequential decisions. The work combined speaker-independent emotion-recognition experiments, an uncertainty-aware interaction policy, structured role-play scenarios, evidence-grounded feedback, a production-oriented full-stack application, and a layered developer-led verification strategy. AffectLab demonstrates one way of using affective information as bounded conversational evidence rather than as a declaration of a user's internal state.

This final chapter synthesises the findings in relation to the four research questions, identifies the principal contributions, and restates the boundaries of the conclusions. It then proposes a prioritised programme of future work spanning deployment-scale performance, operational governance, model generalisation, policy comparison, feedback validation, accessibility, and longer-term product research. The chapter closes by considering what the implemented and tested application establishes and what remains environment- or study-dependent.

6.2 Summary of the Dissertation

The dissertation began from two connected problems. First, emotion recognition in conversation is uncertain, sensitive to context, and vulnerable to distribution shift. Second, a role-play application needs predictable progression, evidence-grounded feedback, safety precedence, and user control even when machine-learning or generative services are unavailable. Treating these as separate problems would make it easy either to build a strong benchmark classifier with no responsible interaction design or to build an interactive application whose affective behaviour could not be evaluated independently.

AffectLab addresses this tension through separation of responsibilities. Contextual text and speech models estimate an affective probability distribution. A versioned adaptation policy considers confidence, agreement, availability, eligibility, and user preference before selecting a small pacing action. Deterministic scenario controllers remain authoritative for dialogue stage, completion, and unresolved out-

comes. Observable language features determine feedback metrics, while an optional language model is restricted to response realisation. Safety handling bypasses affect adaptation, and missing external services preserve the baseline text workflow.

The implementation supports three fixed scenarios: workload negotiation, boundary-setting, and expression of relationship needs. It also provides preparation, replay, rewind, branching, comparison, takeaways, and action cards so that a rehearsal can be inspected and revisited. Authentication, ownership checks, retention controls, transient voice handling, health checks, production configuration validation, containerised deployment, deterministic fallbacks, backup verification, and optional research-data controls make the application suitable for controlled production use and future study.

Evaluation was deliberately divided by claim. IEMOCAP experiments support classification and calibration findings on the benchmark. Authored trajectories, 245 passing backend tests with 13 identified environmental skips, production frontend checks, 76 passing desktop/mobile browser cases, 20 zero-violation automated accessibility scans, database-backed full-stack journeys, restart recovery, and local HTTP timings support application-quality claims. The scoped performance run recorded a 4.659 ms median and 5.836 ms 95th percentile for deterministic conversation turns. These model and engineering evaluations remain the principal evidence. Section 5.6.8 retains the small pilot only as supplementary feedback on the baseline rehearsal workflow: required tasks bypassed multimodal inference and affect-aware pacing, so the pilot did not evaluate the acceptability or benefit of emotion-aware adaptation.

6.3 Conclusions for the Research Questions

6.3.1 RQ1: The Contribution of Conversational Context

RQ1 asked whether preceding conversational turns improve text-based emotion recognition compared with independent utterance classification. Under the declared four-class, speaker-independent IEMOCAP protocol, the answer is yes. Adding the previous three causal dialogue turns increased accuracy from 0.6624 to 0.7304 and macro F1 from 0.6651 to 0.7332. The paired dialogue-bootstrap macro-F1 difference was +0.0681, with a 95% interval from 0.0504 to 0.0857. Accuracy and weighted-F1 intervals also excluded zero.

The result is strengthened by the controlled comparison: architecture, label set, data partitions, seed, and optimisation settings were held constant, leaving the input context as the intended experimental difference. The exploratory six-class comparison produced the same aggregate direction, although its anxiety category was based on only 40 mapped fear examples and cannot support a stable anxiety-recognition claim.

The conclusion remains task-specific. Context improved annotated emotion classification in IEMOCAP, but the experiment does not establish that the model recovers a person's private emotional state or that the same gain will transfer to spontaneous AffectLab conversations. Per-class results also show why the aggregate conclusion should not be generalised indiscriminately: the anger improvement was small and its interval included zero. The contribution of context is therefore

a benchmark classification finding rather than a claim of general emotional understanding.

6.3.2 RQ2: Multimodal Complementarity and Calibration

RQ2 asked whether contextual text and acoustic information are complementary and what trade-off exists between classification and confidence calibration. The audio-only model achieved macro F1 0.6349, below the contextual text result of 0.7332. Nevertheless, combining the two calibrated probability distributions with the predeclared equal-weight rule increased macro F1 to 0.7883 and accuracy to 0.7832. The macro-F1 improvement over contextual text was +0.0551, with a dialogue-bootstrap 95% interval from 0.0407 to 0.0713; all 10,000 bootstrap replicates favoured fusion. Acoustic information therefore contributed complementary benchmark information even though it was the weaker modality in isolation.

The strongest classifier was not the best calibrated configuration. Equal-weight fusion produced ECE 0.1619, compared with 0.0311 for contextual text. Fold-specific validation-fitted fusion reduced ECE to 0.0283 while retaining macro F1 0.7758 on held-out sessions. A separate global chain with text weight 0.59 was selected for operational packaging. Its pooled test-role summary (macro F1 0.7776 and ECE 0.0242) is not independent performance evidence: every utterance label used in that summary had already entered the global fit through the same utterance's validation-role prediction from another fold.

These results answer both parts of RQ2. Text and speech were complementary for classification, but probability combination sharpened confidence enough to require a separate calibration decision. Consequently, an application should not choose a fusion configuration solely because it maximises test-set F1. The distinction between classification performance and operational confidence is central when probabilities influence thresholds, user-facing uncertainty, or adaptive behaviour.

6.3.3 RQ3: Bounded Integration of Affective Evidence

RQ3 concerned the responsible integration of uncertain multimodal predictions into structured role-play. The project provides a constructive engineering answer. AffectLab stores the complete prediction and provenance, validates it against the current session and turn, and maps it through a deterministic policy to one of a restricted set of pacing actions. A fused or component confidence below the configured low-confidence boundary, or modality disagreement, offers the user a pacing choice; matching anger or sadness estimates for which all three confidences meet that boundary may permit a modest acknowledgement, including in the moderate presentation band; an explicit preference can preserve baseline behaviour; unavailable inference falls back without blocking the conversation; and safety interruption takes precedence over all affective adaptation.

This separation prevents the classifier from deciding whether a scenario succeeds, what feedback score is assigned, whether a participant is eligible, or whether a safety response is required. Optional language generation can phrase a selected action but cannot replace the recorded decision. Optimistic persistence, request identifiers, model versions, source fields, and fallback reasons make the resulting behaviour replayable and auditable.

The engineering evaluation supports this answer within the tested conditions. All five authored enhanced dialogue trajectories and all seven affect-pacing cases matched their expected outcomes. The full automated backend suite passed, and two integrated browser journeys completed against the disposable database while surviving actual backend restarts. These results establish policy conformance, recovery, ownership, persistence, and failure tolerance. They do not establish that an affect prediction was correct for a human user or that adaptation improves a user's rehearsal outcome.

6.3.4 RQ4: Integrated Application Quality and Readiness

RQ4 asked whether the completed application satisfies its declared functional and non-functional requirements under reproducible engineering evaluation. The answer is positive for the tested offline deterministic core. The default suite used the memory repository, and the full-stack smoke configuration added a compiled frontend and disposable MongoDB instance; neither configuration exercised live providers. The backend suite reported 245 passes and 13 identified real-MongoDB skips, the frontend passed strict production compilation and linting, 76 browser cases exercised desktop and mobile layouts, 20 embedded axe scans reported no violations, and two full-stack journeys verified real API serialisation, MongoDB persistence, network-failure recovery, branching, export, and state preservation across backend restarts. Authored cases additionally verified the deterministic dialogue and affect-pacing policies.

Responsiveness was tested separately from correctness. In a single-client, in-process HTTP benchmark, liveness, scenario retrieval, session creation, role-play start, and a deterministic conversation turn all remained below 7 ms at the 95th percentile. The benchmark excludes network transit, MongoDB, trained inference, providers, and concurrent users, so it supports a local-regression claim rather than a deployment-capacity claim. Taken together with fail-fast production configuration, health checks, ownership protection, optimistic concurrency, fallbacks, backup exercises, and containerisation, the evidence establishes an implementation-complete, deployment-oriented system. Suitability for a controlled deployment remains conditional on target-environment load, latency, security, observability, backup-and-recovery, and incident-response validation.

6.4 Principal Contributions

6.4.1 Empirical and Machine-Learning Contributions

The first contribution is a controlled, speaker-independent comparison of utterance-only and causal-context text emotion recognition. By pairing predictions at dialogue level and reporting aggregate, per-class, and exploratory six-class results, the study demonstrates a clear contextual benefit while preserving class-specific uncertainty.

The second contribution is a comparative text–audio pipeline that retains unimodal baselines and distinguishes predeclared fusion from validation-fitted operational configurations. Reporting calibration beside accuracy and F1 exposes a trade-off that would be hidden by classification metrics alone. The final pipeline

also records the temperatures, modality weights, label order, and provenance needed to reproduce the operational transformation from logits to probabilities.

6.4.2 System and Interaction-Design Contributions

The principal system contribution is a hybrid architecture in which trained models and generative services can influence presentation without becoming authoritative for dialogue outcomes, assessment, or safety. The uncertainty-aware affect policy turns confidence, agreement, availability, and preference into inspectable behaviour. Deterministic fallbacks preserve a useful interaction when optional services fail.

The three scenario controllers contribute explicit models of different interpersonal objectives. Workload negotiation seeks a practical prioritisation decision, boundary-setting recognises a respectfully maintained refusal without requiring compliance, and relationship practice seeks a specific need or next step while allowing disagreement to remain unresolved. This avoids reducing every role-play to one generic success condition.

Evidence-grounded feedback and reflective tools form a further applied contribution. Metrics point to specific conversational turns rather than issuing an unexplained global judgement. Replay, rewind, branching, and comparison support counterfactual practice from the same context. Together, these features make a conversation an inspectable practice artefact instead of a transient exchange ending in an opaque score.

6.4.3 Application-Verification and Infrastructure Contributions

The dissertation contributes a layered verification framework that keeps model performance, application conformance, performance scope, and supporting user feedback separate. This prevents benchmark accuracy from being interpreted as product reliability and prevents a small pilot from overshadowing the much broader technical verification. The framework combines deterministic fixtures, automated unit and API suites, real persistence checks, desktop/mobile browser and accessibility tests, production builds, recovery journeys, source-hashed evidence artefacts, and a repeatable latency benchmark.

The full-stack application also provides deployment and research infrastructure. Production configuration fails closed on weak secrets, origins, hosts, HTTPS, and rate limiting; health endpoints distinguish core readiness from optional model availability; Docker packaging, database persistence, backups, ownership enforcement, transient audio, and deterministic fallbacks are implemented rather than described only as future requirements. Separate consent, pseudonymous export, withdrawal, and dataset freezing remain available when the optional research workflow is used.

Finally, the evidence package and versioned configurations improve auditability. Model metrics, paired comparisons, policy fixtures, automated tests, performance summaries, smoke manifests, and protocol files identify which claim is supported by which artefact. Some private model-package hashes remain to be finalised, but the overall approach demonstrates how a production-oriented dissertation application can make its evidence boundaries inspectable.

6.5 Limitations of the Conclusions

6.5.1 Model and Data Limitations

The emotion-recognition conclusions derive from IEMOCAP, a corpus of ten actors completing acted or improvised English interactions. The dataset does not represent the full variation of spontaneous emotion, language, accent, age, disability, culture, device quality, or background noise that AffectLab may encounter. The four-class mapping simplifies expressive categories, and the exploratory mapping from fear to anxiety is especially weak. One primary training seed limits the assessment of optimisation variability.

Speaker-independent session folds reduce identity leakage but do not constitute external validation. Calibration can also deteriorate under domain shift even when it is strong on held-out sessions. The global packaging-chain summary adds a further limitation: its calibration fit and descriptive evaluation reuse all 5,531 utterance identities and labels in different fold roles, so it is not an independent held-out estimate. The final full-data packages likewise have no independent test set because all eligible benchmark examples are used for training after configuration selection. Their operational thresholds must therefore be treated as design parameters, not validated psychological boundaries.

6.5.2 Application and Measurement Limitations

The dialogue policies cover three bounded scenarios and cannot represent the ambiguity, power imbalance, culture, history, or stakes of all interpersonal situations. Feature extraction is transparent but lexically limited: paraphrases can be missed, indirect speech can be misread, and detected phrases do not establish communication competence. Generative responses remain dependent on an external provider when enabled, while template fallbacks trade linguistic variety for predictability.

The separately implemented cognitive-pattern, intent, possible-cause, readiness, and resistance fields are lexical support-routing heuristics, despite terminology that may imply psychological assessment. Their fixed rules have no construct- or clinical-validity evidence, and the present tests establish only selected input–output behaviour. The pilot did not isolate the feature or evaluate whether its labels and strategy choices were helpful, inaccurate, stigmatising, or harmful. The dissertation therefore makes no claim that these fields reveal cognition, identify a true cause or intention, quantify readiness or resistance, or select an optimal intervention.

Engineering evaluation used authored cases, deterministic services, synthetic accounts, and a small number of integrated journeys. The local latency benchmark used an in-process transport and memory repository. These establish behaviour and regression performance on the tested paths but not prolonged concurrent capacity, internet latency, deployed MongoDB latency, trained-inference or provider behaviour, adversarial traffic, diverse microphones, or every assistive technology. The present evidence therefore supports implementation completeness, not production readiness. Even a controlled deployment requires target-environment validation, independent security review, monitoring, incident response, secret management, and jurisdiction-specific governance.

6.5.3 Supporting User-Evidence Limitations

The supplementary pilot included 15 enrolled users, 12 of whom completed all three required tasks. Required tasks bypassed multimodal inference and affect-aware pacing, so the pilot did not evaluate whether the emotion-aware adaptation was acceptable, useful, accurate, intrusive, or beneficial. It also had no randomised control condition and used study-specific ordinal items. Incomplete tasks, missing ratings, demand characteristics, repetition, familiarity, and self-selection may all influence the summaries. The pilot therefore informs refinement of the baseline workflow but does not establish sustained skill acquisition, causal benefit, or transfer to real conversations. AffectLab is not a diagnostic, clinical, crisis, or therapeutic system.

6.6 Future Work

Future work should proceed in an order that strengthens the weakest evidence before increasing system autonomy. Table 6.1 summarises the proposed sequence.

Table 6.1: Prioritised future-work roadmap.

Priority	**Work package**	**Evidence produced**
Immediate	Staging performance, load, and resilience evaluation	Response-time distributions, throughput, saturation points, recovery time, and resource use with production persistence, models, providers, and expected concurrency.
Near term	External and stress-condition model evaluation	Cross-corpus classification, calibration under shift, device and noise robustness, and appropriately powered subgroup analyses.
Near term	Validate dialogue features and feedback with human annotation	Construct validity, inter-rater agreement, paraphrase coverage, error categories, and revised evidence rules.
Near term	Reframe or validate cognitive support-routing labels	Decision on removal or plain-language renaming, expert and user review, false-positive and false-negative cases, and comparison with a neutral routing baseline.
Medium term	Controlled comparison of baseline and affect-aware interaction	Supporting evidence about whether bounded adaptation changes pacing preference, task completion, perceived helpfulness, or intrusiveness.
Longer term	Longitudinal, multilingual, and participatory accessibility work	Retention, transfer, broader inclusion, privacy, and governance evidence.

6.6.1 Validate Deployment Performance and Reliability

The immediate next step is to repeat the local performance check in a staging environment that matches the intended deployment. This should use the production Nginx–FastAPI–MongoDB path, packaged text and audio models, configured external providers, realistic network conditions, and representative session histories.

Warm and cold starts, deterministic fallback turns, multimodal inference, transcription, replay, export, and branch creation should be measured separately so that a slow optional service is not hidden inside an aggregate average.

A concurrent-load tool should report throughput, median and tail latency, error rate, queue time, CPU, memory, and database utilisation as client count increases. Service-level objectives should be declared before the run, and tests should identify the saturation point rather than merely recording one successful request. Soak testing, forced provider timeouts, database restart, backup restoration, and process replacement should additionally measure recovery behaviour and verify that incomplete requests do not create duplicate turns or corrupt session versions.

Browser performance should be measured on the deployed build using Core Web Vitals and interaction traces on desktop and representative mobile hardware. These staging results would complement the current responsive-layout and accessibility suite and turn the existing deployment-oriented architecture into deployment-specific operational evidence.

6.6.2 Broaden Model Validation and Improve Uncertainty Handling

The model pipeline should be tested across corpora and, with explicit consent, on data closer to the intended use context. Cross-corpus evaluation would reveal how much of the IEMOCAP result depends on actors, recording conditions, and label conventions. Stress tests should vary microphones, compression, background noise, transcript errors, utterance length, and missing modalities. Multilingual work should begin with a new construct and annotation review rather than directly transferring the English label mapping.

Repeated training seeds and stronger baselines would better quantify optimisation uncertainty. Future comparisons could investigate longer or learned context selection, speaker-aware aggregation, modality-specific missingness training, and alternative late-fusion rules. These additions should retain paired evaluation and should not tune headline claims on the test partition.

Calibration requires continued attention after deployment. Reliability should be monitored by source, device condition, and data period where lawful and statistically justified. Selective prediction, abstention, or distribution-shift indicators may be more appropriate than forcing every input into an affect label. Thresholds should be revised only with held-out or prospectively collected evidence, and the application should preserve an accessible baseline regardless of model availability.

6.6.3 Evaluate and Refine the Adaptation Policy

The existing policy is intentionally small and deterministic. Future evaluation should ablate its inputs to determine whether confidence, modality agreement, and explicit preference each contribute useful information. Logs can identify how often actions are selected, declined, reversed, or followed by manual pacing changes. Qualitative interviews could reveal whether acknowledgements feel supportive, inaccurate, intrusive, or distracting.

Personalisation should remain transparent and opt-in. A user might prefer fewer affect acknowledgements, text-only operation, slower pacing, or different uncer-

tainty wording. Such preferences can be stored as explicit controls rather than inferred as stable psychological traits. Any learned policy should first be evaluated offline against safety and autonomy constraints and compared with the deterministic baseline. Reinforcement learning based on engagement alone would be inappropriate because longer interaction is not equivalent to better rehearsal.

6.6.4 Validate Feedback and Communication Measures

The deterministic feature rules should be compared with independent human annotations of the same turns. This would quantify precision, recall, agreement, synonym coverage, and systematic error for features such as a clear request, boundary maintenance, non-blaming language, and a specific need. The analysis should include indirect phrasing, negation, reported speech, culturally varied wording, and speech-to-text errors.

The cognitive support-routing component should be treated as a separate decision point rather than silently included in that validation. One option is removal or replacement of psychological terms with directly observable descriptions such as "asks for steps", "contains a causal connector", or "pushes back on a suggestion". If the named constructs are retained, the work would require operational definitions, appropriate domain expertise and ethics review, independently annotated and representative cases, participant correction mechanisms, error and subgroup analysis, and a comparison showing that construct-labelled routing adds benefit over a neutral intent menu or clarification question. Clinical wording or use should remain out of scope unless supported by an appropriately governed clinical study.

Semantic or learned detectors could improve paraphrase coverage, but they should return evidence spans and uncertainty rather than opaque trait scores. Feedback usefulness should also be evaluated separately from detector accuracy: a technically correct observation may still be poorly timed or unhelpful. Co-design with communication educators and target users could refine wording while preserving the distinction between observable behaviour and personal judgement.

6.6.5 Study Transfer, Accessibility, and Long-Term Use

Later studies should examine whether users return voluntarily, whether repeated practice remains useful, and whether any measured change persists. Transfer to real conversations would require independent outcomes, such as user-reported application of a planned request or externally rated role-plays, not merely improvement on the same scenario. Such studies should avoid clinical claims unless they adopt an appropriate clinical design and oversight.

Accessibility evaluation should involve users of keyboards, screen readers, speech input, captions, reduced-motion settings, and different cognitive or communication needs. The current interface and automated checks provide a foundation but cannot replace participatory accessibility testing. Broader language and cultural adaptation should include local review of scenario norms, refusal styles, power relationships, consent materials, and safety resources.

6.6.6 Strengthen Deployment, Privacy, and Governance

Before any controlled deployment, the system should complete staging load tests, an independent security review, production backup-and-recovery exercises, dependency and model monitoring, operational alerting, and documented incident response. Validation should include the intended MongoDB service, trained inference, configured providers, real network transit, expected concurrency, and adversarial traffic. Provider changes should be treated as versioned operational factors because they can alter transcription, generation, latency, or failure modes. Model and policy artefacts should be signed or hashed, and the remaining private-package provenance gaps should be closed.

Data minimisation should continue to guide development. Audio should remain transient unless a separately justified protocol requires retention, and derived research records should avoid reconstructable conversation content. Retention, deletion, export access, and withdrawal should be tested periodically in the deployed environment. Governance should also define who may change scenarios, thresholds, prompts, safety resources, or analysis code and how those changes are reviewed and recorded.

6.7 Final Conclusion

This dissertation shows that multimodal affect recognition can be connected to conversational role-play without making the classifier the authority over the interaction. Causal dialogue context improved textual emotion classification on the chosen benchmark, and audio supplied complementary information that improved fused classification. The accompanying calibration results demonstrated that stronger classification does not automatically produce trustworthy confidence. This finding shaped the application design: uncertainty is retained, adaptation is bounded, users can decline it, and missing inference preserves the core workflow.

AffectLab's broader contribution is therefore practical, architectural, and methodological as much as predictive. Deterministic scenario progression, evidence-grounded feedback, constrained generation, safety precedence, provenance, and complementary verification layers make the system's behaviour more inspectable. The completed engineering evidence establishes that this design works end to end across its tested policies, interfaces, persistence, accessibility, recovery, and local-response paths. It is a helpful communication-practice application, not merely a model demonstration.

The completed system is implementation-complete and suitable for controlled deployment after environment-specific validation. Its present evidence covers production builds and configuration checks, health and recovery paths, security and ownership controls, deterministic fallbacks, broad automated coverage, real-database smoke journeys, desktop/mobile accessibility checks, and local responsiveness for the offline deterministic core. It does not cover live trained inference or providers, real network transit, deployed MongoDB latency, representative concurrency, adversarial testing, or independent security review, and private model-file provenance remains incomplete. Those conditions must be validated in the target environment before a production-readiness claim is made. Likewise, benchmark performance does not establish emotional truth, and the small uncontrolled pilot does

not establish participant benefit. Within those boundaries, the dissertation delivers a thoroughly developer-tested and reproducibly evaluated application for emotion-aware communication rehearsal while preserving uncertainty and user agency.

Bibliography

[BAM19a] Tadas Baltrušaitis, Chaitanya Ahuja, and Louis-Philippe Morency. Multimodal machine learning: A survey and taxonomy. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 41(2):423–443, 2019.

[BAM+19b] Lisa Feldman Barrett, Ralph Adolphs, Stacy Marsella, Aleix M. Martinez, and Seth D. Pollak. Emotional expressions reconsidered: Challenges to inferring emotion from human facial movements. *Psychological Science in the Public Interest*, 20(1):1–68, 2019.

[BBL+08] Carlos Busso, Murtaza Bulut, Chi-Chun Lee, Abe Kazemzadeh, Emily Mower, Samuel Kim, Jeannette N. Chang, Sungbok Lee, and Shrikanth S. Narayanan. IEMOCAP: Interactive emotional dyadic motion capture database. *Language Resources and Evaluation*, 42(4):335–359, December 2008.

[BR09] Dan Bohus and Alexander I. Rudnicky. The RavenClaw dialog management framework: Architecture and systems. *Computer Speech & Language*, 23(3):332–361, 2009.

[BSN+12] Hans Martin Bosse, Jobst-Hendrik Schultz, Martin Nickel, Thomas Lutz, Andreas Möltner, Jana Jünger, Sören Huwendiek, and Christoph Nikendei. The effect of using standardized patients or peer role play on ratings of undergraduate communication training: A randomized controlled trial. *Patient Education and Counseling*, 87(3):300–306, June 2012.

[BZMA20] Alexei Baevski, Yuhao Zhou, Abdelrahman Mohamed, and Michael Auli. wav2vec 2.0: A framework for self-supervised learning of speech representations. In *Advances in Neural Information Processing Systems*, volume 33, 2020.

[CWC+22] Sanyuan Chen, Chengyi Wang, Zhengyang Chen, Yu Wu, Shujie Liu, Zhuo Chen, Jinyu Li, Naoyuki Kanda, Takuya Yoshioka, Xiong Xiao, Jian Wu, Long Zhou, Shuo Ren, Yanmin Qian, Yao Qian, Jian Wu, Michael Zeng, Xiangzhan Yu, and Furu Wei. WavLM: Large-scale self-supervised pre-training for full stack speech processing. *IEEE Journal of Selected Topics in Signal Processing*, 16(6):1505–1518, 2022.

[DCLT19] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language

understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 4171–4186. Association for Computational Linguistics, 2019.

[DFC24] Julian De Freitas and I. Glenn Cohen. The health risks of generative AI-based wellness apps. *Nature Medicine*, 30:1269–1275, 2024.

[EAKK11] Moataz El Ayadi, Mohamed S. Kamel, and Fakhri Karray. Survey on speech emotion recognition: Features, classification schemes, and databases. *Pattern Recognition*, 44(3):572–587, 2011.

[Ekm92] Paul Ekman. An argument for basic emotions. *Cognition and Emotion*, 6(3–4):169–200, 1992.

[ESS+16] Florian Eyben, Klaus R. Scherer, Björn W. Schuller, Johan Sundberg, Elisabeth André, Carlos Busso, Laurence Y. Devillers, Julien Epps, Petri Laukka, Shrikanth S. Narayanan, and Khiet P. Truong. The geneva minimalistic acoustic parameter set (GeMAPS) for voice research and affective computing. *IEEE Transactions on Affective Computing*, 7(2):190–202, 2016.

[Eur16] European Parliament and Council of the European Union. Regulation (eu) 2016/679 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data (general data protection regulation). Official Journal of the European Union, L 119, 2016. Accessed 6 October 2026.

[Eur24] European Parliament and Council of the European Union. Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (Artificial Intelligence Act). Official Journal of the European Union, 2024.

[GMG+20] Deepanway Ghosal, Navonil Majumder, Alexander Gelbukh, Rada Mihalcea, and Soujanya Poria. COSMIC: COmmonSense knowledge for eMotion identification in conversations. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pages 2470–2481. Association for Computational Linguistics, 2020.

[GMP+19] Deepanway Ghosal, Navonil Majumder, Soujanya Poria, Niyati Chhaya, and Alexander Gelbukh. DialogueGCN: A graph convolutional neural network for emotion recognition in conversation. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, pages 154–164. Association for Computational Linguistics, 2019.

[GPSW17] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 1321–1330. PMLR, 2017.

[HBT+21] Wei-Ning Hsu, Benjamin Bolte, Yao-Hung Hubert Tsai, Kushal Lakhotia, Ruslan Salakhutdinov, and Abdelrahman Mohamed. HuBERT: Self-supervised speech representation learning by masked prediction of hidden units. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 29:3451–3460, 2021.

[HGC21] Pengcheng He, Jianfeng Gao, and Weizhu Chen. DeBERTaV3: Improving DeBERTa using ELECTRA-style pre-training with gradient-disentangled embedding sharing. arXiv preprint arXiv:2111.09543, 2021.

[HT07] John Hattie and Helen Timperley. The power of feedback. *Review of Educational Research*, 77(1):81–112, 2007.

[KPNK+19] Meelis Kull, Miquel Perello-Nieto, Markus Kängsepp, Telmo Silva Filho, Hao Song, and Peter Flach. Beyond temperature scaling: Obtaining well-calibrated multi-class probabilities with dirichlet calibration. In *Advances in Neural Information Processing Systems*, volume 32, pages 12316–12326, 2019.

[LJG25] Dongxu Lu, Johan Jeuring, and Albert Gatt. Evaluating LLM-generated versus human-authored responses in role-play dialogues. In *Proceedings of the 18th International Natural Language Generation Conference*, pages 20–40. Association for Computational Linguistics, 2025.

[LKK+20] Juhye Lee, Haerim Kim, Kyoung Hwa Kim, Daesung Jung, Tanisha Jowsey, and Craig S. Webster. Effective virtual patient simulators for medical communication training: A systematic review. *Medical Education*, 54(9):786–795, September 2020.

[LL22] Joosung Lee and Wooin Lee. CoMPM: Context modeling with speaker's pre-trained memory tracking for emotion recognition in conversation. In *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 5669–5679. Association for Computational Linguistics, 2022.

[LOG+19] Yinhan Liu, Myle Ott, Naman Goyal, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, and Veselin Stoyanov. RoBERTa: A robustly optimized BERT pretraining approach. arXiv preprint, arXiv:1907.11692, 2019.

[LSR+24] Inna Lin, Ashish Sharma, Christopher Rytting, Adam Miner, Jina Suh, and Tim Althoff. IMBUE: Improving interpersonal effectiveness through simulation and just-in-time feedback with human-language model interaction. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 810–840, Bangkok, Thailand, August 2024. Association for Computational Linguistics.

[LSS+17] Yanran Li, Hui Su, Xiaoyu Shen, Wenjie Li, Ziqiang Cao, and Shuzi Niu. DailyDialog: A manually labelled multi-turn dialogue dataset. In *Proceedings of the Eighth International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 986–995. Asian Federation of Natural Language Processing, 2017.

[LWC+22] Jinchao Li, Shuai Wang, Yang Chao, Xunying Liu, and Helen Meng. Context-aware multimodal fusion for emotion recognition. In *Proceedings of Interspeech 2022*, pages 2013–2017, Incheon, Republic of Korea, September 2022. International Speech Communication Association.

[MPH+19] Navonil Majumder, Soujanya Poria, Devamanyu Hazarika, Rada Mihalcea, Alexander Gelbukh, and Erik Cambria. DialogueRNN: An attentive RNN for emotion detection in conversations. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 33, pages 6818–6825, 2019.

[OFR+19] Yaniv Ovadia, Emily Fertig, Jie Ren, Zachary Nado, D. Sculley, Sebastian Nowozin, Joshua Dillon, Balaji Lakshminarayanan, and Jasper Snoek. Can you trust your model's uncertainty? evaluating predictive uncertainty under dataset shift. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

[PHM+19] Soujanya Poria, Devamanyu Hazarika, Navonil Majumder, Gautam Naik, Erik Cambria, and Rada Mihalcea. MELD: A multimodal multi-party dataset for emotion recognition in conversations. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 527–536. Association for Computational Linguistics, 2019.

[RSLB19] Hannah Rashkin, Eric Michael Smith, Margaret Li, and Y-Lan Boureau. Towards empathetic open-domain conversation models: A new benchmark and dataset. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 5370–5381. Association for Computational Linguistics, 2019.

[Rus80] James A. Russell. A circumplex model of affect. *Journal of Personality and Social Psychology*, 39(6):1161–1178, 1980.

[Sch05] Klaus R. Scherer. What are emotions? and how can they be measured? *Social Science Information*, 44(4):695–729, 2005.

[TBL+19] Yao-Hung Hubert Tsai, Shaojie Bai, Paul Pu Liang, J. Zico Kolter, Louis-Philippe Morency, and Ruslan Salakhutdinov. Multimodal transformer for unaligned multimodal language sequences. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 6558–6569. Association for Computational Linguistics, 2019.

[TSG25] Hovhannes Tamoyan, Hendrik Schuff, and Iryna Gurevych. LLM role-play: Simulating human-chatbot interaction. In *Proceedings of the*

Third Workshop on Social Influence in Conversations, pages 1–26. Association for Computational Linguistics, 2025.

[ZHZ+18] Hao Zhou, Minlie Huang, Tianyang Zhang, Xiaoyan Zhu, and Bing Liu. Emotional chatting machine: Emotional conversation generation with internal and external memory. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, 2018.

Appendix A

Generative AI Tools Usage

Generative AI tools were used as supplementary resources during the development and documentation of this thesis. Their use was limited to supporting literature exploration, improving the clarity of academic writing, assisting with LaTeX formatting, and providing guidance during technical problem-solving. All generated suggestions were reviewed before being incorporated into the thesis.

An overview of the generative AI tools used is presented in Table A.1.

AI Tool	Use Case	Used in	Remarks
ChatGPT 5.6 Sol / 6 Astra	Literature exploration and identification of relevant scientific publications	Chapter 2	Suggested references were independently verified
	Improvement of academic phrasing, grammar, and text clarity	Chapters 1–6	Suggestions were reviewed and adapted
	Assistance with LaTeX syntax, formatting, and document structure	Entire thesis	Formatting suggestions were manually checked
	Technical guidance and debugging suggestions	Implementation chapters	Suggestions were evaluated and tested before use

Table A.1: Overview of generative AI usage

A.1 ChatGPT Details

ChatGPT (versions 5.6 Sol and 6 Astra, developed by OpenAI) was used as a supplementary assistant throughout selected stages of the thesis. Its contributions are described below.

A.1.1 Literature Exploration

Purpose: To support the identification of relevant scientific publications and research directions related to the topics investigated in this thesis.

Method: ChatGPT was prompted to suggest academic publications, relevant research topics, and potential directions for organizing the state-of-the-art review in Chapter 2.

Verification: Suggested publications were checked against academic sources to verify their existence, relevance, and bibliographic information. The original publications were consulted before being referenced.

Implementation: The verified references and research directions were used as supporting resources when developing the literature review.

A.1.2 Academic Writing Assistance

Purpose: To improve the readability, grammatical correctness, and academic style of selected passages.

Method: Selected paragraphs were provided to ChatGPT with requests to improve sentence structure, clarity, or academic phrasing while preserving their original meaning.

Verification: The suggested revisions were reviewed to ensure that the technical meaning and intended interpretation remained unchanged.

Implementation: Selected linguistic improvements were incorporated into the thesis after manual revision.

A.1.3 LaTeX Formatting Assistance

Purpose: To assist with document formatting and the resolution of LaTeX-related issues.

Method: ChatGPT was consulted for guidance on LaTeX commands, table formatting, document structure, and compilation errors.

Verification: Suggested solutions were checked by compiling the document and inspecting the resulting output.

Implementation: Validated formatting solutions were applied where appropriate.

A.1.4 Technical Guidance and Debugging

Purpose: To obtain supplementary explanations and troubleshooting suggestions during the implementation process.

Method: ChatGPT was occasionally consulted to clarify technical concepts, explain error messages, and suggest possible approaches to resolving implementation difficulties.

Verification: Suggestions were assessed against the project requirements and, where applicable, tested in the development environment.

Implementation: Only solutions considered appropriate and successfully validated were incorporated into the implementation.

Generative AI was used exclusively as a supporting tool. The selection of research methods, system design decisions, implementation validation, interpretation

of experimental results, and final conclusions remained the responsibility of the author.